INDI rate control for multicopters in PX4

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Research question . . . 4
- 1.3 Scope . . . 4
- 1.4 Contributions . . . 4
- 1.5 Outcome in brief . . . 5
- 1.6 Structure of the report . . . 5

2. State of the art **5**
- 2.1 From NDI to INDI . . . 5
- 2.2 INDI for multirotors: the MAVLab line . . . 6
- 2.3 Control effectiveness identification . . . 7
- 2.4 Saturation and allocation . . . 8
- 2.5 Outer-loop INDI, cascades and differential flatness . . . 8
- 2.6 Robustness, stability and failure modes . . . 9
- 2.7 Open-source implementations . . . 10
- 2.8 Gaps addressed by this project . . . 11

3. Theoretical background **12**
- 3.1 Rigid-body rotational dynamics of a multirotor . . . 12
- 3.2 Nonlinear dynamic inversion . . . 13
- 3.3 The incremental form . . . 13
- 3.4 Angular acceleration estimation and filter synchronisation . . . 15
- 3.5 Actuator dynamics and delay . . . 15
- 3.6 Closed-loop behaviour . . . 16
  - 3.6.1 Setup . . . 16
  - 3.6.2 Exact actuator model . . . 17
  - 3.6.3 Actuator-model mismatch . . . 17
- 3.7 Saturation and allocation in torque space . . . 19
- 3.8 Assumptions tested in later chapters . . . 19

4. Analysis of the PX4 multicopter control architecture **20**
- 4.1 Overview and data flow . . . 20
- 4.2 Rate controller . . . 21
- 4.3 Attitude controller . . . 22
- 4.4 Angular-velocity and angular-acceleration estimation and filters . . . 22
- 4.5 Control allocation and saturation . . . 22
- 4.6 Actuator feedback available (real hardware vs SITL) . . . 23
- 4.7 How alternative controllers integrate, and why . . . 24
- 4.8 Implications for INDI . . . 24

5. Design and implementation of the INDI rate controller **25**

5.1 Requirements and scope . . . 25
5.2 Control law . . . 26
5.2.1 Notation . . . 26
5.2.2 Incremental law . . . 26
5.2.3 Relation to an incremental PID . . . 27
5.2.4 Actuator-state estimate and filter synchronisation . . . 27
5.2.5 Yaw: rotor spin-up term and yaw filters . . . 28
5.2.6 Effectiveness G_1 . . . 29
5.3 Architecture choices . . . 29
5.3.1 Torque space ahead of the allocator (D-arch-1) . . . 29
5.3.2 Selection inside `mc_rate_control`, law in a new library (D-arch-2) . . . 29
5.3.3 Actuator state from a first-order torque-space model (D-arch-3) . . . 29
5.3.4 Filter matching by filtering the model, not the measurement (D-arch-4) . . . 30
5.3.5 Saturation, arming and landing (D-arch-6) . . . 30
5.3.6 What INDI replaces and what it does not . . . 30
5.4 Yaw-path refinements, in chronological order . . . 31
5.4.1 D-yawfilt: low-pass on the synchronised yaw pair (session 7) . . . 31
5.4.2 D-yawnu: the yaw rate used in ν is filtered too (session 11) . . . 31
5.4.3 D-yawsp: the setpoint is filtered as well (session 13) . . . 32
5.5 Implementation in PX4 . . . 32
5.5.1 Files . . . 32
5.5.2 Runtime selection and cycle . . . 32
5.5.3 Parameters . . . 33
5.5.4 Numerical edge cases . . . 34
5.5.5 uORB message and logging . . . 34
5.6 Unit tests . . . 35
5.7 Limitations . . . 36
5.8 Discrepancies found . . . 36

6. Simulation environment and its limits **37**
6.1 Toolchain and reproducibility . . . 37
6.1.1 Container . . . 37
6.1.2 One run . . . 37
6.1.3 Seeds . . . 38
6.1.4 Speed factor and the D-speed finding . . . 38
6.2 The gz_x500 vehicle model . . . 38
6.2.1 What the INDI controller depends on . . . 39
6.3 Simulator extensions . . . 39
6.3.1 Gusts and payload (`bench/disturbance.py`) . . . 39
6.3.2 Motor loss . . . 40
6.3.3 Plant variants (`bench/model_variant.py`) . . . 40
6.4 Scenarios and metrics . . . 40
6.5 Fidelity limits and their effect on the conclusions . . . 41

7. Tuning methodology and iteration **42**
7.1 Tuning principles . . . 42
7.2 PID baseline retuning . . . 43
7.3 INDI tuning rounds r1 to r4f . . . 44
How each surprise was diagnosed before gains changed . . . 45
7.4 The hover-noise problem and how it limits J . . . 47
7.5 Frozen candidates and per-scenario ratios . . . 47
7.6 Threats to validity . . . 48
Inconsistencies and unverified claims . . . 49

8. Results **49**

8.1 Experimental protocol . . . 49
8.2 Acceptance verdict . . . 51
8.3 Disturbance rejection . . . 52
8.4 Setpoint tracking . . . 58
8.5 Robustness to actuator-model mismatch (H5) . . . 59
8.6 Double failure (H4) . . . 62
8.7 Control effort and saturation . . . 63
8.8 Summary of all scenarios . . . 64

9 Discussion **68**
9.1 Interpretation . . . 69
9.2 Actuator-model sensitivity . . . 70
9.3 Yaw . . . 72
9.4 Limitations of the evidence . . . 73
9.5 What remains before real flight . . . 75

10 Conclusion **76**
10.1 Answer to the research question . . . 76
10.2 Main quantitative findings . . . 76
10.3 The main weakness and its mechanism . . . 77
10.4 Limits and future work . . . 77

Appendix A Reproducing the results **77**
A.1 Requirements . . . 77
A.2 Build the image and the firmware . . . 78
A.3 Run one scenario for one controller and seed . . . 78
A.4 Run a batch over seeds . . . 78
A.5 Summaries and statistics . . . 79
A.6 Figures . . . 79
A.7 Known gaps . . . 79

Bibliography **80**
Published works . . . 80
Open-source implementations and prior PX4 work . . . 80
Project-internal documents . . . 80

1 Introduction

1.1 Motivation

The multicopter flight controller in PX4 is a cascade. A position loop feeds an attitude loop, and the attitude loop feeds an angular-rate loop that produces the torque commands passed to control allocation. The attitude loop is a proportional controller. The rate loop is a PID controller (chapter 4). This structure is widely used, which is a reason to be careful when proposing to replace any part of it.

The PID rate loop has a known limitation. It uses no model of the vehicle and no measurement of how the vehicle is accelerating. A torque disturbance, such as a shifted centre of mass, a gust acting off the centre of gravity or a change of payload, is seen only after it has produced a rate error, and a constant torque offset is removed only as fast as the integrator accumulates it.

Incremental Nonlinear Dynamic Inversion (INDI) addresses this by using the measured angular acceleration. At each cycle it computes the change in torque needed to move the angular

acceleration from its measured value to the desired one, and adds that change to the torque already being applied. A disturbance that changes the measured acceleration is compensated at the next cycle without being modelled. The model dependence of the controller is reduced to the control effectiveness of the actuators. The price is a dependence on sensors: the method needs a clean angular-acceleration estimate, filters that are synchronised with the signals they act on, and an accurate picture of what the actuators are already doing (chapters 2 and 3). INDI therefore trades model dependence for sensor and actuator-state dependence. Whether that trade is favourable on a PX4 vehicle is an empirical question.

1.2 Research question

The question addressed in this report is the following: can an INDI controller replace the PID angular-rate loop of a PX4 multicopter and do better, as judged by a benchmark whose scenarios, metrics and acceptance criteria were fixed in advance?

The words "fixed in advance" carry weight. The benchmark specification (`bench/scenarios.md`, version 1) was frozen before any INDI code existed. It defines the scenarios, the metrics, the paired-seed protocol and four acceptance criteria that lead to one of three outcomes: INDI outperforms PID, INDI matches PID, or INDI does not outperform PID. Scenarios are split into a tuning set, which both controllers may be tuned on, and a held-out set that was never used for tuning. The verdict is computed on the held-out set only, with the candidates frozen beforehand (decision D-freeze). The purpose of these rules is to prevent the common failure of controller comparisons, in which the new controller is tuned until it wins on the scenarios on which it is then reported.

1.3 Scope

The scope is deliberately narrow.

- Only the inner rate loop is replaced. The attitude and position loops are the standard PX4 controllers in both configurations.
- One airframe is used, the `gz_x500` quadrotor model of PX4 SITL.
- All evidence comes from simulation in Gazebo. No hardware flight was made.
- The comparison is against the PX4 PID rate controller. Other alternatives are not tested.

These limits mean that the results say how INDI and PID compare on one simulated vehicle, under one set of scenarios. They do not say how INDI would behave on hardware. The fidelity limits of the simulator, and what they imply for the conclusions, are treated in chapter 6.

1.4 Contributions

The work has four parts.

1. **An INDI rate controller inside PX4.** The controller is implemented as a library with unit tests and integrated in the multicopter rate-control module. It is selected at arming by the parameter `MC_RATE_CTL_TYPE`, and the PID path is the default and is left untouched (chapter 5).
2. **A reproducible benchmark.** A Docker image builds PX4 and runs Gazebo headless. A runner flies seeded, paired runs from a single command, and computes the metrics (chapters 6 and 8). The acceptance verdict was evaluated from those metrics against the frozen rules (`results/acceptance_s15.md`).
3. **Tuning of both controllers.** The PID baseline was searched for better gains and the search found none that beat the PX4 defaults, so the defaults are the retuned PID baseline. The INDI controller went through several tuning rounds, ending in a frozen candidate (chapter 7).

4. **Results.** The frozen candidates were flown on all six held-out scenarios with 10 paired seeds each, together with the tuning and disturbance scenarios, and the outcome is reported under the pre-declared rules (chapter 8).

1.5 Outcome in brief

Under the rules fixed before INDI flew, INDI does not outperform PID (`results/acceptance_s15.md`). This is stated here at the start because the rest of the report is easy to misread otherwise.

INDI has a clear advantage on disturbance rejection. Attitude RMS is lower than PID on every disturbance scenario, for example by a factor of 0.57 on the diagonal gust (H2) and 0.16 on the centre-of-mass shift (H3) (`results/heldout_s14.md`). Two weaknesses decide the verdict. First, under a mismatch between the actuator time constant assumed by the controller and the real one (scenario H5), INDI develops a 5-7 Hz roll limit cycle, and its roll-rate RMS on the rate-pulse part is 1.58 times that of PID (`results/acceptance_s15.md`). PID tolerates the same mismatch. Second, yaw is worse than PID whenever yaw is commanded. Criterion 3 therefore fails, and so does criterion 1, because both controllers crash on every seed of the double motor failure (H4). The verdict does not depend on H4, since criterion 3 fails on H5 alone.

Section 8.2 gives the full verdict, and chapter 9 discusses how far it generalises. The result is a trade rather than a finding that INDI is worse on average, but it is a loss under the criteria that were declared, and it is reported as one.

1.6 Structure of the report

Chapter 2 reviews the state of the art on INDI for multirotors, identification of control effectiveness, saturation, robustness and open-source implementations, and states the gaps addressed here. Chapter 3 gives the theory: rigid-body dynamics, the incremental form, angular-acceleration estimation, actuator dynamics and closed-loop behaviour. Chapter 4 analyses the PX4 multicopter control architecture and what it implies for an INDI controller. Chapter 5 describes the design and implementation of the controller. Chapter 6 describes the simulation environment, the scenarios and metrics, and the fidelity limits of the simulator. Chapter 7 documents the tuning of both controllers and the threats to validity. Chapter 8 presents the results, starting with the acceptance verdict. Chapter 9 interprets them, discusses the actuator-model sensitivity, yaw and the limits of the evidence. Chapter 10 concludes.

2. State of the art

This chapter reviews the literature and open-source code on incremental nonlinear dynamic inversion (INDI) for multicopter rate control. It is written for a control engineer who knows PID rate control but not INDI. Derivations are in chapter 3 and are not repeated. Every statement is traced to a reading note through the bibliography keys. Papers were read from text extractions, not page images, so equations that the notes flag as reconstructed are avoided or flagged in the text. Topics on the reading list that were not read are named as limitations of the review and not written from memory.

2.1 From NDI to INDI

Nonlinear dynamic inversion (NDI) writes the plant as $\dot{\omega} = f(\omega) + G(\omega)u$ and chooses the input that cancels the model and imposes a desired acceleration ν, leaving a single integrator with designed error dynamics [acquatella2017, Sec. 3.1]. The price is that f and G must be known. For a rotorcraft, f contains aerodynamic moments that depend on airspeed and

attitude, and any model error appears directly as acceleration error. Acquatella et al. call the law "sensitive to model uncertainties" [acquatella2017, Sec. 3.1].

INDI removes f by measuring it. A first-order expansion about the state and input one sample earlier gives a command equal to the previous actuator value plus $G_0^{-1}(\nu - \dot{\omega}_0)$, with $\dot{\omega}_0$ the previous angular acceleration [acquatella2017, Eqs. 14-15]. The approximation is that f does not change over one sample, assumed without a bound on the sample time [acquatella2017, Eq. 12]. Only the control effectiveness remains as model. INDI thus trades model dependence for sensor dependence: it needs an angular acceleration and an actuator state that are consistent in time. Wang et al. call it "sensor-based" and note that it still needs the effectiveness; the phrase "model-independent" does not appear in their paper [wang2019, p. 1118], and this report avoids it.

Time-delay control (TDC) estimates the unmodelled part from its value one delay earlier, and Acquatella et al. show that the TDC and INDI laws are the same expression [acquatella2017, Sec. 3.3]. Sampled at t_s with first-order error dynamics it is an incremental PI with $K = (\bar{g}t_s)^{-1}$, where $\bar{g}$ is the effectiveness gain; second-order error dynamics give an incremental PID [acquatella2017, Eqs. 23-30]. The printed parallel-form constants are inconsistent: differencing Eq. 27 gives an integral increment $T_I^{-1}t_s e(k-1)$, which Eq. 28 prints without t_s, and matching Eq. 24 to Eq. 28 gives $T_I^{-1} = k_P/\bar{g}$ rather than the $T_I = k_P^{-1}$ of Eq. 29. Eq. 29 holds for the form $K\,(e + T_I^{-1}\int e)$, which gives an integral gain $k_P/(\bar{g}t_s)$ (checked against the paper's text for this report). Only the structure is used here. The consequence, drawn in the note and not in the paper, is that with a fixed $\bar{g}$ an INDI rate loop is a fixed-gain PI, and any advantage over PID must come from scheduling $\bar{g}$, from measured cancellation of unmodelled terms, or from actuator-state feedback [acquatella2017, note Sec. 7]. The paper's only evidence is one simulated pitch-rate loop under matched conditions, with no mismatch case and no separately tuned PID [acquatella2017, Sec. 4].

Early INDI work justified dropping the state-dependent term by time-scale separation [smeur2016, Sec. III.A]. Wang et al. call this "not mathematically rigorous" [wang2019, p. 1119], and Steffensen et al. say "the classic INDI derivation contains some inaccuracies" [steffensen2023, p. 6]. Smeur et al. credit the second-order filter used in practice to Bacon and Ostroff [smeur2016, Sec. I]. Sieberling et al.'s robustness analysis and angular-acceleration prediction (2010) was on the reading list but was not read for this report; its results are not used, which is a limitation of this review.

2.2 INDI for multirotors: the MAVLab line

The reference derivation is Smeur, Chu and de Croon [smeur2016]. Angular acceleration is written with a first effectiveness matrix G_1 (thrust and drag moments per squared rotor speed) and a second matrix G_2 for the rotor spin-up reaction [smeur2016, Eqs. 6-10]. The rotor-momentum equations are not reproduced here, because the text extraction garbles them; chapter 3 carries the model, with its equations checked against the PDF pages (`review/ch3_equation_check_s23.md`).

Acceleration estimation and filter synchronisation. The gyro is passed through a second-order low-pass $H(s)$, differentiated, and the same filter is applied to the actuator signal so that the increment is built from equally delayed quantities. The values are $\omega_n = 50$ rad/s and $\zeta = 0.55$ at a 512 Hz loop [smeur2016, Sec. III.B]. With matched filtering, "all that remains is the actuator dynamics" [smeur2016, Sec. III.C]. The flight ablation supports this: without delay compensation the vehicle oscillated severely, the doublet was not flown for safety, and the authors call the compensation "crucial" [smeur2016, Sec. VI.A, Fig. 11]. They reject predictive filtering as needing extra modelling and unable to predict disturbances [smeur2016, Sec. I]. Wang et al. add that "synchronization errors still exist in practice" [wang2019, p. 1123]. Tal and Karaman require "identical LPFs" on acceleration and thrust [tal2021, Sec. III-B], and

Blaha et al. put all incremental-model signals through one 15 Hz Butterworth [blaha2024, Sec. II-A.a].

Actuator dynamics and delay. Smeur et al. assume the same first-order actuator on all motors, using measured rotor speed where available and a model otherwise; the time constant is not given in the text read [smeur2016, Sec. III.B-C]. Their cascaded paper gives $\alpha = 0.1$ at 512 Hz for the Bebop and shows that attitude gains designed without the actuator can be unstable with slower actuators ($\alpha = 0.02$) [smeur2018, Eqs. 11-12]. Steffensen et al. show that with equal actuator bandwidth ω, classical INDI makes the output derivative follow the virtual control through a first-order lag of bandwidth ω [steffensen2023, Eq. 29], and that INDI approximates NDI with actuator dynamics as ω grows without reproducing its designed error dynamics [steffensen2023, Appendix]. This is a SISO fixed-wing simulation, and the paper does not say whether u_0 is the measured actuator state or the previous command [steffensen2023, Sec. V, Eq. 18]. That distinction matters when no rotor-speed measurement exists.

G_1 and G_2. G_2 is non-zero only in yaw. Smeur et al. report its effect as "very significant": yaw-acceleration change matched flight with G_2 and was far off without it, flights without it oscillated or could not take off, and flying without it needed an inflated yaw effectiveness at a "severe performance penalty" [smeur2016, Sec. VI.D]. Dropping gyroscopic coupling and treating the rotor as linear cost about 0.2 % and 0.9 % in a least-squares fit on one dataset [smeur2016, Sec. III.A]. Tal and Karaman use $(\mu; T) = G_1 \varpi^{\circ 2} + G_2 \dot{\varpi}$ [tal2021, Eq. 5]. Practitioners disagree: one forum poster says G_2 "can be ignored first" [impl_px4_prior, Sec. 4B], and in the Paparazzi example G_2 is about 60 times the yaw G_1 [impl_paparazzi, B4]. I take the flight evidence as more reliable and treat G_2 as needed for yaw.

Performance evidence. On a 396 g Bebop, a 30 degree roll doublet reached the reference in 0.2 s with very small overshoot, and a hand-tuned PID was similar; INDI "marginally improves" responsiveness [smeur2016, Sec. VI.A]. For a persistent pitch-moment step, INDI returned to zero in about 0.3 s against about 1.5 s for PID [smeur2016, Sec. VI.B]. This is one airframe, one hand-tuned PID, one doublet per axis, no repetition count and no tabulated metrics [smeur2016, note Sec. 7]. It shows INDI rejecting a step moment faster than a PID without a tuned integral, not a comparison with a retuned PID.

2.3 Control effectiveness identification

Fixed, from bench data. Tal and Karaman use "static tests" and a regression throttle map with an integral correction for battery voltage [tal2021, Sec. III-E, V-A]. The paper does not describe the identification procedure or give the identified values. They show in simulation that the incremental loop tolerates large effectiveness error [tal2021, Eqs. 46-48].

Offline from one flight. Smeur et al. solve for G_1, G_2 by least squares from a logged flight, avoiding inertia and actuator-force knowledge [smeur2018, Sec. 2.2].

Online adaptive. Smeur et al. use an LMS filter on the filtered increments, with a slower estimation filter ($\omega_n = 25$ rad/s) because differentiation amplifies noise [smeur2016, Sec. IV]. A coarse initial G is still needed. With added bumpers, adaptation restored nominal doublet performance, but the evidence is experimental, with no convergence proof and no learning-rate values [smeur2016, Sec. VI.C; note Sec. 8]. Adaptation diverged briefly at take-off and landing [smeur2016, Sec. VI.C]. In the cascaded paper the thrust row is also adapted, and converged values from two initial values agreed with the offline value [smeur2018, Sec. 4.2, Table 1]. Blaha et al. identify 52 parameters (effectiveness matrices plus per-motor speed limit, curvature, idle speed and time constant) by recursive least squares with forgetting, from a throw, in 450 ms; 57 launches all stabilised, with logs for 20 [blaha2024, Sec. II-B, III-A]. Live fits differed from bench values by 10 to 35 %, by about a factor two for yaw effectiveness of two motors, and a fitted curvature above 1 is not discussed [blaha2024, Table I]. The authors

say bench values are “not a ground truth”. The method needs high-rate rotor speed and has no PID baseline [blaha2024, Sec. IV].

Two conclusions follow. Effectiveness need not be accurate, as live recovery worked with fits tens of percent from bench values, though no sensitivity study fixes a tolerance [blaha2024, note Sec. 9]. And the safeguards around adaptation matter as much as the estimator (section 2.7). Acquatella et al. note that a constant $\bar{g}$ loses INDI’s self-scheduling and propose scheduling it online [acquatella2017, Remarks 2, 4].

2.4 Saturation and allocation

Saturation is the least settled topic. Smeur et al. describe no saturation, rate limit or anti-windup handling [smeur2016, note Sec. 6]. In the cascaded paper they state that saturation can destabilise the loop, that pseudo-inverse allocation then realises the objective suboptimally, and that prioritised allocation is work in progress [smeur2018, Sec. 8]. Acquatella et al. do not discuss it [acquatella2017, note Sec. 5]. Wang et al. include actuator limits only in simulation, where high gain “degrades the performance” and can diverge [wang2019, p. 1125].

Pseudo-inverse. Smeur et al. invert a matrix that includes G_2 [smeur2016, Eq. 19; smeur2018, Eq. 7]. Steffensen et al. warn that when the allocation matrix differs from the true input matrix, uncontrolled states can depend on the allocation [steffensen2023, p. 4].

Command-space bounds and reallocation. Tal and Karaman move the yaw moment first, then reduce thrust, then clip [tal2021, Sec. III-E]. Their drag-plate test shows the limit: the yaw moment “causes motor speed saturation and very large momentary yaw tracking error”, with a maximum yaw excursion of 51.7 degrees against 5.3 without the plate [tal2021, Sec. V-D, Table V]. The yaw labels of Tables IV and V differ, so they should not be compared across tables [tal2021, note Sec. (d)].

Actuator-space INDI. Blaha et al. pose the incremental model in actuator space and solve by pseudo-inverse or, if bounds must hold, an active-set method. The paper does not say which ran in flight [blaha2024, Sec. II-A; note Sec. 6]. Paparazzi’s weighted least-squares allocator is described in section 2.7.

No paper treats saturation inside the recursion, where the base of the increment must be the command actually applied. Paparazzi does this implicitly by clipping before the actuator model; I found no source that analyses it.

2.5 Outer-loop INDI, cascades and differential flatness

Smeur et al. extend INDI to linear acceleration [smeur2018]. The outer loop rotates the accelerometer measurement to the inertial frame and computes roll, pitch and thrust increments toward a desired acceleration, with aerodynamic partials set to zero as “the best guess” [smeur2018, Sec. 3, Eqs. 16-19]; a position PD supplies the desired acceleration. A nonlinear thrust-vector increment reduced the maximum vertical-acceleration error in one manoeuvre, though more research is needed [smeur2018, Sec. 4.4, 7]. In a 10 m/s open-jet flow the maximum position deviation was 0.21 m on entry and 0.20 m on exit for INDI against 1.51 m for PID, over 7 repetitions each. Outdoors in a 5.1 m/s mean wind the average maximum error was 0.24 m (12 flights analysed) against 0.85 m (12 flights) [smeur2018, Sec. 5-6]. The note lists cautions: the PID baseline shared the INDI attitude loop and was hand-tuned, outer gains were not matched (“crude” in the authors’ words), INDI has no position integral while the PID does, INDI also used the 512 Hz accelerometer while position arrived at 4 Hz, and the outdoor blocks were not interleaved [smeur2018, Sec. 5; note Sec. (e)]. The gains printed in Sec. 2 ($K_\Omega = 28.0$, $K_\eta = 21.4$) and in Table 2 ($K_\Omega = 10.7$, $K_\eta = 28.0$) conflict, so they cannot both describe the flown design [smeur2018, Sec. 2, Table 2].

Tal and Karaman add differential flatness [tal2021]. Reference jerk and snap give body-rate and angular-acceleration feedforward, INDI laws track linear and angular acceleration, and a nonlinear inversion with closed-loop speed control from optical encoders reaches motor level [tal2021, Sec. II-III]. The position loop is a PD, not INDI, and the authors claim the incremental form removes the need for integral action [tal2021, Sec. III-B]. They show that, with ideal modelling, the angular-acceleration response is set by the motor dynamics, and that disturbance rejection is limited by the filter and motor bandwidth [tal2021, Sec. IV-A]. On a 609 g vehicle at 2 kHz they report 6.6 cm RMS position error at up to 12.9 m/s and 2.1 g; disabling jerk and snap feedforward raised RMS error from 9.0 to 16.8 cm on another trajectory [tal2021, Tables IV-V]. There is no hardware comparison with PID; the only non-incremental baseline is simulated and has no integral action [tal2021, note summary]. The claim that off-the-shelf rate loops cannot use snap is an argument from structure [tal2021, Sec. III-C]. Blaha et al. put an NDI position loop and an attitude loop on their INDI inner loop [blaha2024, Sec. II-A].

Limitation of this review. The UZH comparison of nonlinear MPC and differential-flatness-based control (sun2022) was on the reading list but was not read, so no claim from it is made here.

These designs show what a good inner loop enables, but the effectiveness, filter and actuator issues are decided in the inner loop, and their outer gains are platform-specific.

2.6 Robustness, stability and failure modes

Formal results. Wang et al. [wang2019] drop time-scale separation and treat sampled INDI as NDI with a perturbation $\delta(z;\Delta t)$ from the neglected Taylor terms. If it is bounded and the internal dynamics are input-to-state stable, the state is ultimately bounded by a class-K function of the bound; a local version needs only local exponential stability of the zero dynamics [wang2019, Thms. 1-2, pp. 1119-1120]. Disturbances enter the controlled states only through their per-sample increments, which shrink with the sampling interval, but the internal states see a term that does not shrink [wang2019, Cor. 1, p. 1121]. The NDI mismatch term depends on the whole control and not on Δt, while the INDI term depends on the increment and "can be reduced by increasing the sampling rate" [wang2019, Eq. 62, p. 1122]. The paper rejects an earlier claim that effectiveness mismatch "has no influence", because that derivation assumed $\dot{x} = \dot{x}_0$ [wang2019, pp. 1122-1123].

The limits matter here. Proofs assume ideal actuators and sensing, and the bounds contain unevaluated constants, so the paper gives no numeric sampling or delay bound [wang2019, note Sec. (c)]. Its only statement on lag is that the loop "is able to sustain sufficiently small lags" [wang2019, p. 1123]. Support is one rigid aircraft with gust loads: with effectiveness 30 % too high, INDI RMS errors were at least 3 times smaller than NDI's, and a 1000-sample Monte Carlo gave 31 NDI failures and none for INDI [wang2019, Tables 1-2]. NDI had the other model term exact, and the 31 NDI failure cases were discarded before the box plots and interquartile ranges were computed [wang2019, Sec. IV.D]. There is no multirotor and no flight test. The paper cites 100 Hz as adequate for rigid aircraft and 512 Hz for quadrotors, citations unchecked [wang2019, p. 1121].

The reading note on Acquatella et al. derives, and the paper does not state, that with true effectiveness G and law gain $\bar{g}$ the acceleration error obeys $e_k = (1 - G/\bar{g})e_{k-1}$, stable for $0 < G/\bar{g} < 2$ [acquatella2017, note Sec. 7; note author's derivation]. A fixed-gain PID tuned at $\bar{g}$ should scale the same way. Steffensen et al. add that state-derivative feedback "could influence the stability and robustness" under uncertainty and noise [steffensen2023, p. 13].

Failure modes in the sources. - Filter and delay mismatch: severe oscillation without delay compensation [smeur2016, Fig. 11]. A higher cutoff rejects gusts faster but passes more noise, and INDI was judged noisier than PID in steady flow [smeur2018, Sec. 4.5, 5.1]. - Missing G_2:

yaw oscillation or no take-off [smeur2016, Sec. VI.D]. - Actuator-model error: the effect of a modelled rather than measured actuator state is not analysed in the papers read; Steffensen et al. assume a measured noise-free state and Tal and Karaman use encoders [steffensen2023, Sec. V; tal2021, Sec. III-E]. No source read for this review analyses INDI with a modelled rather than measured actuator state; chapter 9 reports what this project observed on that question (the H5 limit cycle). - Saturation: section 2.4. - Ground contact and engagement: adaptation diverged at take-off and landing [smeur2016, Sec. VI.C]; Paparazzi suppresses the increment on the ground (section 2.7). - Sensor bias: accelerometer bias gives a steady position offset in the outer loop [smeur2018, Sec. 4.4]. - Linearised inversion: yaw oscillation when motor time constant and sample time differ greatly [tal2021, Sec. V-E]. - Single-vehicle evidence: each flight paper uses one airframe, and none compares a retuned PID under fixed metrics.

2.7 Open-source implementations

Paparazzi. The note covers `stabilization_indi_simple.c` and `stabilization_indi.c` at commit c04ffe40, both read in full [impl_paparazzi].

Simple controller. Effectiveness is a scalar per axis in rad/s^2 per raw command unit, with no matrix or allocation; the standard mixer follows [impl_paparazzi, A1]. The rate loop is a P term (default 28) producing an acceleration reference, and the attitude loop divides its gain (default 600) by it, giving a parallel PD with no integral [impl_paparazzi, A1]. Angular acceleration is the backward difference of a second-order Butterworth on the gyro, default 8 Hz (3.2 Hz in the example airframe) [impl_paparazzi, A2]. The actuator is a first-order model per axis command, set by a corner frequency in rad/s and discretised as $1 - e^{-\omega_{act}/f}$. Its state goes through a Butterworth with the gyro cutoff, and that filtered state is the base of the increment [impl_paparazzi, A3]. There is no explicit delay term. For yaw, spin-up enters as a per-sample gain, $\Delta u_r = (\nu_r - \dot{\omega}_f + g_2 \Delta u_{r,prev})/(g_1 + g_2)$ [impl_paparazzi, A4]. LMS adaptation is off by default, runs only in flight, is unnormalised, clamps effectiveness from below at 0.01 and has no excitation check [impl_paparazzi, A5]. Each axis command is bounded (default ± 4500 of 9600) before it feeds the actuator model, which is an implicit anti-windup, but saturation after mixing is invisible [impl_paparazzi, A6]. On the ground at low thrust it outputs the proportional term only, because otherwise commands would "increment to the maximum before even getting in the air" [impl_paparazzi, A7]. The note records no non-finite checks, a fixed assumed 512 Hz period, and filters initialised to zero rather than to the current gyro rate [impl_paparazzi, A7-A8].

Full controller. Effectiveness is a matrix over actuators, solved as an absolute command in a disturbance-estimate form equal to the incremental law when unconstrained [impl_paparazzi, B1]. The default allocator is a weighted least-squares active-set solver limited to 10 iterations, with roll and pitch weighted far above thrust and thrust above yaw, and absolute bounds [impl_paparazzi, B6]. The clipped result feeds the actuator model. Rotor-speed feedback, when available, replaces the model, mapped linearly into command units with no latency compensation [impl_paparazzi, B3]. Adaptation is off by default, in flight only, bounded to between half and twice the initial value, with a TODO for the missing excitation check [impl_paparazzi, B5]. By reading the code, the adaptation scales its input by `ADAPTIVE_MU` but not its measured output, so the prediction is consistent only when `ADAPTIVE_MU` equals $1/$`INDI_G_SCALING` $= 10^{-3}$, the code default; the example airframes set 10^{-4} [impl_paparazzi, B5]. This is a static reading, not tested in simulation or flight.

Prior PX4 work. All evidence is repository and forum text read from local copies [impl_px4_prior]. - PR 25216 (multicopter INDI rate control) was closed by the stale bot, not by a maintainer decision. It added a module using ESC rotor speed (low-passed) as actuator state, in torque space with metric rotor parameters, publishing the PID torque plus an INDI term; its final state was not checked [impl_px4_prior, Sec. 1]. A maintainer

found an increment on top of the rate-controller output “a bit strange”, since the controller is robust because it “uses measurements to correct its commands”. Maintainers asked for a separate module started in place of the rate controller, no changes to the commander or allocator, and objected to duplicated parameters and high-rate logging. No results were reported. - PR 25166 (fixed-wing iNDI) is an outer-loop, acceleration-level design, stalled as a draft with no results [impl_px4_prior, Sec. 2]. PRs 25863 and 25864 closed within minutes and the diff was unavailable, so no conclusion is drawn [impl_px4_prior, Sec. 3]. - Forum. A 2019 thread proposed NDI for multicopters; maintainers advised against an offboard rate loop [impl_px4_prior, Sec. 4A]. In a 2024 thread, ports of the lei-ss code to SITL are reported. One poster’s vehicle took off and shook, with 8 Hz on motor speed against 20 Hz on rates and G_2 ignored, and wrote “If same, the vehicle can not take off” about setting the two cutoffs equal [impl_px4_prior; PX4 forum thread 36293, post of 2024-03-05]. The cause of the shaking is not established in the thread. Another ran INDI in the rate controller at 800 Hz, working in Gazebo and jittering on an F450 [impl_px4_prior, Sec. 4B]. No logs or patches were posted, and a late request asks for exactly that information. - lei-ss/indi (2017) ports the Paparazzi simple controller into a 2016 PX4 attitude controller: scalar per-axis effectiveness, incremental form on the filtered previous command, first-order command model without rotor speed, thrust gating. The README claims only that “flight performance is not bad” on an F450, with no data, and it is incompatible with current PX4 [impl_px4_prior, Sec. 6].

None of this reports a measured result, so it offers design lessons only.

Limitation of this review. indiflight, the Betaflight-derived INDI firmware with rotor-speed telemetry, allocation and onboard identification, was cloned but its code was not read for this report; it is not described here.

2.8 Gaps addressed by this project

These gaps are limited to what the notes cover and may shrink as unread sources are added.

1. **No fair INDI-versus-PID benchmark.** The strongest evidence uses one hand-tuned PID and unmatched outer loops [smeur2016, Sec. VI; smeur2018, Sec. 5]. Tal and Karaman, Blaha et al. and Acquatella et al. have no retuned PID baseline. Because a fixed-gain INDI rate loop is an incremental PI(D), a credible comparison must retune the PID consistently with the same effectiveness, sample time and filters, with repeated runs and fixed metrics. This is the benchmark of chapters 7 and 8.
2. **No integrated INDI rate loop in PX4.** Prior attempts closed on process grounds, sat on top of the PID output, or targeted another vehicle class or an obsolete PX4. The maintainers’ feedback is a specification: an optional, parameter-selected module replacing the standard rate controller, reusing existing allocation geometry, with no commander change. This is the architecture of chapters 4 and 5.
3. **Actuator state without rotor-speed feedback.** The papers use measured rotor speed or assume a measured state, and none analyses a modelled one (section 2.6). A first-order model with a configurable time constant (`MC_INDI_ACT_TC`) is an engineering choice whose consequences must be measured here, not quoted.
4. **Saturation and allocation in the recursion.** The incremental law has not been analysed with PX4’s allocator, and the practical options (per-axis bounds, WLS priorities, yaw reallocation) are uncompared (section 2.4).
5. **Engagement and guards.** The sources agree on ground handling, bumpless engagement, adaptation limits and a matched filter chain, but the implementations leave checks to the user (section 2.7).
6. **A reproducible baseline.** The public attempts report no logs. The simulation measurements of chapters 6 to 8 are meant to supply one; they do not address hardware.

3. Theoretical background

This chapter states the model and the control law on which the rest of the report rests. It uses the notation of section 5.2.1, so that chapter 5 can refer back to it: ω and ω_{sp} are the measured and desired body rates, ν the virtual control (desired angular acceleration), a_f the filtered measured angular acceleration, τ a normalised torque (dimensionless, 1 = the allocator's reference maximum for that axis), τ_f the filtered modelled torque, H the filter chain, G_1 and G_2 the control effectiveness and the yaw spin-up term, K the rate-tracking bandwidth, T_a the actuator time constant (`MC_INDI_ACT_TC`) and T_s the cycle time. All quantities are per body axis. Where a symbol is needed that chapter 5 does not define, it is introduced here.

Equations taken from the literature were read from text extractions of the papers, not from page images. Where an equation used below is marked as reconstructed in the reading notes, it was afterwards checked against the page image of the PDF; the result of each check is in `review/ch3_equation_check_s23.md` and is cited where the equation appears. Statements that the notes do not support are marked `[UNSOURCED]`.

3.1 Rigid-body rotational dynamics of a multirotor

For a rigid vehicle with inertia tensor I_v and body rate Ω, the Euler equation is $I_v\dot{\Omega} = -\Omega \times I_v\Omega + M$ [smeur2016, Eq. 1; acquatella2017, Eq. 1]. In this chapter Ω is written ω when it is the measured rate, as in chapter 5. The moment M has three parts [smeur2016, Eq. 6]:

$$I_v\dot{\omega} = -\omega \times I_v\omega + M_c(\varpi) + M_a(\omega, v) - M_r(\varpi, \dot{\varpi}, \omega),$$

where ϖ_i is the speed of rotor i, M_c the control moment produced by the rotors, M_a the aerodynamic moment (drag, gusts, any offset of the centre of mass) and M_r the moment due to rotor angular momentum. The note on smeur2016 flags the exact form of the rotor-momentum equations (Eqs. 2-5) as partly garbled in the extraction, so only the structure is used here checked against the PDF page, `review/ch3_equation_check_s23.md`.

Control moment. Each rotor produces a thrust $k_1\varpi_i^2$ and a reaction (drag) torque $k_2\varpi_i^2$ about its axis, with the sign of the latter alternating with the spin direction. For an X-quad with half-width b and half-length l, roll and pitch moments come from differences of thrust times a lever arm, and the yaw moment from the signed sum of the drag torques [smeur2016, Eq. 7: $M_c = B\,[\varpi_1^2, \dots, \varpi_4^2]^T$ with B built from bk_1, lk_1 and k_2; the signs of some entries were assumed from the pattern in the note checked against the PDF page, `review/ch3_equation_check_s23.md`]. Tal and Karaman write the same relation as $(\mu; T) = G_1\varpi^{\circ 2} + G_2\dot{\varpi}$ [tal2021, Eq. 5].

Control effectiveness G_1. Dividing by the inertia gives the angular acceleration the actuators can produce per unit of actuator input:

$$\dot{\omega} = F(\omega, v) + G_1\,u_{act} + G_2\,\dot{u}_{act} + \dots, \qquad F = I_v^{-1}(M_a - \omega \times I_v\omega).$$

Smeur et al. take the rotor speed squared as input and obtain $G_1 = 2I_v^{-1}B$ [smeur2016, Eq. 9; read from the extraction, "not stated explicitly" in the note checked against the PDF page, `review/ch3_equation_check_s23.md`]. In this project the input is a normalised torque, so G_1 is a diagonal matrix with units rad/s^2 per unit torque: $G_1 \approx I_v^{-1}M_{max}$ per axis, with M_{max} the physical torque corresponding to $\tau = 1$ [UNSOURCED: this normalisation is a project convention, section 5.2.6]. Because I_v is nearly diagonal for a symmetric airframe, the three axes decouple in G_1. Two further properties matter later. First, G_1 is operating-point dependent: with ϖ_i linear in the command, thrust is quadratic in it, so $\partial T_i/\partial\varpi_i$ grows

with rotor speed. Smeur et al. keep the factor $\mathrm{diag}(\varpi_f)$ in the full law and show that treating the rotor as linear raised a least-squares error by about 0.9 % on one dataset [smeur2016, Sec. III.A]. Second, G_1 depends on inertia, battery voltage and airspeed, which is the motivation for adaptive estimation [smeur2016, Sec. IV; blaha2024, Sec. II-B].

Yaw spin-up term G_2. Yaw torque is rotor drag torque. To change it the rotors must be accelerated, and their inertia I_{rzz} produces a reaction torque on the airframe proportional to the rotor acceleration. Smeur et al. write the corresponding effectiveness $G_2 = I_v^{-1} T_s^{-1} [0; 0; I_{rzz}(1, -1, 1, -1)]$, where T_s is the sample time, so that G_2 multiplied by the change in rotor speed over one sample gives the reaction torque of the rotor acceleration, non-zero only in the yaw row [smeur2016, Eq. 10; checked against the PDF page, `review/ch3_equation_check_s23.md`]. They report that its effect is "very significant" in yaw: the measured yaw-acceleration change matched the prediction with G_2 and was far off without it, and flights without G_2 oscillated or needed inflated yaw effectiveness [smeur2016, Sec. VI.D]. In chapter 5 the term enters as $G_2\,\dot{\tau}$ with units rad/s^2 per unit torque rate, where $\dot{\tau}$ is evaluated as a finite difference over T_s (section 5.2.5). The sign convention is that of section 5.2.5. Gyroscopic coupling of the spinning rotors with the roll and pitch rates, the G_3 term of Smeur et al., was dropped by them at a cost of about 0.2 % in the same least-squares fit [smeur2016, Sec. III.A], and is not modelled here.

3.2 Nonlinear dynamic inversion

Write the rate dynamics in control-affine form, $\dot{\omega} = f(\omega, v) + G(\omega)\, u$ [acquatella2017, Eq. 4; the definitions of f and G are garbled in the extraction and were read as $f = I^{-1}(M_a - \omega \times I\omega)$, $G = I^{-1}(M_c)_u$ checked against the PDF page, `review/ch3_equation_check_s23.md`]. Nonlinear dynamic inversion chooses the input that cancels the model and imposes a desired acceleration ν:

$$u = G^{-1}(\omega)\,\big(\nu - f(\omega, v)\big) \quad \Rightarrow \quad \dot{\omega} = \nu.$$

The virtual control ν is then the output of a linear outer law, for example $\nu = K(\omega_{sp} - \omega)$, which gives the first-order error dynamics $\dot{e} = -Ke$ for a constant setpoint [acquatella2017, Eqs. 6-10].

The strength of the method is that the closed loop is linear and has designed dynamics. Its weakness is where the model enters. The inversion is exact only if f and G are exact. The inertia, the gyroscopic term, drag, rotor effects and any external moment must all be known, and an error in any of them appears directly as an acceleration error. Acquatella et al. describe the law as "sensitive to model uncertainties" [acquatella2017, Sec. 3.1]. For a multirotor this is a practical problem. M_a depends on airspeed and attitude, and its partial derivatives are rarely available. A payload or a battery shift changes I_v and the centre of mass in a way that no fixed model captures.

3.3 The incremental form

Taylor expansion about the current state. Incremental NDI avoids evaluating f. Let $(\omega_0, u_0, \dot{\omega}_0)$ be a recent measurement, with $\dot{\omega}_0 = f(\omega_0, v_0) + G(\omega_0)u_0$. A first-order expansion of the dynamics about it gives

$$\dot{\omega} \;\approx\; \dot{\omega}_0 + \left.\frac{\partial f}{\partial \omega}\right|_0 (\omega - \omega_0) + \left.\frac{\partial f}{\partial v}\right|_0 (v - v_0) + G(\omega_0)\,(u - u_0).$$

Time-scale separation. The method assumes that over one controller step the state moves little, so that the first two correction terms can be dropped, while the input moves fast: the actuator increment $G\,\Delta u$ dominates. Smeur et al. state this condition as validity "only if actuators are fast relative to aerodynamic and precession moments" and neglect the partials with respect to ω and v [smeur2016, Sec. III.A, text after Eq. 13]. Acquatella et al. express it as $\epsilon_{INDI} = f(\omega(t-\lambda)) - f(\omega(t)) \approx 0$ for small delay λ [acquatella2017, Eq. 12], together with fast actuators, $1/\tau_a$ above the closed-loop bandwidth. Smeur et al. 2018 drop the same partials and treat the aerodynamic effect as disturbance because no airspeed model is available [smeur2018, Sec. 2, p. 3].

Wang et al. object that this argument "is not mathematically rigorous and is especially deficient for unstable nonlinear plants" [wang2019, Sec. III.A, p. 1119]. They keep the dropped terms as a perturbation $\delta(z;\Delta t)$ of the closed loop, $y^{\rho} = \nu + \delta(z;\Delta t)$ [wang2019, Eqs. 10-13, reconstructed in the note checked against the PDF page, `review/ch3_equation_check_s23.md`], and prove ultimate boundedness with a bound that shrinks with the sampling interval [wang2019, Theorem 1, p. 1119]. Their proofs assume ideal actuators; actuator dynamics and limits appear only in simulation [wang2019, p. 1119, p. 1124]. For this report the practical content is the same under both views: the incremental form is accurate when the sampling period is short compared with the dynamics that were dropped.

Resulting law. Dropping those terms and requiring $\dot{\omega} = \nu$ gives the increment

$$\Delta u = u - u_0 = G_1^{-1}\,(\nu - \dot{\omega}_0), \qquad u = u_0 + G_1^{-1}(\nu - \dot{\omega}_0),$$

which is the form of [acquatella2017, Eqs. 14-15] and, with the reconstruction caveats of the note, [smeur2016, Eq. 19 and Eq. 21]. This is the law of section 5.2.2 with $\dot{\omega}_0 \to a_f$ and $u_0 \to \tau_f$.

From model dependence to sensor dependence. The vector $f(\omega, v)$ does not appear in the law. Everything that moves the vehicle and that is not an actuator effect (gyroscopic terms, drag, gusts, a centre-of-mass offset) is contained in the measured $\dot{\omega}_0$. Smeur et al. state the consequence: only the control-effectiveness matrices remain as a model [smeur2016, Abstract; Sec. I]. They verified this with a step moment from an added weight, which the controller removed with no steady-state error, against a PID that took about five times longer [smeur2016, Sec. VI.B]. That is one airframe and one disturbance, and is cited here only to show that the mechanism was demonstrated, not that it is general.

The dependence on the model is replaced by a dependence on two signals:

1. a measured angular acceleration $\dot{\omega}_0$, which in a flight controller comes from differentiating the gyro and is therefore noisy and delayed (section 3.4);
2. the actuator state u_0 that corresponds to that measurement. This is not the last command but what the actuators actually produced. Smeur et al. use rotor-speed feedback when it exists and an actuator model otherwise [smeur2016, Sec. III.B, Fig. 4]; Tal and Karaman close a motor-speed loop with optical encoders [tal2021, Sec. I, Sec. III-E]; Blaha et al. use rotor-speed telemetry, or an emulated first-order lag when the thrust coefficient is unknown [blaha2024, Sec. II-A.a, Sec. II-B.c]. Steffensen et al. treat the actuator state as measured and propose no estimator [steffensen2023, Eqs. 1-2, p. 3].

The effectiveness G_1 stays in the law as a gain. Acquatella et al. show that with G_1 replaced by a fixed $\bar{g}$, the law is algebraically an incremental PI(D) with $K = (\bar{g}\,t_s)^{-1}$ and $T_I = k_P^{-1}$ [acquatella2017, Eq. 29; the printed parallel-form equations 26 and 28 are inconsistent with 29, see the note checked against the PDF page, `review/ch3_equation_check_s23.md`]. Their equivalence is an identity between two control laws, not a statement about a mismatched plant [acquatella2017, note section 4].

3.4 Angular acceleration estimation and filter synchronisation

Estimation. Angular acceleration is not measured directly. It is obtained by filtering the gyro and differencing: $\dot{\omega}_f \approx (\omega_f - z^{-1}\omega_f)/T_s$ [smeur2016, Eq. 17]. Smeur et al. use a second-order low-pass $H(s) = \omega_n^2/(s^2 + 2\zeta\omega_n s + \omega_n^2)$ with $\omega_n = 50$ rad/s and $\zeta = 0.55$ [smeur2016, Eq. 15 and text]. Tal and Karaman use a second-order Butterworth at 30 Hz [tal2021, Sec. V-A]; Blaha et al. use one at 15 Hz [blaha2024, Sec. II-A.a]. The filter is needed because differentiation amplifies noise, and it adds delay. In this project the chain is the one of section 5.2.4, built from `IMU_GYRO_CUTOFF` and `IMU_DGYRO_CUTOFF`.

Why the actuator state must pass through the same filter. The Taylor expansion of section 3.3 is valid when the acceleration and the actuator state it is paired with refer to the same instant. After filtering, the measured acceleration is $a_f = H\dot{\omega}$, a delayed and smoothed copy. If u_0 is the unfiltered actuator state, the increment compares an old acceleration with a fresh input. Smeur et al. therefore re-evaluate every operating-point term of the expansion at its filtered value, $(\cdot)_f$, which makes the terms synchronous [smeur2016, Sec. III.B, Eq. 16]. Tal and Karaman require "identical LPFs ... to ensure that equal phase lag is incurred by acceleration and thrust measurements" [tal2021, Sec. III-B, p. 6], and Blaha et al. use one filter on ν_0, u_0 and $\dot{\Omega}_0$ "so Eq. (1) stays valid" [blaha2024, Sec. II-A.a].

The effect is visible in a one-axis calculation. Let the actuator be $A(s)$ with a perfect model, the plant $\dot{\omega} = G_1 A u + d$, and the law $u = \tau_0 + G_1^{-1}(Ke - a_f)$ with $a_f = H\dot{\omega}$. If the model state is filtered, $\tau_0 = HAu$, then $u(1 - HA) = G_1^{-1}(Ke - HG_1Au - Hd)$ and, since HAu appears on both sides,

$$u = G_1^{-1}\,(Ke - Hd). \qquad \text{(synchronised)}$$

If the model state is not filtered, $\tau_0 = Au$, the same algebra gives

$$u\,\big(1 - A(1 - H)\big) = G_1^{-1}\,(Ke - Hd). \qquad \text{(not synchronised)}$$

The extra factor $A(1 - H)$ is the synchronisation error. For $\omega T_H \ll 1$, with T_H the group delay of H, $1 - H \approx j\omega T_H$ grows linearly with frequency, so the unsynchronised loop carries a frequency-dependent distortion of the effective gain and an extra phase lag equivalent to a pure delay T_H. The filter delay then adds to the loop delay that limits the bandwidth. This is my derivation for a first-order picture; the experimental counterpart is that Smeur et al. observed severe oscillation when the filter-delay compensation was removed and did not fly the doublet for safety [smeur2016, Sec. VI.A, Fig. 11]. Steffensen et al. also note the cost of filtering derivative signals, "filtering leads to delay, which will lead to deterioration of the controller performance", and cite filter synchronisation as the INDI remedy [steffensen2023, Discussion, p. 13].

Synchronisation is also imperfect in practice. Wang et al. state that "synchronization errors still exist in practice" even with a shared filter [wang2019, p. 1123]. The remaining sources in this project are listed in section 5.2.4: the sensor filters run at the gyro rate and the controller copy at the controller rate, the backward difference adds half a raw sample, and notch filters are not replicated.

3.5 Actuator dynamics and delay

First-order model. The motor-propeller chain is modelled as a first-order lag in torque, $\dot{\tau}_{act} = (\tau_{cmd} - \tau_{act})/T_a$, so $A(s) = 1/(T_a s + 1)$. This is the model of Smeur et al. [smeur2016,

Sec. III.B; same A_z for all motors, Sec. III.C], of Steffensen et al. ($\dot{u} = \Omega(u_c - u)$ [steffensen2023, Eq. 2]) and of Tal and Karaman ($\dot{\varpi} = \tau_m^{-1}(\varpi_c - \varpi)$ with $\tau_m = 20$ ms in simulation [tal2021, Eq. 32, Sec. IV-A]). Smeur et al. note that the motors of their vehicle differ from each other, contrary to the equal-actuator assumption [smeur2016, Sec. VI.C text on Fig. 20]. Real ESC and motor responses are also not symmetric in rising and falling commands [UNSOURCED; the asymmetry is mentioned only in the session-16 review, which reports separate up and down constants in the Gazebo model].

Discretisation. With the command held constant over one cycle (zero-order hold), the exact solution of the lag over T_s is $\tau_{act}(t + T_s) = e^{-T_s/T_a}\tau_{act}(t) + (1 - e^{-T_s/T_a})\tau_{cmd}$, so

$$\tau_{act}[k] = \tau_{act}[k-1] + \alpha_a(\tau_{cmd}[k-1] - \tau_{act}[k-1]), \qquad \alpha_a = 1 - e^{-T_s/T_a}.$$

The input is the command of the previous cycle, which is the right alignment because the gyro sample at t_k reflects what was commanded one cycle earlier. Smeur et al. 2018 give the discrete actuator as $A(z) = \alpha/(z-(1-\alpha))$ with $\alpha = 0.1$ at 512 Hz [smeur2018, Eq. 11, p. 4]. Read as the same ZOH form, this corresponds to $T_a = -T_s/\ln(1-\alpha) = (1/512)/0.1054 \approx 18.5$ ms (arithmetic done here, not stated in the paper).

Pipeline delay. Beyond the lag there is transport delay: sensor sampling and batching, the half-sample added by the backward difference, publication between modules, and the ESC protocol period. A pure delay e^{-sT_d} multiplies $A(s)$. Smeur et al. compensate delay only through the filtered linearisation, not by prediction, and reject predictive filtering because it needs extra modelling [smeur2016, Sec. I, Sec. III.B]. In the project law an integer delay of d samples can be inserted on τ_{act} before H (section 5.2.2), which can represent whole cycles only.

Inner-loop consequence. With synchronised signals and an exact model, angular acceleration follows ν through the actuator: $\dot{\omega} = A\,\nu$ (section 3.6). Steffensen et al. state the same for INDI with equal bandwidths, $y^{(r+1)} = \omega_{act}(\nu - y^{(r)})$, "$y^{(r)}$ follows ν with the dynamics of the actuator" [steffensen2023, Eq. 29, p. 8], and Smeur et al. that the inner loop response is "simply the actuator dynamics" [smeur2016, Sec. V.A]. The actuator therefore bounds the achievable bandwidth: K must lie below $1/T_a$ for the time-scale separation of section 3.3 to hold. Acquatella et al.'s simulation, with $1/\tau_a = 100$ rad/s and $k_P = 50$ rad/s, has a separation of only a factor 2, which their note flags as weak [acquatella2017, note section 6].

3.6 Closed-loop behaviour

This section derives the closed loop for one axis in the Laplace domain, first with an exact actuator model and then with a mismatched one. The symbol u is the normalised torque command τ_{cmd} of section 5.2.2, without saturation. Time constants of the model and of the real actuator are written T_e and T_t (the session-16 review uses τ_e, τ_t). The model is $A_{est} = 1/(T_e s + 1)$ and the plant $A_{true} = 1/(T_t s + 1)$.

3.6.1 Setup

The plant, with d an unmodelled disturbance acceleration, and the law of section 5.2.2 are

$$\dot{\omega} = G\,A_{true}\,u + d, \qquad a_f = H\dot{\omega}, \qquad \tau_f = H A_{est}\,u, \qquad u = \tau_f + G_1^{-1}\,(Ke - a_f),$$

with $e = \omega_{sp} - \omega$ and G the true effectiveness (equal to G_1 for now). The sensor-side and model-side filters are the same H, which is the synchronisation condition of section 3.4.

3.6.2 Exact actuator model

Take $A_{est} = A_{true} = A$. Substituting,

$$u = HAu + G_1^{-1}(Ke - HG_1Au - Hd) \;\Rightarrow\; u = \frac{K}{G_1}\,e - \frac{1}{G_1}Hd.$$

The command is a proportional law on the rate error, $u = (K/G_1)e$, plus a cancellation of the filtered disturbance. The feedback is taken "around the actuator state": the increment is added to τ_f, not to the previous command.

Two views of the same loop are useful. Writing $\Delta = G_1^{-1}(Ke - a_f)$ for the increment, $u = HA_{est}u + \Delta$ gives

$$u = \frac{1}{1 - HA_{est}}\,\Delta.$$

At low frequency $HA_{est} \to 1$, so $1/(1 - HA_{est})$ has a pole at the origin: the map from the increment to the command is an integrator. The integral action sits in this structure, not in a separate integrator. The session-16 review reaches the same statement [review_s16, section 1]. Substituting u into the plant gives the acceleration and the rate:

$$\dot{\omega} = AKe + (1 - AH)\,d, \qquad \omega = \frac{KA\,\omega_{sp} + (1 - AH)\,d}{s + KA}.$$

With $A = 1$ and $H = 1$ this is $\omega/\omega_{sp} = K/(s + K)$, the first-order response of section 5.2.2. In general the loop transfer function is $L = KA/s$, so the rate loop is an integrator and a first-order lag with gain K and no model of f or d. The disturbance enters through $1 - AH$. Both A and H are unity at DC, so a constant disturbance has no steady-state effect on ω (final value theorem with $1 - AH \to 0$), and no integrator is needed. Above the bandwidth of AH, $1 - AH \to 1$ and the disturbance passes unattenuated: rejection is a high-pass-complement shape set by the actuator and the filters. This is the quantitative version of the statement of Smeur et al. that "disturbances in the angular acceleration are rejected as long as the actuator dynamics and the designed filter are stable" [smeur2016, Sec. III.C]. Their discrete-time analysis (Eq. 27) gives the disturbance term as $1 - A(z)H(z)z^{-1}$; the extra z^{-1} is the one-sample delay of the incremental update and equals 1 at DC, so the steady-state conclusion is the same. The continuous-time form and the $K/(s + K)$ result above are this report's own simplification, not equations of the paper (checked against the PDF page, `review/ch3_equation_check_s23.md`).

Effectiveness error. If $G = gG_1$, the same algebra gives $u\,[1 + (g-1)HA] = G_1^{-1}(Ke - Hd)$. At DC the acceleration is Ke regardless of g; at frequencies where $HA \to 0$ the loop gain is multiplied by g. A discrete analysis in the note on Acquatella et al., made there and not in the paper, gives an acceleration-error recursion $e_k = (1 - G/\bar{g})e_{k-1}$, stable for $0 < G/\bar{g} < 2$ [acquatella2017, note section 7]. An error in G_1 therefore changes the bandwidth, not the DC behaviour.

3.6.3 Actuator-model mismatch

Now $A_{est} \neq A_{true}$. The same substitution gives

$$u = HA_{est}u + G_1^{-1}(Ke - HG_1A_{true}u - Hd) \;\Rightarrow\; u\,(1 - H(A_{est} - A_{true})) = G_1^{-1}(Ke - Hd),$$

that is

$$u = \frac{K}{G_1}\,\frac{e}{1 - H\,\Delta A}, \qquad \Delta A = A_{est} - A_{true}.$$

This agrees with the session-16 review [review_s16, section 5], which wrote it without the disturbance term. The difference of two first-order lags is

$$\Delta A = \frac{1}{T_e s + 1} - \frac{1}{T_t s + 1} = \frac{(T_t - T_e)\,s}{(T_e s + 1)(T_t s + 1)},$$

a band-pass with a zero at the origin and two poles, as in the review. Its magnitude is $|\Delta A(j\omega)| = (T_t - T_e)\,\omega \,/\, \sqrt{(1 + T_e^2\omega^2)(1 + T_t^2\omega^2)}$. Maximising over ω is the same as minimising $(1+T_e^2\omega^2)(1+T_t^2\omega^2)/\omega^2 = \omega^{-2} + T_e^2 + T_t^2 + T_e^2 T_t^2 \omega^2$, whose minimum is at $\omega^{*2} = 1/(T_e T_t)$, where it equals $(T_e + T_t)^2$. Hence

$$f^* = \frac{1}{2\pi\sqrt{T_e T_t}}, \qquad |\Delta A|_{max} = \frac{|T_t - T_e|}{T_t + T_e}.$$

The review's expression for the peak is truncated in the text; the closed form above is my result, and its numeric value for 20/40 ms matches the review's 0.33. At ω^* the two pole phases sum to -90°, which cancels the $+90^\circ$ of the zero, so $\Delta A(j\omega^*)$ is real, with the sign of $T_t - T_e$.

Numbers for $T_e = 20$ ms, $T_t = 40$ ms.

- $\sqrt{T_e T_t} = \sqrt{0.020 \times 0.040} = \sqrt{8 \times 10^{-4}} = 0.02828$ s, so $\omega^* = 35.36$ rad/s and $f^* = 35.36/(2\pi) = 5.63$ Hz.
- $|\Delta A|_{max} = (0.040 - 0.020)/(0.040 + 0.020) = 0.333$, positive (plant slower than model).
- With $H \approx 1$ at f^* (the 30 Hz filters lie well above 5.6 Hz), the factor $1/(1 - H\Delta A) = 1/(1 - 0.333) = 1.50$, a 50 % rise of the controller gain in the band around 5.6 Hz.
- With the phase lag of H at f^* taken as 26° (the value given in the review, not recomputed here): $1 - 0.333\,e^{-j26^\circ} = 0.700 + j0.146$, magnitude 0.716, factor 1.40.
- At the nominal crossover of $L = KA_{true}/s$ for $K = 20$ rad/s and $T_t = 40$ ms, ω_c solves $\omega\sqrt{1 + (0.04\,\omega)^2} = 20$, so $\omega_c \approx 16.6$ rad/s (2.6 Hz; the review gives about 3 Hz). There $|\Delta A| = 0.262$ at $+38^\circ$, and with $H = 1$ the factor is 1.23.

Which side is dangerous. Take $\Delta A(j\omega^*) > 0$ when $T_t > T_e$: the denominator $1 - H\Delta A$ falls below 1 and the gain rises. This is the case of a plant slower than the model, or, equivalently, a model faster than the plant (`MC_INDI_ACT_TC` too small). If $T_t < T_e$ the sign reverses: for $T_e = 40$ ms and $T_t = 20$ ms, $|1 + 0.333\,H| \approx 1.33$ (1.31 with the 26° lag) and the gain falls by a factor of about 0.75. The loop is sluggish but not boosted, so over-estimating `MC_INDI_ACT_TC` is the safe side. The review reaches the same conclusion, noting that a model 1.5 to 3 times slower than the plant gives $\Delta A < 0$, and that a model much faster than the plant moves f^* to a frequency where $|L|$ is already small [review_s16, section 5]. I find no disagreement with the review in sign or in algebra.

What the boost does and does not do. Two limits should be kept apart. First, the map $e \to u$ is itself stable for any pair of lags: $|\Delta A| \le |T_t - T_e|/(T_t + T_e) < 1$ and $|H| \le 1$ for the low-pass chain (and for an added delay), so $|H\Delta A| < 1$ and the small-gain theorem rules out right-half-plane zeros of $1 - H\Delta A$ (my derivation). The limit does not hold once a model-plant delay mismatch is added, because then $|\Delta A|$ can approach 2. Second, the outer

loop sees $L = KA_{true}/[s(1 - H\Delta A)]$ times the delays of section 3.5. The boost multiplies the loop gain in the 3 to 8 Hz band, which is close to the crossover, where the phase margin is already reduced by the real actuator lag, the lag of H and the pipeline delay. The review attributes a roll limit cycle at 5 to 7 Hz to this mechanism [review_s16, section 5, marked there as suspected, not reproduced]. This report treats it as a hypothesis for chapters 7 to 9 to test, not as a result.

3.7 Saturation and allocation in torque space

The derivation above is linear. Three nonlinearities act in the implementation (section 5.2.2): a clamp on the increment, $\Delta\tau = \text{sat}_{\Delta_{max}}(G_1^{-1}(\nu - a_f))$; a clamp on the output, $\tau_{cmd} = \text{sat}_1(\tau_f + \Delta\tau)$; and the allocator, which maps the three torques and the thrust to four motor commands and may not reach all of them. The papers do little here. Smeur et al. describe no saturation handling [smeur2016, Sec. VI notes], Acquatella et al. do not discuss it [acquatella2017, note section 5], Smeur et al. 2018 state that saturation can destabilise the loop and point to prioritised allocation as work in progress [smeur2018, p. 15], and Tal and Karaman reallocate yaw torque first, then reduce thrust, then clip [tal2021, Sec. III-E]. Blaha et al. use an active-set solution when bounds must hold [blaha2024, Sec. II-A].

Torque space has one useful property. The law adds the increment to τ_f, the filtered modelled achieved torque, not to the last command. If the model state is driven by the torque the allocator actually produced, τ_{alloc}, then a command beyond the achievable range does not accumulate in the state, and the loop does not wind up in the sense of an integrator, because the integration of section 3.6.2 acts on achieved torque. This follows from the structure of the law (my reasoning) and holds only as far as the allocator's feedback of achieved torque is current. Chapter 5 gives the clamps, the allocator feedback path and its staleness limit (section 5.3.5; the session-16 review records a case where the feedback is stale on part of the cycles).

3.8 Assumptions tested in later chapters

The derivations above rest on the following assumptions. Each is checked later in the report, in the simulation setup (chapter 6), the tuning (chapter 7), the benchmark (chapter 8) or the discussion (chapter 9).

1. **Time-scale separation.** Actuator and sampling dynamics are fast compared with the neglected terms (section 3.3). The proof of [wang2019] gives boundedness with a sampling-dependent perturbation, but no numeric bound for a multirotor.
2. **Known and constant G_1.** A single hover-point value per axis is used (section 3.1, section 5.2.6). An error scales the high-frequency loop gain (section 3.6.2). Model-mismatch scenarios test it.
3. **First-order, symmetric actuator with known T_a.** Errors in T_a add the band-pass of section 3.6.3, and the direction of the error matters.
4. **Synchronised filters.** The actuator model passes through the same H as the gyro derivative (section 3.4). Residual errors are the rate difference, half a sample of delay and notch filters.
5. **Pipeline delay representable in whole cycles.** The delay parameter covers integer samples only (section 3.5).
6. **Decoupled axes.** G_1 is diagonal and gyroscopic coupling (G_3) is ignored [smeur2016, Sec. III.A].
7. **Yaw spin-up.** For yaw, G_2 is either identified or taken as zero (section 5.2.5). Smeur et al. found it very significant on their vehicle [smeur2016, Sec. VI.D]; this project's default of zero is an assumption to be tested.

8. **Achieved torque is observable.** Saturation handling assumes the allocator reports what it produced (section 3.7).
9. **No advantage without a differing mechanism.** With fixed G_1 the law is an incremental PID [acquatella2017, Eqs. 29-30], so any benefit over a tuned PID must come from the achieved-torque feedback, the measured-acceleration disturbance rejection, or saturation handling (section 5.2.3).

4. Analysis of the PX4 multicopter control architecture

This chapter describes the part of PX4 that the INDI controller replaces and the parts it must live with. It states what the code does, not what the code should do. All references are to PX4 `main` (merge-base 7d362c6f38 of the working branch). Where a line number points to `MulticopterRateControl.cpp` it refers to `main`, because the working branch has INDI additions that shift lines. Facts were checked against the source; the checking log, including corrections to earlier notes, is in `indi_project/plan/px4_analysis_verification.md`. Delay figures are analytic estimates, not measurements.

4.1 Overview and data flow

PX4 controls a multicopter with a cascade of four modules connected by uORB topics. The position loop (not treated here) produces an attitude and thrust setpoint. `mc_att_control` turns the attitude setpoint into a body-rate setpoint. `mc_rate_control` turns the rate setpoint and the measured rate into a normalised torque. `control_allocator` turns torque and thrust into motor commands. The gyro is conditioned upstream by the `vehicle_angular_velocity` publisher in the sensors module.

```
 sensor_gyro(_fifo)                 (raw gyro, kHz)
        |
        v
 sensors/vehicle_angular_velocity   notches -> LPF2p(IMU_GYRO_CUTOFF) -> rate
            |                                        difference of filtered rate -
> AlphaFilter(IMU_DGYRO_CUTOFF) -> accel
      | vehicle_angular_velocity {xyz, xyz_derivative}  published at IMU_GYRO_RATEMAX (400 Hz)
        v
  mc_rate_control    <--  vehicle_rates_setpoint  {roll,pitch,yaw,thrust_body}  <--
mc_att_control <-- vehicle_attitude_setpoint
        |   ^                                                                     ^
        |   +-- control_allocator_status (<= every 5 ms) --+                     +-
- vehicle_attitude
        |                                                  |
        | vehicle_torque_setpoint (normalised xyz)         |
        | vehicle_thrust_setpoint (normalised xyz, pass-through of thrust_body)
        v                                                  |
 control_allocator  ---------------------------------------+
        | actuator_motors (normalised [0,1], NaN = stopped)
        v
 output driver (mixer_module: THR_MDL_FAC inverse, linear map to PWM/DShot/gz velocity) -
> ESCs / motors
```

Three scheduling facts matter for a rate-loop replacement. First, `mc_rate_control` has no timer. It is a work item on the `rate_ctrl` queue triggered by each new `vehicle_angular_velocity` sample (`MulticopterRateControl.cpp:50`, `MulticopterRateControl.hpp:109`, `MulticopterRateControl.cpp:69`), so its rate equals the publication

rate of the filtered gyro. Second, the control allocator is triggered by each new `vehicle_torque_setpoint` and runs on the same queue (`ControlAllocator.cpp:57,96`); thrust is latched and used only when a torque message arrives (`ControlAllocator.cpp:383-402`). A controller must therefore publish torque every cycle. Third, `timestamp_sample` is propagated from the gyro sample through torque (`MulticopterRateControl.cpp:275`) to `actuator_motors` (`ControlAllocator.cpp:753`), so the whole chain can be time-aligned in logs.

4.2 Rate controller

`mc_rate_control` reads the filtered rate ω and angular acceleration $\dot{\omega}_f$ from `vehicle_angular_velocity` (`MulticopterRateControl.cpp:139-140`) and the rate setpoint from `vehicle_rates_setpoint`. A non-finite setpoint axis is replaced by the measured rate (`:185-187`), and setpoint values are held between updates. The time step is Δt from `timestamp_sample`, clamped to $[0.125, 20]$ ms (`:136`). In acro mode the module generates the rate setpoint itself from the sticks (`:159-182`). It runs only while `flag_control_rates_enabled` is set (`:192`), and it resets its integrator and gain compression when disarmed (`:195-197`).

The control law is a parallel PID with a derivative on measurement, in normalised units (`src/lib/rate_control/rate_control.cpp:75-78`). With element-wise products over the three axes:

$$\tau = K_p\,(\omega_{sp} - \omega) + I - K_d\,\dot{\omega}_f + K_{ff}\,\omega_{sp}$$

The "D term" is thus $-K_d\dot{\omega}_f$ with $\dot{\omega}_f$ taken straight from the sensor module; there is no setpoint kick and no filtering inside the controller. The defaults are $K_p = 0.15$, $K_i = 0.2$, $K_d = 0.003$ for roll and pitch (yaw 0.2, 0.1, 0), and $K_{ff} = 0$; they are dimensionless torque per rad/s and per rad/s^2, not SI.

The integrator is updated after the torque is computed, so it uses the previous state, and only when the vehicle is not landed (`rate_control.cpp:81-83`). Anti-windup has two parts. Each axis is clamped by conditional integration: if the allocator flagged positive saturation the error is limited to $\min(e, 0)$, and for negative saturation to $\max(e, 0)$ (`rate_control.cpp:92-99`). The error is also attenuated by $i = \max(0,\, 1 - (e/\mathrm{rad}(400^\circ))^2)$ (`:107-108`), and the state is advanced by forward Euler and clamped to the limit $\pm L_i$ (`:111-115`):

$$I_{k+1} = \mathrm{sat}_{L_i}\!\left(I_k + i\,K_i\,e_k\,\Delta t\right)$$

The saturation flags are derived in the module from `control_allocator_status`: when `torque_setpoint_achieved` is false, the sign of `unallocated_torque[i]` beyond `FLT_EPSILON` sets the positive or negative flag (`MulticopterRateControl.cpp:208-224`). A TODO states that the unallocated value itself should be used (`:223`). Flags persist between status messages.

After the law, the module applies optional gain compression (`MC_GC_EN`, off by default; `:232-233`) and always a low-pass on the yaw torque at `MC_YAW_TQ_CUTOFF` (default 2 Hz; `:236`). Non-finite torque components are set to zero (`:249-251`), optional battery scaling multiplies thrust and torque and clips to $[-1, 1]$ (`:259-266`), and the result is published (`:271-277`). Thrust is not computed here; it is the `thrust_body` field of the rate setpoint, passed through unchanged (`:188,248`). For VTOL the module is started with an argument that selects the `_virtual_mc` topic variants (`:51-52`). The only unit test of the library is a zero-input case (`rate_control_test.cpp:39-44`).

4.3 Attitude controller

mc_att_control is triggered by vehicle_attitude (mc_att_control_main.cpp:83,254) and publishes vehicle_rates_setpoint together with the body thrust vector (:366,382-389). Its core, AttitudeControl::update, is a quaternion proportional law with a reduced-attitude yaw weighting (AttitudeControl.cpp:153-179). The error is $e_q = 2\,\mathrm{imag}(q^{-1}q_d)$, the rate setpoint is $\omega_{sp} = K_q\, e_q$ with per-axis gains (:182-190), and the result is limited per axis (:211-213). On this main the desired attitude first passes a second-order critically damped reference model, and the model's error-driven rate, scaled by a feed-forward gain and limit, is added to the setpoint (:94-137,197-208). The rate setpoint seen by the rate loop is therefore a smooth, limited signal, but it is a rate, not an angular acceleration: no acceleration setpoint exists anywhere upstream.

4.4 Angular-velocity and angular-acceleration estimation and filters

VehicleAngularVelocity is triggered by sensor_gyro_fifo or sensor_gyro and filters every raw sample at the raw gyro rate, which can be several kHz. Publication is throttled by IMU_GYRO_RATEMAX (default 400 Hz, range 100-2000, imu_gyro_parameters.yaml:63-81; gate at VehicleAngularVelocity.cpp:930). The published values are those of the last sample of the processed batch, with no averaging (:772,855-856). Consequently the rate loop runs at about 400 Hz while the filters see the full-rate signal.

The rate chain, applied in place in this order, is: dynamic notch from ESC rpm (IMU_GYRO_DNF_EN, default 0), dynamic notch from FFT peaks, static notches (IMU_GYRO_NF*, off by default), then a second-order Butterworth low-pass LowPassFilter2p at IMU_GYRO_CUTOFF, default 40 Hz (:733-773). The published rate is the calibrated filter output minus the in-run gyro bias (:937).

The acceleration estimate is derived from the output of that same chain. For each raw sample it forms a backward difference of the already filtered rate, $a_n = (\omega_{f,n} - \omega_{f,n-1})/T_s$ (:781, with $1/T_s$ from the FIFO sample time at :836), and feeds it to a first-order AlphaFilter at IMU_DGYRO_CUTOFF, default 20 Hz (:782, VehicleAngularVelocity.hpp:177). The published xyz_derivative is rotated but not bias-corrected (:941). The parameter documentation calls this filter a second-order Butterworth (imu_gyro_parameters.yaml:84-92), but the code is a first-order exponential filter, $y \leftarrow y + \alpha(x - y)$ with $\alpha = \Delta t/(\tau + \Delta t)$ and $\tau = 1/2\pi f_c$ (AlphaFilter.hpp:90,132-135). Setting the cutoff to 0 disables it (:183-189).

At low frequency the group delay of the rate path is about $\sqrt{2}/(2\pi f_c) \approx 5.6$ ms at 40 Hz. The acceleration path adds the alpha filter with $\tau \approx 8.0$ ms at 20 Hz, so $\dot{\omega}_f$ lags ω by roughly 8 ms and the actuator command by more. Both are estimates; the chapter on identification will measure them. Two sharing effects are worth noting. IMU_GYRO_CUTOFF affects both signals, so changing it also changes the legacy D term (imu_gyro_parameters.yaml:52). And the filter state is re-seeded whenever a cutoff or the sample rate changes (VehicleAngularVelocity.cpp:163-195), which causes a short glitch in the acceleration.

4.5 Control allocation and saturation

The allocator forms a six-element control vector $c = [M_x\ M_y\ M_z\ F_x\ F_y\ F_z]^\top$ from the torque and thrust setpoints (ControlAllocator.cpp:414-419). Torque components that are not finite are zeroed (:387-391). The method is selected by CA_METHOD (default 2, auto; module.yaml:50) and a multirotor requests sequential desaturation with normalised roll/pitch (ActuatorEffectivenessMultirotor.hpp:47-55).

The effectiveness matrix is static, rebuilt only on parameter changes. For rotor i with unit

axis a_i, position p_i relative to the centre of gravity, thrust coefficient c_t (`CA_ROTOR*_CT`, default 6.5) and moment ratio k_m (`CA_ROTOR*_KM`, default 0.05), the column is built from

$$F_i = c_t\, a_i, \qquad M_i = c_t\,(p_i \times a_i) - c_t\, k_m\, a_i$$

(`ActuatorEffectivenessRotors.cpp:194,197`). The model is linear in the normalised motor command $u \in [0, 1]$, although the parameter text speaks of u^2 (`module.yaml:211`). There is no rotor inertia, gyroscopic or drag term. The allocator then forms $\text{mix} = \text{geninv}(B)$ and divides its columns by a scale. Roll and pitch share one scale, the larger of $\sqrt{\|\text{mix}_{:,j}\|^2/(n_j/2)}$, $j \in \{\text{roll}, \text{pitch}\}$; yaw uses the maximum of its column (`ControlAllocationPseudoInverse.cpp:66-126,155-170`). The scale is fixed at set-up and not refreshed after an actuator failure (`:72`). It is not published. A unit of normalised torque is therefore a geometry-dependent quantity, not a torque in N m, and the same number gives a different angular acceleration on a different airframe. The allocated control reported back uses the same scale (`ControlAllocation.hpp:146-153`).

Desaturation order depends on `MC_AIRMODE`, whose default is 0 (`src/lib/mixer_module/params.yaml:5-21`; switch at `ControlAllocationSequentialDesaturation.cpp:52-64`). With airmode disabled the mix is built without yaw, thrust may only be reduced, then roll and then pitch are scaled down until feasible (`:174-200`), and finally yaw is added and desaturated last, with a small extra headroom above the maximum, followed by a thrust-only reduction (`mixYaw`, `:207-228`). The priority is thus thrust, roll and pitch, then yaw. With airmode on roll/pitch (`:122-143`) or roll/pitch/yaw (`:146-171`) the collective thrust may be raised to keep authority, and yaw is desaturated separately last. After allocation an optional slew-rate limit (only if a slew parameter is set; `ControlAllocator.cpp:111-120,454-456`) and a hard clip to the actuator limits are applied (`:458`).

The feedback to the controller is `control_allocator_status`, published at most every 5 ms (`:469-470`). `unallocated_torque` is the control setpoint minus the allocated control, in the same normalised units (`:675-682`), and `torque_setpoint_achieved` holds when its squared norm is below 10^{-6} (`:691-693`). The allocated control is computed from the clipped actuator setpoint, so it contains saturation and slew effects but nothing measured from the motors. The allocator does not apply `THR_MDL_FAC`; the output driver inverts it (`FunctionMotors.hpp:81-105`), and with the default 0 the command-to-thrust mapping is assumed linear.

4.6 Actuator feedback available (real hardware vs SITL)

The only measurement of the actuators in the PX4 data model is `esc_status`, an array of `EscReport` with `esc_rpm`, voltage, current and temperature fields, each filled only "if supported" (`msg/EscReport.msg`). `actuator_outputs` is the post-mixer command, not a measurement. `mc_rate_control` does not subscribe to `esc_status`; the one consumer on the control path is the dynamic notch of the gyro filter (`VehicleAngularVelocity.cpp:582-588`).

On hardware, rpm comes from DShot bidirectional telemetry (`DShot.cpp:793,803`, eRPM divided by pole pairs), UAVCAN or Cyphal ESCs (`uavcan/actuators/esc.cpp:141`) and a few other drivers (`TAP_ESC.cpp:298`). Rate and latency of this telemetry were not measured.

In Gazebo SITL the content of `esc_status` is not a measurement. `GZMixingInterfaceESC` subscribes to `/<model>/command/motor_speed` and advertises on the same topic (`GZMixingInterfaceESC.cpp:41-50`); it publishes the mixer outputs as rotor velocity commands (`:98-103`) and builds `esc_status` from that same message, with `esc_rpm = actuators.velocity(i)`, a fixed 50 degree temperature and online and armed flags (`:132-143`). The field named rpm is thus an echo of the command in the output range `SIM_GZ_EC_MIN..MAX` of 150 to 1000 for the x500 (`4001_gz_x500:40-48`), carrying no motor dynamics. No rotor-speed

topic is bridged back. Whether the gz motor plugin can publish one could not be verified, because its source is not in the repository. The software-in-the-hole simulator (SIH) differs: its `esc_rpm` is $r_{max}\sqrt{u}$ of a throttle state that already includes a first-order lag with `SIH_T_TAU` (`sih.cpp:481,548`, `aero.hpp:267-269`).

The x500 motor model in Gazebo is a first-order velocity model. All four plugin blocks use `timeConstantUp` 0.0125 s, `timeConstantDown` 0.025 s, `maxRotVelocity` 1000 rad/s, `motorConstant` 8.54858×10^{-6} (thrust $= k\,\Omega^2$), `momentConstant` 0.016, rotor drag 8.06428×10^{-5} and rolling moment 10^{-6} (`Tools/simulation/gz/models/x500/model.sdf:11-73`). The vehicle has mass 2.0 kg and inertia $I_{xx} = I_{yy} = 0.02167$, $I_{zz} = 0.04$ kg m^2 (`x500_base/model.sdf:9-16`). Two mismatches with the PX4 side are visible in the files. The PX4 airframe uses `CA_ROTOR*_KM` = 0.05 against the plugin's 0.016, so the allocator's yaw model is wrong by this ratio. And thrust is quadratic in Ω while Ω is linear in u (`mixer_module.cpp:565`, `THR_MDL_FAC` unset), whereas the allocator assumes thrust linear in u. The discretisation of the time constants inside the plugin is not verifiable here.

4.7 How alternative controllers integrate, and why

Two controllers already coexist with the standard chain: `mc_nn_control` and `mc_raptor`. Both are optional modules (started from `rc.mc_apps:49-58` when `MC_NN_EN` or `MC_RAPTOR_ENABLE` is 1) that replace the whole cascade. They register an external mode with an arming check (`mc_nn_control.cpp:295-296`, `mc_raptor.cpp:259-263`; `mc_raptor` can also take over the OFFBOARD mode through `MC_RAPTOR_OFFB`), request the setpoint type `TYPE_DIRECT_ACTUATORS` (`mc_nn_control.cpp:319`, `mc_raptor.cpp:529`) and publish `actuator_motors` directly (`mc_nn_control.cpp:533`, `mc_raptor.cpp:864-885`). The commander maps that setpoint type to a control mode with no flag set (`setpoint_types.cpp:51-53`), so `flag_control_rates_enabled` is false and `mc_rate_control` stays idle (`MulticopterRateControl.cpp:192`), and `flag_control_allocation_enabled` is false, so the allocator does not publish (`ControlAllocator.cpp:363 368,747-749`). Exclusion is by control-mode flags driven by the active mode, not by a check in `mc_rate_control`.

The reason for this design is that a neural policy outputs motor commands from state and has no use for a torque interface or the attitude cascade, so it needs a mode in which the standard loops are disabled. The price is a separate flight mode: the controller is not active in Stabilized, Position or Mission, and the allocator, with its prioritisation and failure handling, is bypassed. For a rate-loop law that must operate inside the standard modes and downstream of the same attitude loop, this pattern is the wrong shape, but it demonstrates the registration and flag mechanism and that two controllers may coexist when exactly one publishes `actuator_motors`.

4.8 Implications for INDI

The architecture poses a set of questions that the design must answer. They are listed here without answers.

1. Placement. The only point where angular acceleration, rate and rate setpoint are all available is the rate loop, and its output contract is a normalised torque. Should the incremental law publish torque ahead of the allocator, or actuator commands, and what does each choice do to the allocator's prioritisation, airmode and failure handling? How is the choice between the PID and the INDI law made, and when may it change?
2. Units. The allocator's normalised torque is dimensionless and geometry-dependent, and the scale is not published. How is the control effectiveness G, in rad/s^2 per unit of normalised torque, obtained per airframe: from inertia and the matrix, or by identification?
3. Reference signal. There is no angular acceleration setpoint. How is the desired acceleration formed from a rate setpoint, and does the feed-forward of the attitude reference model have a role?

4. Synchronisation. The measured acceleration lags the rate by several milliseconds through filters that are shared with other parameters and live at a different sample rate from the controller. What signal stands for the previous actuator state, and how is it brought to the same delay and gain as the acceleration estimate? Which filter parameters may be changed without disturbing the PID?
5. Actuator state. In Gazebo `esc_status` is an echo and carries no dynamics, while hardware telemetry is partial and its latency is unknown. What model of the motors, and in which space, supplies the actuator state, and how is it kept consistent with what the allocator actually achieved?
6. Saturation. The allocator clips and desaturates after the controller, and its status arrives at most every 5 ms and only reports static saturation. How is the incremental state kept from winding up, and how are airmode and the yaw-last priority taken into account?
7. Existing post-processing. The yaw torque low-pass, gain compression and battery scaling sit after the PID law. Which of them apply to an incremental law, and which would break its time alignment?
8. Operating states. The PID freezes only its integrator when landed and resets when disarmed. What happens to the incremental state on the ground, at arming, at a parameter change and at a filter re-initialisation?
9. Loop rate. The controller runs at the publication rate of the gyro (400 Hz by default) with a Δt clamped to $[0.125, 20]$ ms. How sensitive is the law and its synchronisation filter to this rate?
10. Fairness of comparison. Since the PID can be retuned against the same filters, what must be held equal in the benchmark so that gains are not attributed to INDI that a retuned PID also obtains?

5. Design and implementation of the INDI rate controller

This chapter describes the controller that was built: what it is required to do, the control law, why each architectural choice was made and which alternatives were rejected, how the yaw path was refined after measurements, and how the controller is implemented in PX4. Decision identifiers (D-arch-1 and so on) refer to `indi_project/DECISIONS.md`. Equations use the symbols of `src/lib/indi_rate_control/indi_rate_control.{hpp,cpp}`. The PX4 data flow and the PID controller being replaced are described in chapter 4 and are not repeated. Tuning and benchmark results are in later chapters; only the measurements that motivated a design change are quoted here. Where `plan/architecture.md` and the code differ, the code is followed and the difference is listed in section 5.8.

5.1 Requirements and scope

The controller is stage 1 of the project: it replaces only the angular-rate control law of a multicopter (the inner loop, `mc_rate_control`). The following are unchanged: the attitude loop (`mc_att_control`), the position loop, the control allocator and its prioritisation (thrust, then roll/pitch, then yaw), airmode, the gyro and gyro-derivative filtering in the sensors module, and the PID rate controller library (`src/lib/rate_control/`). Outer-loop (linear-acceleration) INDI is stage 2 and starts only after stage 1 has passed the held-out scenarios.

The requirements that drive the design are:

1. **PID stays the default.** `MC_RATE_CTL_TYPE` defaults to 0 (PID). With the default, the flown behaviour is the existing PX4 behaviour (the PID branch is the old code, re-indented).
2. **Standard flight modes keep working.** The controller publishes the same `vehicle_torque_setpoint` and `vehicle_thrust_setpoint` as the PID, so every mode, the allocator and failsafes see no difference.

3. **No windup, safe on the ground.** Saturation, landing, disarming and invalid data must not produce unbounded or non-finite output (D-arch-6).
4. **Fair comparison.** The controller must not obtain an advantage that a retuned PID could also have. This rule (D-session3-1) is why, for example, the yaw filtering added in 5.4 is only kept because PID already has an equivalent.
5. **Static memory, deterministic runtime.** No allocation in the control loop; fixed-size state.

5.2 Control law

5.2.1 Notation

All quantities are per body axis (roll, pitch, yaw), element-wise.

Symbol	Code name	Meaning	Unit
ω, ω_{sp}	`rate`, `rate_sp`	measured and desired body rate	rad/s
a_f	`angular_accel`, `_a_f`	filtered angular acceleration, `vehicle_angular_velocity.xyz_derivative`	rad/s^2
K	`_gain_k`	rate-tracking bandwidth	rad/s
ν	`nu`, `_nu`	virtual control: desired angular acceleration	rad/s^2
τ_{alloc}	`torque_allocated_prev`, `t_alloc`	normalised torque the allocator actually produced from the previous command	1
τ_{act}	`_t_act`	modelled actuator torque	1
H	`_lpf`, `_alpha`	filter chain matching the one that produced a_f	
τ_f	`t_f`, `_t_f`	filtered modelled torque	1
G_1, G_2	`_g1`, `_g2_yaw`	control effectiveness, yaw rotor spin-up term	rad/s^2 per unit torque; rad/s^2 per unit torque rate
$\Delta\tau$	`dt_cmd`, `_dt_cmd`	torque increment	1
τ_{cmd}	return value of `update()`	normalised torque command to the allocator	1
T_s	`dt`	controller cycle time	s

Normalised torque is the unit of `vehicle_torque_setpoint`: dimensionless, in $[-1, 1]$, with 1 the allocator's reference maximum for that axis.

5.2.2 Incremental law

INDI starts from the rigid-body rate dynamics, linearised in the torque about the current operating point:

$$\dot{\omega} = \dot{\omega}_0 + G_1 \left(\tau - \tau_0\right)$$

where $(\dot{\omega}_0, \tau_0)$ is a past measurement of acceleration and of the torque that produced it. Choosing the torque so that $\dot{\omega}$ equals the desired value ν gives the increment

$$\Delta\tau = G_1^{-1} \left(\nu - \dot{\omega}_0\right)$$

The reference acceleration is the measurement a_f, which already contains everything that moves the vehicle (gyroscopic terms, aerodynamic drag, gusts, centre-of-mass offset); none of it has to be modelled. The torque the measurement corresponds to is not the last command but what the motors actually produced, so the controller adds the increment to the modelled achieved torque τ_f. For roll and pitch, in the order of the code:

$$\begin{aligned}
e &= \omega_{sp} - \omega, & \nu &= K \odot e \\
\tau_{act}[k] &= \tau_{act}[k-1] + \alpha_a \left(\text{sat}_1(\tau_{alloc}) - \tau_{act}[k-1]\right), & \alpha_a &= 1 - e^{-T_s/T_a} \\
\tau_f &= H(z^{-d}\, \tau_{act}) \\
\Delta\tau &= \text{sat}_{\Delta_{max}}\Big(G_1^{-1} \left(\nu - a_f\right)\Big) \\
\tau_{cmd} &= \text{sat}_1(\tau_f + \Delta\tau)
\end{aligned}$$

with T_a = `MC_INDI_ACT_TC`, d = `MC_INDI_DELAY` samples, Δ_{max} = `MC_INDI_DT_MAX` and sat_L the symmetric clamp to $[-L, L]$.

With a perfect G_1 and perfect synchronisation, substituting into the plant gives $\dot{\omega} = \nu$, hence $\dot{e} = -Ke$ for a constant setpoint: a first-order rate response $\omega/\omega_{sp} = K/(s + K)$. K is therefore a physical bandwidth in rad/s (defaults 20, 20, 10 for roll, pitch, yaw). The setpoint feed-forward term $K_{ff}\, \dot{\omega}_{sp}$ of the architecture draft is not implemented (see 5.8).

There is no integrator. A constant disturbance moment shows up in a_f and is cancelled at the acceleration level; the steady-state acceleration error is zero by construction.

5.2.3 Relation to an incremental PID

With fixed G_1 and $\tau_f = H(A(\tau))$ the law is algebraically an incremental PID (Acquatella 2017, eq. 29-30, `acquatella2017`). The practical consequence, kept in view for the whole project, is that any benchmark gain must be attributable to what INDI does differently: (i) the feedback is the modelled achieved torque, not the commanded one, (ii) the measured acceleration rejects unmodelled moments immediately, (iii) saturation is handled without an integrator. Gains that a retuned PID also obtains must not be credited to INDI. This is why the PID baseline is tuned with the same budget and cost (D-session3-1).

5.2.4 Actuator-state estimate and filter synchronisation

The measured a_f is the backward difference of the gyro after notch and a second-order low pass at `IMU_GYRO_CUTOFF`, followed by a first-order `AlphaFilter` at `IMU_DGYRO_CUTOFF` (chapter 4). It is therefore a delayed, smoothed version of the true acceleration. Subtracting it from an un-delayed ν and adding the result to an un-delayed torque would act on mismatched time scales and destabilise the loop. The incremental form is only valid if a_f and the torque it is paired with, τ_f, have passed through the same filters (Smeur 2016, `smeur2016`, applies the same filter to the gyro-derived acceleration and to the actuator signal; Tal 2021, `tal2021`, Sec.

III-B, requires identical low-pass filters for equal phase lag; the Paparazzi implementation, `impl_paparazzi`, passes the actuator state through the same Butterworth as the gyro). The code therefore applies to the modelled torque

$$H = \underbrace{\mathrm{LPF2p}(\texttt{IMU_GYRO_CUTOFF})}_{\texttt{_lpf}} \circ \underbrace{\mathrm{AlphaFilter}(\texttt{IMU_DGYRO_CUTOFF})}_{\texttt{_alpha}}$$

with both cutoffs read from the IMU parameters so that they stay matched when those are changed. Filters are designed at the controller rate $1/T_s$ and redesigned when it moves by more than 10 % (`kRedesignRatio`) or when a cutoff changes; the redesign resets the filter states to the current τ_{act}, so no transient is introduced. A cutoff ≤ 0 makes a stage pass-through, as in the sensor chain.

Two approximations remain and are documented rather than hidden. First, the sensor filters run at the raw gyro rate and ours at the controller rate; in the Gazebo x500 both are 250 Hz (IMU rate from `x500_base/model.sdf`, architecture.md section 10), so they coincide in SITL. Second, the backward difference adds half a raw sample of delay and the publication adds up to one period of transport delay; an integer delay of `MC_INDI_DELAY` samples (0 to 4, default 0) on τ_{act} is available to absorb that, but half a sample cannot be represented. Notch filters are not replicated (default off in the IMU settings; if enabled the INDI synchronisation would be wrong: a limitation, 5.7).

5.2.5 Yaw: rotor spin-up term and yaw filters

Yaw torque comes from rotor drag, and changing yaw torque means accelerating rotors, whose inertia produces a reaction torque proportional to the rate of change of the commanded torque. Following the torque-space form of Smeur 2016, yaw adds a term $G_2\,\dot{\tau}$:

$$\dot{\omega}_z = a_f + G_1\,\Delta\tau + G_2\,\frac{\Delta\tau - (\tau_f - \tau_{f,prev})}{T_s}$$

Setting $\dot{\omega}_z = \nu_z$ and solving gives the yaw increment implemented in `update()`:

$$\Delta\tau_z = \frac{\nu_z - a_{f,z} + \dfrac{G_2}{T_s}\,(\tau_{f,z} - \tau_{f,z,prev})}{G_1 + \dfrac{G_2}{T_s}}$$

For roll and pitch G_2/T_s is set to 0 in the loop, which recovers 5.2.2. With the default $G_2 = 0$ (not identifiable from the yaw step data, D-g1-v1) the yaw axis is the same law as the other two with its own G_1.

Three further low-pass filters at one common cutoff `MC_INDI_YAW_CUT` act only on yaw. They are optional, off by default, and are introduced and justified in 5.4. Written with L_f for a second-order low-pass at that cutoff and z for the yaw component:

$$a_{f,z} \leftarrow L_f(a_{f,z}), \qquad \tau_{f,z} \leftarrow L_f(\tau_{f,z}), \qquad \nu_z = K_z\,L_f(\omega_{sp,z} - \omega_z)$$

The first two are applied to the pair in the same way, so the pair stays synchronised. Disabled means bypassed, not pass-through, so that behaviour with the parameter at 0 is bit-identical to the code without the filters (checked by a unit test, 5.6).

5.2.6 Effectiveness G_1

G_1 is a fixed per-axis parameter (D-arch-5, not adapted online). It was identified from in-loop rate steps in SITL (D-g1-v1): roll = pitch = 241 rad/s^2 (fits 237 and 245, R^2 0.94-0.95), yaw 7.4 rad/s^2 (fits 7.2-7.6), source `results/sync_check_v0.md`, overlay `bench/params/indi_v1.params`. Identification is a one-off step, not a tuning variable: G_1 is identified first, then K, `MC_INDI_ACT_TC` and the attitude gains are tuned. Two alternatives were rejected. The first identification, an open-loop fit on PID logs (321 and 211), gave different values for the two axes of a symmetric vehicle and fitted worse. Adaptive estimation (LMS in Smeur 2016, RLS in Blaha 2024, `blaha2024`) is deferred as the first extension if payload or plant-mismatch scenarios show that G_1 errors dominate. That a fixed G_1 is adequate for stage 1 is an assumption, not a cited result: it is tested by the model-mismatch scenarios, not assumed.

The plant is nonlinear in the actuator: in Gazebo thrust $\propto \omega_{rotor}^2$ with ω_{rotor} linear in the motor command, while the allocator assumes thrust linear in command (`THR_MDL_FAC = 0`). G_1 is therefore operating-point dependent (roughly proportional to rotor speed near hover). The PID has the same mismatch; INDI sees it as a G_1 error. G_1 is a hover-point value.

5.3 Architecture choices

Each choice below lists the decision, the reason and the alternatives that were rejected. The common theme is minimal intrusion into PX4: the controller must be selectable, comparable with the PID on the same airframe, and removable.

5.3.1 Torque space ahead of the allocator (D-arch-1)

Chosen. INDI outputs a normalised torque and publishes the normal `vehicle_torque_setpoint`. The allocator, its thrust > roll/pitch > yaw prioritisation and airmode work unchanged, and every flight mode that works with the PID works with INDI.

Rejected: actuator-space INDI ($u = u_{prev} + B^{+}(\nu - a)$, one increment per motor). It needs either a new allocator input or bypassing the allocator, which loses prioritisation, airmode and the standard modes, and the earlier PX4 attempt (PR #25216, `impl_px4_prior`) drew reviewer requests for no changes to `ControlAllocator`. It remains a possible later branch. The price of the choice is that the actuator model has to live in torque space (5.3.3).

5.3.2 Selection inside `mc_rate_control`, law in a new library (D-arch-2)

Chosen. A new library `src/lib/indi_rate_control/` holds the law and its unit tests; the PID library is untouched. `mc_rate_control` gets one enum parameter `MC_RATE_CTL_TYPE` (0 PID, 1 INDI) and branches around the single call to the law. The selection is latched while disarmed: a change in flight applies at the next arming, because switching laws mid-air is a transient nobody needs.

Rejected: a separate module chosen in the startup script. It would leave the PID module byte-identical, but would duplicate acro-mode setpoint generation, thrust and battery handling, autotune hooks and status publishing, and would put two publishers on the torque topic, which is a hazard. **Rejected: the external-mode pattern of `mc_nn_control` / `mc_raptor`**, which bypasses the allocator and standard modes and is the wrong fit for a rate law.

5.3.3 Actuator state from a first-order torque-space model (D-arch-3)

INDI needs the torque the vehicle is producing now, τ_f. There is no measurement of it in SITL: Gazebo's `esc_status.esc_rpm` only echoes the command (integration analysis), as on

any vehicle without RPM telemetry. An onboard model is therefore required.

Chosen (stage 1). The model input is the allocated torque, i.e. the previous cycle's published `vehicle_torque_setpoint` minus `control_allocator_status.unallocated_torque` (the latter only if the allocator status is newer than the previous torque publication, otherwise the unmodified previous setpoint). The model is a discrete first-order lag with time constant `MC_INDI_ACT_TC` (default 0.02 s, between the x500 values 0.0125 s rising and 0.025 s falling). Using the allocated and not the commanded torque is what makes saturation harmless (5.3.5).

Rejected: ESC feedback (gz echoes the command, so it would add nothing). **Rejected: a per-motor model.** The allocator's normalisation scale is not published, so mapping per-motor states back to normalised torque needs a copy of the effectiveness matrix, which would couple the controller to one airframe's allocator and break the torque-space interface. The torque-space lag is exact when no motor is saturated and the rising and falling time constants are equal, and is wrong by their asymmetry otherwise. The trigger to upgrade is recorded in D-arch-3: the benchmark shows that motor asymmetry or saturation limits performance.

5.3.4 Filter matching by filtering the model, not the measurement (D-arch-4)

The synchronisation of 5.2.4 could be obtained either by filtering the modelled torque like the measurement, or by changing the sensor filters. **Chosen:** apply H to the modelled torque, reading `IMU_GYRO_CUTOFF` and `IMU_DGYRO_CUTOFF`, and leave the shared IMU filter defaults alone. **Rejected:** retuning the shared sensor filters for INDI, because they are shared with the PID D-term and every axis. If tuning later shows that INDI needs a higher `IMU_DGYRO_CUTOFF`, it is a per-controller parameter set and the retuned PID gets the same freedom (D-session3-1). A pure delay on τ_{act} is provided for residual lag identified in flight.

5.3.5 Saturation, arming and landing (D-arch-6)

- **Saturation.** $\tau_{cmd} = \tau_f + G_1^{-1}(\nu - a_f)$ has no accumulator, and τ_f follows what the allocator produced. A saturated axis therefore cannot wind up: when the allocator clips the command, the modelled torque is the clipped one and the next increment is computed from it. τ_{cmd} is clamped to $[-1, 1]$ and each increment to $\pm$`MC_INDI_DT_MAX` (default 1, no extra limit). Rejected: copying the PID anti-windup (conditional integration, integral clamp), which is the machinery INDI is meant not to need.
- **Landed or maybe landed.** Ground contact makes $a_f \approx 0$ whatever the torque, so an incremental loop would integrate. The law holds the actuator model and filters at zero ($\tau_f = 0$) so the output is the bounded proportional term $G_1^{-1}(\nu - a_f)$, mirroring the PID freezing its integrator. Rejected: letting the law run normally on the ground.
- **Disarmed, first run, selection change.** All dynamic states are reset to zero, once per cycle while disarmed.
- **Invalid data** is handled by a reset and zero output with a fault flag (5.5.4).

5.3.6 What INDI replaces and what it does not

INDI replaces the call to `RateControl::update()`, and with it the PID's P, I, D and feedforward terms. It does not replace the sensors-side filtering, thrust (passed through from `vehicle_rates_setpoint.thrust_body`) or the allocator. Two PID-branch-only steps are skipped on the INDI branch: the 2 Hz yaw output low-pass (`MC_YAW_TQ_CUTOFF`) and battery scaling. A low-pass after the INDI output would sit in series with the actuator, which the law assumes is described by τ_f alone, and would break the synchronisation. Battery-voltage effects change G_1, which the incremental form partly absorbs; the gz simulation has no battery model, so this is untested. The gain-compression step is also PID-only. `rate_ctrl_status` is published only on the PID branch because its integrator fields are meaningless for INDI.

5.4 Yaw-path refinements, in chronological order

The architecture of 5.2 and 5.3 was frozen first (session 5) and flown. The roll and pitch paths needed no change. The yaw path needed three, each triggered by a measured excess of control effort (motor-command rate RMS, "effort") relative to PID, and each applied before tuning continued. They share one parameter, `MC_INDI_YAW_CUT`, which grew in scope; the order matters because each step corrected what the previous one left.

5.4.1 D-yawfilt: low-pass on the synchronised yaw pair (session 7)

Problem. The identified G_1 for yaw is small (7.4), so the yaw acceleration noise reaches the yaw torque with gain $1/G_{1,z}$, four times the gain of the first (unidentified) value of 30. Hover effort was already 8 times the PID's with $G_{1,z} = 30$ (0.300 against 0.037 per second, `results/sync_check_v0.md` section 3) and rose to 1.616 (44 times PID) with $G_{1,z} = 7.4$ (`results/yawcut_check_s8.md`).

Chosen. An extra second-order low-pass on the yaw axis, `MC_INDI_YAW_CUT` (Hz, 0 = off), applied identically to $a_{f,z}$ and to $\tau_{f,z}$ so that the synchronisation of 5.2.4 holds. It costs yaw phase lag only; yaw bandwidth is low anyway because yaw is drag-torque limited. For fairness, PID's own yaw output filter `MC_YAW_TQ_CUTOFF` enters the PID tuning space.

Alternatives rejected. Inflating $G_{1,z}$ above the identified value (this is what the first version did by accident): it detunes the cancellation and hides the model in a gain. Lowering `IMU_DGYRO_CUTOFF`: shared with PID's D-term and all axes, while roll and pitch noise were already at PID's level. Reusing `MC_YAW_TQ_CUTOFF` on the INDI output: filtering after the increment breaks synchronisation.

Measured effect (hover, seed 1, `results/yawcut_check_s8.md`, one flight each, smoke level): a 10 Hz cutoff reduced hover effort from 1.616 to 0.228 (PID 0.037), a 7-fold reduction, confirming the diagnosis; step_yaw effort did not change (3.69 against 3.77, PID 0.559), so it came from the step response and not from noise, which led to the next two steps.

5.4.2 D-yawnu: the yaw rate used in ν is filtered too (session 11)

Problem. With $a_{f,z}$ and $\tau_{f,z}$ filtered at PID's 2 Hz, hover effort was still about 2.1 times PID's (`results/tuning/r2f_yc2.md`). The remaining chatter was gyro noise in $\nu_z = K_z(\omega_{sp,z} - \omega_z)$, which reaches the torque with gain $K_z/G_{1,z} \approx 0.8$.

Chosen. When `MC_INDI_YAW_CUT` > 0, the measured yaw rate in ν passes through a third low-pass of the same design (the setpoint was left unfiltered at this stage). One knob then sets the yaw noise bandwidth of every feedback path of the yaw increment, as `MC_YAW_TQ_CUTOFF` does for PID's whole yaw output. A cutoff of 0 stays bit-identical to the preceding code. Results of rounds 1 and 2 were obtained with the old semantics and are labelled accordingly.

Alternatives rejected. Filtering the whole yaw increment (adds lag in series with the actuator and breaks the a_f/τ_f synchronisation). Only shrinking K_z: the tuner already had K_z in its space and settled at 5.8, and lowering it trades away yaw tracking that is INDI's measured advantage. A separate parameter: it enlarges the tuning space for a knob whose physical meaning (yaw noise bandwidth) is the same.

Measured effect (hover, seeds 1-3, `results/yawnu_check_s11.md`): hover motor-rate effort 0.077-0.080 before, 0.041-0.043 after, against 0.037 for PID, i.e. about 2.1 times down to about 1.1 times; yaw torque change per sample dropped from 0.0003 to 0.0000.

5.4.3 D-yawsp: the setpoint is filtered as well (session 13)

Problem. After D-yawnu and a tuning round centred at lower yaw gains (D-r3), one INDI penalty remained under the frozen cost: step_yaw effort was about 5 times PID's (4.66-5.89). Measurement (`bench/yaw_step_effort.py`, quoted in D-r3): on each 45 degree yaw step the setpoint term $K_z\,\omega_{sp,z}/G_{1,z}$ peaked at 2.75 normalised torque (the rate setpoint was 3.49 rad/s, clipped by the yaw rate limit), so yaw torque hit its limit of 1.0, against a PID peak of 0.33. Motor-rate RMS around the step was 11.1 against 1.55 for PID (7 times). PID smooths the same step with its default 2 Hz yaw torque output filter; INDI passed it through unfiltered.

Chosen. $\nu_z = K_z\,L_f(\omega_{sp,z} - \omega_z)$: the filter acts on the rate error, setpoint included (firmware 99634a6b71). This gives INDI the structure PID already has by default, so it is not an advantage offered only to INDI.

Alternatives rejected. A separate yaw setpoint pre-filter parameter (one more parameter for the same effect; D-r3 also rejected it earlier as unfair to PID unless offered to both). Changing the effort metric or its weight: the cost is frozen (`bench/scenarios.md` v1), and changing it after seeing INDI lose would be after-the-fact selection. Lowering K_z further: r3 already showed yaw tracking degrading while the torque step remained, since it scales with K_z.

Measured effect (`results/tuning/r3f_yawsp.md`, candidate c13, seeds 1-3, 15 flights): step_yaw effort ratio to PID 1.42 (was 4.66-5.89); on seed 1 motor-rate RMS around the steps 1.13 (4.70 before, PID 0.79) and peak yaw torque 0.45 (0.78 before, PID 0.33). Cost: step_yaw score ratio 0.876 to 1.056, the lag PID also pays with its 2 Hz filter. Roll and pitch were unchanged within noise, as expected for a yaw-only change.

A consequence is that the library header describes `MC_INDI_YAW_CUT` as a filter on three signals: $a_{f,z}$, $\tau_{f,z}$ and the yaw rate error. Its default stays 0, so the stand-alone law is unfiltered; the tuning rounds from r3 on are centred at 2 Hz (D-r3).

5.5 Implementation in PX4

5.5.1 Files

File	Content
`src/lib/indi_rate_control/indi_rate_control.hpp, .cpp`	the law, class `IndiRateControl`, no PX4 module dependencies
`src/lib/indi_rate_control/test_indi.cpp`	unit tests (5.6)
`src/modules/mc_rate_control/MulticopterRateControl.hpp, .cpp`	selection, parameter wiring, allocator feedback, `runIndiRateControl()`, status publication
`src/modules/mc_rate_control/mc_rate_control_params.c`	`MC_RATE_CTL_TYPE` and `MC_INDI_*`
`msg/IndiRateStatus.msg`, `msg/CMakeLists.txt`, `src/modules/logger/logged_topics.cpp`	the `indi_rate_status` topic, registered and logged

The module depends on the library through its CMake `DEPENDS`. The PID library and `ControlAllocator` are not modified.

5.5.2 Runtime selection and cycle

`MulticopterRateControl::Run()` is triggered by each `vehicle_angular_velocity` sample, as for the PID. While the vehicle is disarmed (or not a rotary wing) the module latches `_use_indi = (MC_RATE_CTL_TYPE == 1)`, resets the INDI states and zeroes the stored previous torque;

first run, change of selection and disarm are therefore the same code path. When armed with `_use_indi`, each cycle:

1. reads ω and a_f from `vehicle_angular_velocity` (`xyz`, `xyz_derivative`) and the rate setpoint as for the PID, with the module's `dt` clamped to [0.000125, 0.02] s;
2. builds τ_{alloc}: the previously published torque, minus the stored `unallocated_torque` if the allocator status timestamp is newer than that publication;
3. calls `IndiRateControl::update(rates, rate_setpoint, angular_accel, torque_allocated_prev, dt, landed)` with `landed = _maybe_landed || _landed`;
4. publishes `indi_rate_status` and then the usual `vehicle_torque_setpoint` and `vehicle_thrust_setpoint` (thrust unchanged, a non-finite torque component is replaced by 0 as in the PID branch). The published torque is stored for the next cycle.

On the PID branch the code is as before (re-indented). `IMU_GYRO_CUTOFF` and `IMU_DGYRO_CUTOFF` are read with `param_get` in `parameters_updated()` and passed to `setFilterCutoffs()`.

5.5.3 Parameters

All are only used when `MC_RATE_CTL_TYPE` = 1. Values are the firmware defaults; the identified and tuned values are applied by parameter overlays in `indi_project/bench/params/` and are reported in the tuning chapters.

Parameter	Default	Range	Unit	Meaning
`MC_RATE_CTL_TYPE`	0 (PID)	0, 1	enum	rate law, latched while disarmed
`MC_INDI_G1_R`	320	0.001-5000	rad/s^2 per unit torque	roll effectiveness
`MC_INDI_G1_P`	210	0.001-5000	same	pitch effectiveness
`MC_INDI_G1_Y`	30	0.001-5000	same	yaw effectiveness (not identified for any airframe)
`MC_INDI_G2_Y`	0	0-1	rad/s^2 per unit torque rate	yaw spin-up term, 0 disables
`MC_INDI_K_R`, `_K_P`, `_K_Y`	20, 20, 10	1-100	rad/s	rate-tracking bandwidth
`MC_INDI_ACT_TC`	0.02	0.001-0.2	s	actuator lag time constant
`MC_INDI_DELAY`	0	0-4	cycles	pure delay on the modelled torque
`MC_INDI_DT_MAX`	1.0	0.01-1	unit torque	per-cycle, per-axis increment limit
`MC_INDI_YAW_CUT`	0 (off)	0-100	Hz	yaw low-pass, 5.4

The firmware defaults for G_1 (320, 210, 30) are the first estimates from open-loop PID logs, not the later in-loop values (241, 241, 7.4) of D-g1-v1; the latter are used through the overlay. Because $G_{1,z}$ is unidentified for other airframes, a different airframe must identify G_1 before using INDI.

5.5.4 Numerical edge cases

Case	Handling in the code
Non-finite `rate`, `rate_sp`, `angular_accel` or previous torque; `dt` outside [0.000125, 0.02] s	`update()` calls `reset()`, sets `fault`, returns zero torque for that cycle. The module also clamps `dt` before the call, so only invalid sensor data triggers it in practice.
Zero, negative, NaN G_1 (below `kMinG1` = 1e-3) or negative or non-finite K	same: `paramsValid()` is checked every cycle, so a bad parameter set never divides. The parameter minimum already excludes most of these.
Non-finite G_2, cutoffs, delay	setters sanitise: G_2 <= 0 or NaN becomes 0; `ACT_TC` clamped to [0.001, 0.2] s and NaN replaced by 0.02; filter and yaw cutoffs <= 0 or NaN become 0 (off / pass-through); delay clamped to 0-4; `DT_MAX` <= 0 or NaN becomes 1.
Overflow in $\Delta\tau$ from absurd inputs	the increment is checked for finiteness before use, with the same reset-and-fault outcome.
Allocated torque outside [-1, 1]	clamped before the actuator model, so a corrupt allocator value cannot overflow the lag.
Zero thrust	thrust is not used by the law (torque space, no thrust-dependent scaling); the law's output is bounded whatever the thrust. Whether the allocator can realise the torque at low thrust is the allocator's concern and appears in τ_{alloc}.
Saturation	5.3.5: no accumulator; τ_f follows the allocated torque; clamps on $\Delta\tau$ and τ_{cmd}.
Landed	`resetStates()` every cycle, so $\tau_f = 0$, delay buffer and filters are cleared and the output is the bounded term $G_1^{-1}(\nu - a_f)$. After take-off the filters start from zero, consistent with the actuator model.
Disarm, first run, selection change	`_indi.reset()` and `_prev_torque_setpoint.zero()` in the disarm block.
Loop-rate change > 10 % or cutoff change	filters redesigned, states settled at the current values (no transient).
Yaw cutoff >= 0.45 x sample rate	treated as disabled (stricter than `LowPassFilter2p`, which disables at 0.5).

The `fault` flag in `indi_rate_status` makes a rejected cycle visible in the log. Static memory only: `Vector3f` state, ring buffer of 5 samples (`kMaxDelaySamples`+1), fixed filter objects.

5.5.5 uORB message and logging

`indi_rate_status` (`msg/IndiRateStatus.msg`) is published by `mc_rate_control` only while INDI is selected, every cycle:

Field	Type	Content
`timestamp`	uint64	microseconds
`nu[3]`	float32	desired angular acceleration ν used (rad/s^2)
`angular_accel_filtered[3]`	float32	a_f used, after the yaw low-pass if enabled
`torque_filtered[3]`	float32	τ_f used, after the yaw low-pass if enabled
`torque_increment[3]`	float32	$\Delta\tau$ (clamped)
`fault`	bool	inputs or parameters rejected this cycle

It is added with `add_optional_topic("indi_rate_status")` at full rate, so tuning and the report can plot the synchronisation (a_f against τ_f) directly. `nu[2]` reports the yaw value actually used, which is filtered when `MC_INDI_YAW_CUT` > 0.

5.6 Unit tests

`indi_rate_control_test.cpp` contains 18 gtests (14 after the first yaw-cutoff change, 18 after D-yawnu and D-yawsp; `plan/integration_log.md` records 14 of 14 passing, and `results/unittest_s13.log` records the ctest target passing in session 13 without a per-test count). They use a simulated roll plant (rigid body, first-order actuator, G_1 = 100, acceleration filtered like PX4's gyro derivative, 400 Hz). They cover:

- **Zero in, zero out:** zero inputs for 400 cycles give exactly zero torque and no fault.
- **Closed-loop rate step:** a 1 rad/s step reaches 95 % within 0.5 s, overshoots less than 20 % and settles to within 5 %, torque bounded (expected first-order response).
- **Disturbance rejection without an integrator:** a constant moment disturbance of -30 rad/s^2 is driven to a rate below 0.05 rad/s within 2 s.
- **No windup:** the allocated torque is clipped to ± 0.05 for 2 s under a large setpoint, then released; the rate returns below 0.05 rad/s with bounded, finite output.
- **Landed:** with a constant rate error and no measured acceleration, the output equals the bounded proportional increment (0.1 here) and the filtered torque stays zero.
- **Invalid input:** NaN and infinity on each input, zero G_1, and bad Δt give zero output and a fault, with state reset.
- **Yaw G_2:** with G_2 = 0 yaw equals the roll/pitch form; a non-zero G_2 changes the yaw output only.
- **Filter design:** a sample-rate change redesigns the filters without a transient; pass-through filters and the pure delay behave as specified.
- **Yaw cutoff (D-yawfilt):** cutoff 0 leaves outputs unchanged; noise on the yaw acceleration input gives lower yaw output deviation (below 0.7 times) with roll and pitch equal; enabling causes no transient; invalid cutoffs (NaN, infinity, negative, 120 Hz) disable the filter.
- **Yaw rate filter (D-yawnu, D-yawsp):** cutoff 0 unchanged, noise reduction on the yaw rate path with roll/pitch equal, steady-state $\nu_z = K_z(\omega_{sp,z} - \omega_z)$, and smoothing of a yaw setpoint step.

These tests check the algebra and the safety behaviour in a linear idealised plant. They do not check the synchronisation against the real sensor chain, a multi-axis plant, or the PX4 module wiring; those are covered only by the SITL benchmark.

5.7 Limitations

- **Torque-space actuator model.** A single first-order lag per axis cannot represent motor asymmetry (different rising and falling constants) or per-motor saturation; the error is bounded by the asymmetry in normal flight and unquantified in heavy saturation or motor loss.
- **Fixed G_1.** It is a hover-point value for one airframe, and effectiveness varies with rotor speed. There is no online adaptation; payload, battery and airframe changes are errors in G_1.
- **$G_{1,z}$ and G_2.** The yaw effectiveness is small and noisy, and G_2 was not identifiable (left at 0). The yaw filters are a pragmatic noise control, not a model.
- **Synchronisation is approximate.** Half a raw sample of difference delay cannot be represented by an integer delay; notches are not replicated; filter matching assumes the IMU filters run at the controller rate.
- **No battery model.** Battery scaling is skipped for INDI and the simulator has no battery sag; its effect was not tested.
- **No setpoint feed-forward**, no gain compression, no autotune support for INDI.
- **Simulation only.** Gazebo x500; no hardware flights, no real motor lag or rotor-speed measurement.
- **Single airframe, single loop rate.** The parameters were identified at 250 Hz on the x500.

5.8 Discrepancies found

Notes where `plan/architecture.md` and the code differ; the chapter follows the code.

- Note: architecture.md says `K_ff` defaults to 0 and gives $\nu = K \odot (\omega_{sp} - \omega) + K_{ff}\,\dot{\omega}_{sp}$; code has no `K_ff` and no setpoint derivative, $\nu = K \odot (\omega_{sp} - \omega)$ only.
- Note: architecture.md says the allocated torque uses `unallocated_torque` "only when fresher than 20 ms"; code uses it only when the allocator status timestamp is newer than the previous torque publication (no 20 ms limit).
- Note: architecture.md says that for non-finite inputs the law outputs a "zero torque increment"; code returns a zero torque vector (total command zero) after resetting all states.
- Note: architecture.md says the landed output is "$G_1^{-1}\nu$"; code computes $G_1^{-1}(\nu - a_f)$ with $\tau_f = 0$. They are equal only if $a_f = 0$ on the ground (the unit test supplies $a_f = 0$).
- Note: architecture.md says $G_1 \leq \epsilon$ is validated "at update" and the loop rate uses "the module's clamped dt"; code validates G_1 (>= 1e-3) and K (>= 0) in `update()` every cycle, and rejects `dt` outside [0.000125, 0.02] s in the library in addition to the module's clamp to the same interval.
- Note: architecture.md section 6 gives initial G_1 321/211 (open-loop fit) and the integration log 320/210/30 as defaults; the values actually used in the tuned configurations are 241/241/7.4 (D-g1-v1) through an overlay, not the firmware defaults.
- Note: architecture.md describes only the yaw G_2 term; the yaw low-pass (D-yawfilt), the filtered rate in ν (D-yawnu) and the filtered setpoint (D-yawsp) were added later and are not in architecture.md (it was not updated); the code header is current.
- Note: architecture.md cites `wang2019` for tolerance to G_1 errors, which is not in `research/sources.md`; it is not used in this chapter.

Unverified: the 18-test count and the test thresholds come from reading `indi_rate_control_test.cpp`; the tests were not re-run for this chapter; the yaw-filter figures are copied from the cited result files and decisions as written.

6. Simulation environment and its limits

Every number in chapters 7 and 8 was produced in one simulator: PX4 software-in-the-loop (SITL) with the Gazebo Harmonic `gz_x500` quadrotor, run headless inside a container. This chapter describes that environment, the extensions added to it so that the disturbance and plant-change scenarios could be flown, the frozen benchmark that runs on it, and the ways in which the simulator is easier on the INDI controller than a real vehicle would be. Decision identifiers (D-speed and so on) refer to `indi_project/DECISIONS.md`. Facts about the simulated vehicle come from `indi_project/bench/sim_fidelity.md`. Where the project data do not support a statement, the text says so and lists it as a limitation.

6.1 Toolchain and reproducibility

6.1.1 Container

All builds and flights run in the Docker image `px4-indi-sim`, defined in `indi_project/docker/Dockerfile`. It starts from Ubuntu 24.04 and runs the repository's own `Tools/setup/ubuntu.sh --no-nuttx`, so the PX4 build dependencies, Gazebo Harmonic, MAVSDK and the Python test requirements come from the same script that PX4 developers use. The stock `Tools/setup/Dockerfile` has no Gazebo (`--no-sim-tools`), hence the dedicated image. It adds a software OpenGL driver and `xvfb` for a headless Gazebo server, `procps` and `scipy`. The gz-sim version in the image is 8.15 (`DECISIONS.md`, D-dist).

`indi_project/docker/run.sh` is the single entry point. It runs one command as the host user with the repository mounted at its own path, `~/.ccache` shared, no TTY, no privileged options, no published ports and a wall-clock timeout (default 3600 s). With `-N` the container gets a private network namespace with only loopback. The benchmark runner works in this mode: Gazebo, PX4 and the MAVLink link between the runner and PX4 all live inside it (`indi_project/bench/RUNNER_NOTES.md`, step 2).

6.1.2 One run

`indi_project/bench/run_scenario.py` flies one scenario for one controller and one seed (`--scenario`, `--controller`, `--seed`, `--speed`, `--params`, `--out`, `--instance`). It sets `HEADLESS=1`, `PX4_SIM_MODEL=gz_x500` and `PX4_SIM_SPEED_FACTOR` from `--speed`, and starts the Gazebo server itself (`gz sim -r -s --seed N`) before PX4, because the PX4 start script attaches to an already running world. The runner talks to PX4 with pymavlink, and setpoint timing follows `time_boot_ms` (PX4 simulation time), so scenario timing does not depend on the speed factor.

Each run uses a fresh PX4 working directory under its output folder. The shared build rootfs would keep `parameters.bson` between runs; a fresh directory means every run starts from the airframe defaults and parallel runs are safe. The logger profile is raised (`SDLOG_PROFILE=17`) so that rates, attitude and motors are logged at full rate. The record `run.json` stores scenario, controller, seed, speed factor and the extension parameters applied.

PX4 and Gazebo run in lockstep (`DECISIONS.md` D-hover-noise; `bench/sim_fidelity.md`). The project does not configure this; it is the stock SITL behaviour. `boards/px4/sitl/sitl.cmake` enables the lockstep scheduler for every SITL build except replay, and the Gazebo bridge (`src/modules/simulation/gz_bridge/GZBridge.cpp`, `clockCallback`) sets the PX4 monotonic clock from the Gazebo `/world/<name>/clock` topic on every message, through `px4_clock_settime`, which advances the lockstep scheduler (`platforms/posix/src/px4/common/drv_hrt.cpp`). PX4 time therefore advances only when the simulator steps.

6.1.3 Seeds

The seed has three effects. It seeds the Gazebo noise generator (`gz sim --seed N`), from which the IMU, magnetometer and barometer noise are drawn. It jitters the disturbance profile: gust onset time uniform within +-0.5 s and wind direction within +-10 deg (`indi_project/bench/scenarios.md` section 3.3). The same seed list is used for every controller (paired design). Tuning evaluations use 3 seeds (2 in round 1), the final validation 10.

Two things are not seeded. The barometer simulation inside PX4 calls `srand(1234)` with a fixed value that cannot be changed without a firmware change (`RUNNER_NOTES.md`, step 1). The trajectory start phase is not randomised either (`scenarios.md`, change log, session 15). Nor does the seed make a run exactly repeatable (6.5).

6.1.4 Speed factor and the D-speed finding

Running at real time (factor 1) is slow, so the speed factor was checked before the benchmark (D-speed, session 4, `indi_project/results/speedcheck_pid_default.md`). On `pid_default`, seeds 1 to 3, at factor 1 and factor 4, the attitude-step and rate-step metrics (rate RMS, attitude RMS, rise time, overshoot, motor effort) agreed within the seed-to-seed spread. Hover position RMS did not: 0.027 m (range 0.021 to 0.033) at factor 1 against 0.064 m (0.038 to 0.104) at factor 4.

The decision was to run attitude and rate scenarios at factor 4 and every scenario whose metric is a position (hover, trajectory tracking, gust position deviation) at factor 1. Factor 1 everywhere would cost four times the CPU; factor 4 everywhere was rejected because it changes position-hold results. The cause was left open (candidates: MAVLink setpoint timing at factor 4, EKF and sensor timing). The check covered `pid_default` only, so agreement for the tuned PID and INDI at factor 4 is untested (`report/07_tuning.md`).

6.2 The gz_x500 vehicle model

Table 6.1 lists what the stock model contains, read in session 17 from the model files (`indi_project/bench/sim_fidelity.md`). The right-hand column is the gap to a real vehicle, as stated in that file.

Item	gz_x500 value	Source file	Gap to a real vehicle
Motor dynamics	first order, time constant 0.0125 s up, 0.025 s down	`Tools/simulation/gz/models/x500/model.sdf`	no ESC model, no voltage sag
Thrust and moment constants	k_f = 8.54858e-6, k_m/k_f = 0.016	same	allocator assumes `CA_ROTOR*_KM` 0.05
Rotor speed feedback	none measured; `esc_rpm` echoes the command	chapter 4, `plan/px4_analysis_verification.md`	INDI uses its own motor model
IMU rate	250 Hz	`Tools/simulation/gz/models/x500_base/model.sdf`	real IMU 1 to 8 kHz with anti-aliasing
Gyro noise	white Gaussian, std 8.7e-4 rad/s (0.05 deg/s) per axis, no bias, no drift	same	no vibration, no bias instability
Accelerometer noise	white Gaussian, std 0.00637 m/s^2 (x, y), 0.00686 (z), no bias	same	no vibration-induced noise

Item	gz_x500 value	Source file	Gap to a real vehicle
Barometer, magnetometer	baro 50 Hz, std 3 Pa; mag 100 Hz, std 3e-7 T	same	

Table 6.1: gz_x500 model values and gaps (*bench/sim_fidelity.md*).

6.2.1 What the INDI controller depends on

The controller of chapter 5 uses three quantities that are properties of the simulated plant, and the model supplies each in an idealised form.

1. **Angular acceleration.** It is obtained by differentiating the gyro, filtered in the sensors module. The gyro in the model is white, bias-free and vibration-free. Differentiation amplifies exactly the high-frequency content that this model leaves out.
2. **Actuator state.** The controller does not read rotor speed. In gz the `esc_rpm` message only echoes the command, so there is nothing to feed back, and INDI uses its own first-order motor model with time constant `MC_INDI_ACT_TC` (chapter 5; tuned value 0.01142 s in `DECISIONS.md` D-freeze). The plant itself is first order with 0.0125 s up and 0.025 s down. Controller model and plant have the same structure, which is the favourable case for actuator compensation (assessment, not tested).
3. **Control effectiveness.** The gain G1 (241, 241 and 7.4 for roll, pitch and yaw in D-freeze) is a property of this vehicle. They were fitted in-loop from flight data of the first INDI flights, regressing filtered angular acceleration on the filtered torque increment (`results/sync_check_v0.md`, roll/pitch 237/245 averaged to 241, R^2 0.95; yaw 7.4), and stored in `bench/params/indi_v1.params`.

The thrust and moment constants contain one deliberate inconsistency that both controllers inherit. The model has k_m/k_f = 0.016 while the PX4 allocator assumes `CA_ROTOR*_KM` = 0.05 (airframe `4001_gz_x500`), a model error of about a factor of 3 in yaw moment (`sim_fidelity.md`). It is the same for PID and INDI.

6.3 Simulator extensions

The stock model has no wind, no payload change, no per-motor degradation and no way to change inertia per run. Four extensions were added, each in the harness and none in the model files; `Tools/simulation/gz/models` is never modified. Following `scenarios.md` section 3, each controller experiences the same change and disturbance with the same seeds.

6.3.1 Gusts and payload (`bench/disturbance.py`)

`indi_project/bench/disturbance.py` runs inside the container next to the runner. It loads Gazebo's `ApplyLinkWrench` system at runtime through the world service, so no world or model file is edited (D-dist, session 10). It then publishes a one-step wrench on the `base_link` of the x500 at every Gazebo clock message. The profile is a function of Gazebo simulation time, so it does not depend on the speed factor. The runner writes the onset (PX4 time) to `disturbance_trigger.json`, and the applied wrench is logged to `disturbance.csv`.

Four profiles are implemented.

- **gust_x (T6).** A base wind of 3 m/s along +x ramped in 2 s and held for 10 s, then a 1-cosine gust of +6 m/s over 1 s (peak 9 m/s), then 15 s of hold. Wind is modelled as quasi-steady drag, F = 0.5 rho C_d A v|v|, with rho = 1.225 kg/m^3 and C_d A = 0.06 m^2, applied 5 cm above the centre of mass. C_d A and the centre of pressure height

are assumptions of the project (plan/disturbance_spec.md, as referenced in the source), not measured values.

- **gust_diag (H2).** The same shape with a 4 m/s base wind towards 45 deg and a gust of 8 m/s towards 120 deg lasting 0.6 s, angles measured from gz world +x towards +y.
- **payload_add (T7).** A downward force of 25 % of the vehicle weight (mass 2.0 kg plus four rotor links, g = 9.8 m/s^2) held for the scenario window. The mass and inertia of the plant are not changed.
- **com_shift (H3).** A downward force of 15 % of the weight whose line of action is 5 cm along +y from the centre of mass, with the lever arm fixed in the world frame. This is a level-hover approximation that is valid while the vehicle holds its spawn yaw and stays near level (scenarios.md, change log).

6.3.2 Motor loss

Motor loss (T8 motor_loss_0, H4 motor_loss_2) is applied by the runner, not by the injector (bench/run_scenario.py, bench/scenarios.py). At the start of the fault segment the runner reads SIM_GZ_EC_MIN and SIM_GZ_EC_MAX of the chosen motor channel and sets both to the square root of the thrust scale times their original value. Rotor speed is MIN + (MAX - MIN) c for command c, and gz thrust is proportional to speed squared, so the thrust of that motor is the chosen fraction of nominal at every command. T8 uses motor 0 at 0.7, H4 motor 2 at 0.6. The fault changes the thrust constant only. The PX4 control allocator is not aware of the fault: SIM_GZ_EC_MIN/SIM_GZ_EC_MAX are read only by the gz bridge output mapping (src/modules/simulation/gz_bridge/GZMixingInterfaceESC.cpp), downstream of control_allocator, whose effectiveness matrix is unchanged. Both controllers therefore face an unannounced loss of effectiveness.

6.3.3 Plant variants (bench/model_variant.py)

Mismatch scenarios need a different plant, not a different disturbance. indi_project/bench/model_variant.
builds a per-run copy of the x500 and x500_base models in the run folder, with the multiplicative scales ixx, iyy, izz (base link inertia), mass (base link only, propeller link masses unchanged) and tau (motor time constants up and down, all four rotors). The runner points PX4_GZ_MODELS at the copy. T9 uses Ixx and Iyy x1.3. H5 uses inertia x0.8 on all three axes, mass x1.15 (2.3 kg as reported by gz) and motor time constants x2, i.e. 0.025 s up and 0.05 s down (D-h5, session 15). The controllers are not told about any of these changes.

6.4 Scenarios and metrics

The benchmark is specified in indi_project/bench/scenarios.md, frozen on 2026-11-13 (session 3) before any INDI code existed. Later changes were allowed only to repair a harness defect, are logged at the end of that file and apply to both controllers. All scenarios start from a 5 m hover in offboard mode. Three controllers are compared: pid_default (PX4 airframe defaults), pid_tuned and indi, the last two tuned on the tuning set with the same procedure, budget and cost function.

Set	Scenarios
Tuning (T)	T1 hover; T2 step_roll, step_pitch; T3 step_yaw; T4 rate_step_roll; T5 traj_lemniscate; T6 gust_x; T7 payload_add; T8 motor_loss_0; T9 mismatch_inertia
Held-out (H)	H1 traj_circle_fast; H2 gust_diag; H3 com_shift; H4 motor_loss_2; H5 mismatch_combined; H6 step_diag

Table 6.2: scenario sets (*bench/scenarios.md* sections 3.1 and 3.2).

Each H scenario varies the parameters of one T mechanism (H1 of T5, H2 of T6, H3 of T7, H4 of T8, H5 of T9, H6 of T2), so that tuning cannot memorise it. Held-out runs are made only for final validation and at most once per design iteration, without driving gain changes. Stage-1 tuning used T1 to T4 only, because T5 to T9 needed the extensions of 6.3 (change log, v1.1). The tuning cost is the mean, over tuning scenarios and seeds, of the primary metric divided by that of `pid_default` on the same scenario and seed, plus an effort penalty above a ratio of 1.25 and 10 per failed run.

Metrics are computed from the ULog alone by `indi_project/bench/metrics.py`, inside the window from offboard engagement to exit: rate and attitude error, step rise time, overshoot and settling, disturbance peak and recovery time, control effort (RMS of the motor-output derivative, and of the outputs about their mean), saturation time and a safety flag. A run with attitude error above 60 deg, ground contact or divergence fails, and failed runs are reported, never dropped. Before any tuning run, the step metric became RMS attitude error instead of IAE because `metrics.py` computes RMS (change log, v1.1).

Final comparisons are paired by seed (10 seeds) with a two-sided Wilcoxon signed-rank test per scenario at alpha = 0.05. INDI outperforms `pid_tuned` only if no run fails, it is significantly better by at least 10 % on at least 4 of the 6 H scenarios, it is not significantly worse by more than 10 % on any, and its hover effort is at most 1.5 times that of `pid_tuned`. The criteria were applied literally (D-accept). The outcome, given in chapter 8, is “INDI does not outperform PID”.

Two held-out scenarios exceed what the airframe and default position controller allow, and were flown unchanged, since softening a held-out scenario after seeing a result would be selection after the fact (D-h5). H1 needs about 52 deg of tilt against `MPC_TILTMAX_AIR` of 45 deg, and in H4 both controllers crash (D-accept). Part of the benchmark outcome therefore reflects scenario severity, not rate-loop quality.

6.5 Fidelity limits and their effect on the conclusions

`scenarios.md` section 7 asked for a fidelity check before T6 to T9 and H were flown. The note was written in session 17, after the benchmark (`sim_fidelity.md`). It changes no result, but it was not available when the scenarios were designed.

Sensor model. The gyro and accelerometer are white, bias-free and vibration-free, sampled at 250 Hz against 1 to 8 kHz with anti-aliasing on real IMUs. INDI differentiates the gyro, so this favours INDI; the project’s own assessment is that the disturbance-rejection margins measured here are likely optimistic for real flight (`sim_fidelity.md`, not tested). The hover-effort criterion is likewise evaluated on a clean signal.

Actuators. The plant is a first-order lag with no ESC delay and no voltage sag, and gz provides no rotor-speed feedback (`esc_rpm` echoes the command), so INDI relies on its own motor model. Its structure matches the plant, which probably helps INDI. The measured weakness under actuator mismatch (rate pulses 1.58 times worse in H5, D-accept) must be read with this in mind.

Latency. End-to-end sensor-to-actuator latency in SITL was not characterised; lockstep makes it one or two control cycles (`sim_fidelity.md`). No experiment in this project varies the delay, so the sensitivity of INDI to added transport delay (section 3.5) is not quantified here.

Aerodynamics and disturbances. Gusts and payload are external wrenches on a rigid body, with assumed drag coefficient, area and centre-of-pressure height (6.3.1). There is no rotor wake, ground effect or blade flapping (`sim_fidelity.md`). The payload and CoM scenarios apply a force and leave plant mass and inertia unchanged; only H5 changes them.

x500_base/model.sdf contains no plugin at all, and x500/model.sdf adds only four gz-sim-multicopter-motor-model-system plugins and the MotorFailurePlugin, so there is no aerodynamic model beyond the rotor drag terms of the motor plugin. The moment-constant mismatch (k_m/k_f 0.016 in gz, 0.05 in the allocator) is common to both controllers.

Run-to-run non-determinism (D-hovernoise). Re-flying one seed gives hover position RMS up to about 2 times apart, while attitude and rate step metrics re-fly within about 2 % (D-hover-noise, session 9). The suspected cause is thread scheduling between PX4 and Gazebo under lockstep, which the seed does not control; this was not tested (report/07_tuning.md). In session 13 the hover term of the tuning cost was found to be dominated by sporadic vertical excursions, present for both controllers, with tight and equal horizontal RMS. The frozen cost was kept, J differences below about 0.1 are not treated as meaningful, and hover is reported across seeds with a horizontal and vertical split. The cause of the excursions is open, and whether they are related to the speed-factor effect of 6.1.4 was not tested.

Effect on the conclusions. Both controllers fly the same plant, noise, disturbances and seeds, so the comparison is fair, and the negative acceptance outcome is not weakened by simulator optimism: a noisier real gyro would, if anything, hurt INDI. The positive finding, INDI's disturbance-rejection gain, is the one most exposed to the idealised gyro and motor model and should be read as an upper bound. Differences in hover and position metrics cannot be resolved with the seeds used. Nothing here supports a claim about real flight; a hardware test would have to measure the noise floor and vibration, ESC and sensor latency and the true motor response, none of which was done.

7. Tuning methodology and iteration

This chapter shows how the two controllers were tuned, so a reader can judge whether the PID baseline was tuned fairly and how INDI reached its frozen candidate. The headline: after four tuning rounds and three structural firmware fixes, INDI and PID tie on the tuning cost, J = 0.978 for INDI against about 0.96 for PID, a difference inside the noise of the hover term (results/tuning/r4f.md; DECISIONS.md D-freeze). On the tuning set this is a tie, not a win. Paths are relative to indi_project/. Held-out and disturbance results are in later chapters.

Sources are cited in parentheses as paths relative to indi_project/. The chapter covers only the tuning set; the held-out and disturbance results are reported in later chapters.

7.1 Tuning principles

Initial values come from the model and identification. The INDI control effectiveness G1 is identified, not tuned: roll = pitch = 241 from in-loop fits of 237 and 245 (R^2 0.94 to 0.95), yaw 7.4 from fits of 7.2 to 7.6. An open-loop fit on PID logs (321 and 211) was rejected because it disagreed between two axes of a symmetric vehicle (DECISIONS.md D-g1-v1). Round 1 started from K_R = 20, K_P = 20, K_Y = 10, actuator time constant 0.02 s, yaw cut-off 10 Hz and attitude P gains 4, 4, 2.8 (results/tuning/indi/r1/c00/params), the time constant being near the Gazebo motor constants of 0.0125 s up and 0.025 s down (results/tuning/indi_r1.md). The PID baseline starts from the PX4 defaults; the gz_x500 airframe file sets no rate or attitude gains (results/tuning/pid_r1.md).

Same procedure, same budget. The fairness rule requires the same sweep design, number of evaluations, cost and scenarios for pid_tuned and indi, with both tuning logs kept (bench/scenarios.md sec. 2). One script, bench/tune.py, serves both families (plan/tuning_driver_spec.md). It draws a Latin hypercube in log space, each parameter in [centre / span, centre x span] clamped to the PX4 limits, candidate 0 being the centre. Each candidate is flown on every scenario and seed through the same runner and metric code

as the plain batches, and rounds are resumable (`bench/tune.py` docstring). This is random search, identical for both controllers; its limit is discussed in 7.6.

The spaces differ in size, which is inherent to the controllers. In `bench/tune.py` PID has 12 parameters (roll and pitch rate P, I, D; yaw rate P, I; MC_YAW_TQ_CUTOFF; three attitude P gains) and INDI has 8 (K_R, K_P, K_Y, MC_INDI_ACT_TC, MC_INDI_YAW_CUT, three attitude P gains). Both yaw filters were put in on purpose, because PID already has a 2 Hz yaw torque low-pass (`DECISIONS.md` D-yawfilt, D-indi-r1).

Tuning versus held-out set. The tuning set T (T1 to T9) is used for tuning and development; the held-out set H (H1 to H6) is never used for tuning, and each H scenario is a variant of a T mechanism with other parameters so tuning cannot memorise it. H runs are allowed only for the final validation and at most one intermediate check per iteration, which must not drive gain changes (`bench/scenarios.md` sec. 3.2).

Stage-1 scope. Scenarios T5 to T9 needed simulator extensions that did not yet exist, so stage-1 tuning used T1 to T4, with a planned re-tune of both controllers on the full T set later (`bench/scenarios.md`, change log v1.1). The five scenarios used are `hover`, `step_roll`, `step_pitch`, `step_yaw` and `rate_step_roll`, with primary metrics position RMS 3D, RMS attitude error of the stepped axis, and roll-rate RMS (`bench/tune.py`). The full-T re-tune was never run (`DECISIONS.md` D-noretune); see Inconsistencies.

Cost J. Per (scenario, seed): the ratio of the primary metric to the `pid_default` reference flight on the same scenario and seed, plus 0.2 x (effort ratio - 1) when the motor-rate effort ratio exceeds 1.25, plus 10 for a failed run; J is the mean (`bench/scenarios.md` sec. 5; `bench/tune.py`). J = 1 means "same as the PX4 default reference flights", lower is better. The cost was frozen before INDI flew and never changed, since changing it after seeing INDI lose would be after-the-fact selection (`DECISIONS.md` D-yawsp).

Seeds and speed factor. Round 1 used 2 seeds, later rounds 3, to average the non-repeatable hover metric (`DECISIONS.md` D-r2). Attitude and rate scenarios run at speed 4 and hover at speed 1: on `pid_default` seeds 1 to 3, step metrics agree between speeds within the seed spread, but hover position RMS does not, 0.027 m (0.021 to 0.033) at speed 1 against 0.064 m (0.038 to 0.104) at speed 4, cause open (`DECISIONS.md` D-speed). **Stopping rule:** several iterations without improvement, with an explanation of the remaining gap (`DECISIONS.md` D-freeze).

7.2 PID baseline retuning

The PID baseline was given two sampling rounds with the shared driver, centred on the PX4 defaults.

Round	Candidates	Seeds	Flights	Span	Best J	PX4 defaults (c00) J	Source
r1	24	2	240	2	0.946 (c00 itself)	0.946	`results/tuning/pid_r1`
r2	16	3	240	1.5	0.958 (c10)	0.964 (re-flown)	`results/tuning/r2.md`

Both rounds had 0 failures (`results/tuning/pid_r1.md`; `results/tuning/r2.md`). Over the two rounds that is 40 candidates (24 + 16, counted from `results/tuning/pid/r1/summary.csv` and `results/tuning/pid/r2/summary.csv`) and 480 flights.

Round 1. No sampled candidate beat the defaults in a factor-2 box; the second best, c12, scored 0.978 and the median of 24 was 1.060. Some candidates improved one axis (step_roll down to 0.78) at the expense of others (`results/tuning/pid_r1.md`). The round also exposed the hover problem of 7.4: the reference gains themselves scored a hover ratio of 0.752 instead of about 1.

Round 2. The best candidate, c10, scored J = 0.958 against 0.964 for re-flown defaults, a 0.6 % gain inside the roughly 2 % run-to-run noise. Its ratios were hover 0.876, step_roll 0.873, step_pitch 0.945, step_yaw 1.059, rate_step_roll 0.954; the worst candidate scored 1.145. c10 (roll P 0.1681, pitch P 0.122, attitude P 5.142 and 4.509; `results/tuning/pid/r2/c10/params`) gains on roll and pitch but loses on yaw, and was noted for a confirmation re-fly, not adopted (`results/tuning/r2.md`).

Outcome. `pid_tuned` is the PX4 default gains (`results/tuning/r2.md`; `DECISIONS.md` D-freeze), with the effort documented as two rounds, 40 candidates, 480 flights. The defaults are at, or within noise of, the optimum of this cost inside the sampled boxes (`results/tuning/r2.md`). Because `pid_tuned` equals `pid_default`, the benchmark effectively compares INDI against PX4 as shipped. This claim is limited by the search design, see 7.6.

7.3 INDI tuning rounds r1 to r4f

INDI went through four sampling rounds (r1 to r4) and four single-candidate re-scores after a firmware change (r2f_yc2, r2f_yawnu, r3f_yawsp, r4f). J is the frozen cost over the five tuning scenarios, except for r4, which flew only step_yaw and hover and is ranked by its step_yaw cost. “PID J” is the best or default PID value from the matching round. Cells marked “n/a” are not given in the cited file.

Round	Candidates x seeds	What was varied	Best INDI J	PID J	Key finding	Source
r1	24 x 2 (240 flights)	all 8 parameters, centre K 20/20/10, ACT_TC 0.02, YAW_CUT 10 Hz, span 2	1.167 (c07)	0.946	INDI tracks better than PID on all five scenarios but pays 0.32 of J in effort	`results/tuning/indi_r1`
r2	16 x 3 (240)	all 8, centre c07 with YAW_CUT 3.5 Hz, span 1.5	1.150 (c11)	0.958 (best), 0.964 (defaults)	effort still 2.3 to 2.6x PID; traced to the yaw path	`results/tuning/r2.md`
r2f_yc2	1 x 3 (15)	c11 with YAW_CUT 2 Hz (PID’s value)	1.182	0.958 to 0.964	a_f chatter gone, hover effort still ~2.1x: remaining source is the nu term	`results/tuning/r2f_yc2`

Round	Candidates x seeds	What was varied	Best INDI J	PID J	Key finding	Source
r2f_yawnu	1 x 3 (15)	same candidate, firmware D-yawnu	1.197	0.958 to 0.964	hover effort 1.07 to 1.12x, but step_yaw effort 6.65 to 7.79x	`results/tuning/r2f_yawn`
r3	16 x 3 (240)	all 8, centre K_Y 3.0, YAW_P 4.0, span 1.5	1.055 (c13)	0.958 to 0.964	step_yaw effort 4.66 to 5.89x, structural (unfiltered setpoint path)	`results/tuning/r3.md`
r3f_yawsp	1 x 3 (15)	c13, firmware D-yawsp	1.058	0.958 to 0.964	step_yaw effort 1.42x, yaw ratio 0.876 to 1.056; hover outlier 1.306	`results/tuning/r3f_yaws`
r4	12 x 3 on 2 scenarios (72)	K_Y, YAW_CUT, YAW_P only, span 2 around c13	n/a (best step_yaw cost 1.089, c10)	n/a	no yaw candidate beats PID at acceptable effort	`results/tuning/r4.md`
r4f	1 x 3 (15)	c13 roll/pitch + c10 yaw, firmware D-yawsp	0.978	0.958 to 0.964	tie within hover noise; frozen	`results/tuning/r4f.md`

Summing the flight counts in the table gives 852 flights (the four 15-flight re-scores included). The decision log states "~600 flights" for r1 to r4 (`DECISIONS.md` D-freeze); the three 240-flight sampling rounds plus r4's 72 flights alone give 792. See Inconsistencies.

How each surprise was diagnosed before gains changed

The pattern in every step below is the same: INDI scored worse than PID under the frozen cost, the effort term was identified as the reason, the effort was measured per axis on logs, and a structural change was made only when the measurement located a specific cause. Gain sampling alone never closed the gap.

Hover effort 44x PID, then 6x (sessions 7 to 8). With the identified G1_Y = 7.4, yaw acceleration noise reaches the yaw torque through 1/G1_Y, four times more than in the earlier v0 setup that used 30. Hover motor-rate effort was 1.616 against 0.037 for PID, about 44x (`results/yawcut_check_s8.md`). The fix was a yaw low-pass inside the INDI law applied identically to the angular-acceleration and torque feedback paths so they stay synchronised (`DECISIONS.md` D-yawfilt). At 10 Hz this cut hover effort to 0.228, about 6x

PID (`results/yawcut_check_s8.md`). Inflating G1_Y to hide the noise was rejected because it detunes the cancellation (`DECISIONS.md` D-yawfilt).

Round 1. The best INDI candidate had a lower tracking ratio than PID on all five scenarios (yaw step -15 %, roll step -17 %, pitch step -9 %, hover -10 % relative to PID c00) but J = 1.167 against 0.946, about 0.32 of it effort penalty (`results/tuning/indi_r1.md`). The top candidates sat at the low edge of the yaw cut-off box.

Which axis? (session 9). Hover torque statistics per axis showed roll and pitch activity about 2x PID's per sample, but yaw torque standard deviation 9x PID's (0.0027 against 0.0003 at 10 Hz). With G1_yaw = 7.4 against 241, each rad/s^2 of yaw acceleration noise becomes 33x more torque (`results/effort_axes_s9.md`). Round 2 therefore widened the yaw cut-off box toward PID's 2 Hz (`DECISIONS.md` D-r2).

Round 2 did not close the gap. J improved only 1.5 % (1.167 to 1.150) and effort stayed at 2.3 to 2.6x PID: a noise-transmission problem, not a gain-tuning one (`results/tuning/r2.md`). A matched-filter control (YAW_CUT = 2 Hz) followed.

Matched filter and the unfiltered rate term (session 11). The 2 Hz cut removed the a_f chatter (yaw torque sample-to-sample change 0.00018 to 0.00004) but hover effort stayed ~2.1x PID. The term split located the rest in nu = K_Y (r_sp - r) built from the raw gyro rate, with gain K_Y / G1_Y = 5.83 / 7.4 = 0.79, against PID's yaw P of 0.2 behind a 2 Hz output filter (`results/tuning/r2f_yc2.md`). D-yawnu filters that rate with the same cut-off; filtering the whole increment was rejected because it adds lag in series with the actuator and breaks synchronisation (`DECISIONS.md` D-yawnu). Hover effort became 0.041 to 0.043 against 0.037 for PID (`results/yawnu_check_s11.md`).

Hover fixed, J unchanged (r2f_yawnu). Hover effort fell to 1.07 to 1.12x, under the 1.25 threshold, but J rose to 1.197. The remaining penalty was almost entirely step_yaw transient effort at 6.65 to 7.79x PID, worth about 0.24 of J, while yaw tracking was only about 10 % better than PID (`results/tuning/r2f_yawnu.md`). The file labels the cause, an unfiltered setpoint step, as "probable, not yet verified".

Verified before acting (round 3). `bench/yaw_step_effort.py` on step_yaw seed 1 showed the setpoint term K_Y r_sp / G1_Y peaking at 2.75 normalised torque, saturating the yaw torque at 1.0 against 0.33 for PID; motor-rate RMS along yaw was 11.1 against 1.55. The yaw loop crossover was near K_Y = 5.8 rad/s against about 1.5 for PID (0.2 x 7.4), so INDI had been tuned about 4x faster in yaw (`DECISIONS.md` D-r3). Round 3 was centred at K_Y 3.0 and YAW_P 4.0, a region never sampled while hover effort dominated the cost, and J improved from 1.197 to 1.055. The yaw torque no longer saturated (peak 0.78) but still jumped at the setpoint step, which PID's 2 Hz output filter smooths; no gain in the box removed it (`results/tuning/r3.md`).

Structural fix D-yawsp, and its price. YAW_CUT now filters the yaw rate error including the setpoint, which gives INDI the same structure that PID already has by default (`DECISIONS.md` D-yawsp). A separate setpoint pre-filter was rejected as an INDI-only advantage, and changing the effort metric was rejected as after-the-fact selection (`DECISIONS.md` D-yawsp). Result: step_yaw effort fell from 4.66 to 5.89x to 1.42x, and yaw tracking ratio rose from 0.876 to 1.056 (`results/tuning/r3f_yawsp.md`). INDI paid the same lag PID pays.

Round 4: the yaw frontier. With D-yawsp no candidate beat PID on step_yaw at acceptable effort: a ratio of 0.98 needed 3.8x effort (c11), and at no more than 1.25x effort the best ratio was 1.05 to 1.09. The yaw loop is now a P-type rate loop behind a 1.5 to 2 Hz filter, as in PID, so near-parity is expected, and the weak yaw authority (G1_Y = 7.4) leaves little for inversion to gain (`results/tuning/r4.md`). The selection rule was fixed before the results; c10 won it and was combined with the c13 roll and pitch gains (`DECISIONS.md` D-r4). Fairness consequence: INDI received more rounds and structural changes than PID, because PID needed none.

7.4 The hover-noise problem and how it limits J

Hover position RMS is not repeatable per seed. The same `pid_default` hover flights re-flown with seeds 1 to 3 gave 0.036, 0.037 and 0.046 m against the reference 0.033, 0.027 and 0.021 m, ratios 1.08, 1.38 and 2.19 (`results/tuning/pid_r1.md`). Step and rate metrics re-fly within about 2 % (`DECISIONS.md` D-hover-noise). The likely source is thread scheduling between PX4 and Gazebo in lockstep, which the seed does not control; this is a hypothesis, not a measurement (`DECISIONS.md` D-hover-noise).

Consequences for the tuning cost:

- The hover ratio of the reference's own gains is biased below 1: PID c00 scored 0.752 in r1 and 0.856 in r2 (`results/tuning/pid_r1.md`; `results/tuning/r2.md`). This is why J below 1 must not be read as "better than PX4 defaults", and why PID's own J is 0.946 to 0.964 and not 1.
- Identical parameters and seeds gave very different hover ratios in different rounds. INDI c13 scored 0.740 in r3 (earlier firmware), 1.306 in r3f_yawsp (one run at 0.067 m with other seeds at 0.017 and 0.026 m) and 0.767 in r4, the last two with the same firmware (`results/tuning/r3.md`; `results/tuning/r3f_yawsp.md`; `results/tuning/r4.md`). Similarly the c10 yaw set scored 0.782 in r4 and the same parameters 0.945 in r4f (`results/tuning/r4.md`; `results/tuning/r4f.md`).
- The 3-D hover RMS is driven by sporadic vertical excursions that occur for PID too. The horizontal RMS, the part attitude control acts on, is tight and equal: median 0.0204 m (max 0.0253) for the 48 PID r2 candidates and 0.0194 m (max 0.0242) for the 48 INDI r3 candidates; vertical medians are 0.0075 m and 0.0072 m but the maxima reach 0.0699 m and 0.0410 m (`results/tuning/r3f_yawsp.md`). The cause of the excursions is open.
- With 3 seeds, one such run moves the hover ratio by about 0.5 and J by about 0.1 (`results/tuning/r3f_yawsp.md`). The tuning rule that follows: J differences below about 0.1 are not meaningful (`DECISIONS.md` D-hovernoise). The tie of INDI at 0.978 with PID at about 0.96 is a 0.02 gap, well under that threshold.

The remedy chosen was to average hover over more seeds (3 in tuning, at least 8 in the final benchmark) and to report the horizontal and vertical split next to the frozen metric (`DECISIONS.md` D-hover-noise, D-hovernoise). The frozen cost was left unchanged.

7.5 Frozen candidates and per-scenario ratios

PID tuned is the PX4 default set (`results/tuning/pid/r2/c00/params`): roll and pitch rate P 0.15, I 0.2, D 0.003; yaw rate P 0.2, I 0.1; MC_YAW_TQ_CUTOFF 2 Hz; attitude P 4, 4 and 2.8 for roll, pitch and yaw.

INDI tuned is `results/tuning/indi/r4f/c00/params`, frozen by D-freeze (`DECISIONS.md`):

Parameter	Value	Origin
MC_RATE_CTL_TYPE	1	selects INDI
MC_INDI_G1_R, MC_INDI_G1_P	241, 241	identified, not tuned
MC_INDI_G1_Y	7.4	identified, not tuned
MC_INDI_K_R	23.89	r3 c13
MC_INDI_K_P	12.29	r3 c13
MC_INDI_K_Y	2.118	r4 c10
MC_INDI_ACT_TC	0.01142 s	r3 c13
MC_INDI_YAW_CUT	1.572 Hz	r4 c10
MC_ROLL_P, MC_PITCH_P	6.275, 6.185	r3 c13
MC_YAW_P	3.308	r4 c10

(Parameter values: `results/tuning/indi/r4f/c00/params`; origin: `results/tuning/r3.md`, `results/tuning/r4.md`, `results/tuning/r4f.md`.) The firmware is the D-yawsp build, commit 99634a6b71 (`results/tuning/r4f.md`).

Ratios to PID defaults on the tuning set, mean over seeds 1 to 3, lower is better (`results/tuning/r4f.md`):

	J	hover	step_roll	step_pitch	step_yaw	rate_step_roll
PID defaults (r2)	0.958 to 0.964	1	1	1	1	1
INDI r4f	0.978	0.945	0.878	0.935	1.096	1.021
INDI motor-rate effort ratio		0.99-1.03	1.16-1.26	0.63-0.71	1.11-1.15	0.67

The reading is: INDI tracks roll steps 12 % and pitch steps 7 % better (pitch with about 30 % less effort), equals PID on rate pulses (+2 %, with 33 % less effort) and on hover, and is 10 % worse on yaw steps, a structural consequence (`results/tuning/r4f.md`; `results/tuning/r4.md`). The hover ratio of 0.945 is not a meaningful advantage given 7.4. The PID row is the reference itself, so its 1.0 entries are by construction; its J of about 0.96 is below 1 only because of the hover reference noise (7.4). Per-scenario ratios on the held-out and disturbance scenarios are outside the sources of this chapter and are not quoted here.

7.6 Threats to validity

- **Noise.** Hover is non-repeatable (7.4), and J differences under about 0.1 are not meaningful (`DECISIONS.md` D-hovernoise). The tie, 0.978 against about 0.96, cannot rank the controllers either way. Step and rate metrics are more stable, about 2 % re-fly noise (`results/tuning/r2.md`).
- **Few seeds.** Tuning used 2 seeds in round 1 and 3 afterwards, and the final re-score r4f used 3. Candidate differences of a few percent per scenario are within that noise. The benchmark's statistical test (10 seeds, paired Wilcoxon) applies only to the final validation (`bench/scenarios.md` sec. 6), not to tuning.
- **Tuning on five scenarios only.** The frozen parameters were optimised for hover, three attitude steps and one rate pulse series. Gust, payload, motor-loss, trajectory and mismatch scenarios (T5 to T9) played no role in tuning in the documented rounds, and the held-out scenarios were not used. This is safe against leakage but means the candidates may be suboptimal where INDI is expected to differ from PID, namely disturbance rejection and plant change (`bench/scenarios.md` sec. 1).
- **Random search, small budget.** The PID search was 24 candidates in a 12-dimensional box and 16 in a narrower one. "No candidate beat the defaults" supports "the defaults are hard to beat by random sampling", not "the defaults are the optimum". A per-axis or gradient-based search might find a better PID, and PID candidate c10 improved roll and pitch steps by 13 % and 5.5 % at a cost on yaw (`results/tuning/r2.md`). The same limit applies to INDI.
- **Unequal effort.** PID got two rounds and 480 flights; INDI got four rounds, three structural fixes and 852 flights. The fairness rule asks for equal budgets (`bench/scenarios.md` sec. 2); PID stopped early because defaults won, but a reader may still view this as an asymmetry in INDI's favour.
- **The cost is relative to the default reference.** J is a ratio to `pid_default` flights, so the PID default is advantaged by construction on J = 1, and the effort penalty only charges noise above 1.25x. The cost function was frozen before INDI flew and never changed.

- **Structural changes were made on the tuning set and affect INDI only.** D-yawfilt, D-yawnu and D-yawsp were motivated by effort measured on tuning scenarios. They were justified as giving INDI the same filtering structure PID already has, but the PID baseline did not get an equivalent search over its own structure (for example its D-term filtering).
- **Simulator limits.** Hover non-determinism of unknown origin, and a speed factor of 4 validated only on `pid_default` seeds 1 to 3 (`DECISIONS.md` D-speed).

Inconsistencies and unverified claims

1. **PID candidate count (resolved).** `DECISIONS.md` D-freeze and `results/tuning/r2.md` once said "64 candidates over two rounds"; the summary files hold 24 + 16 = 40 rows (`results/tuning/pid/r1/summary.csv`, `results/tuning/pid/r2/summary.csv`), consistent with the 480 flights. Both sources were corrected to 40 in session 17.
2. **INDI flight count.** D-freeze says "~600 flights" for r1 to r4 (`DECISIONS.md`); the per-round counts in the result files give 792 for the four rounds and 852 with the four re-scores.
3. **PID default hover effort.** `results/yawcut_check_s8.md` gives 0.037 1/s for `pid_default` hover (seed 1), and `results/effort_axes_s9.md` gives 0.054 for the same scenario and seed. The "44x" and "6x PID" ratios of 7.3 use 0.037; with 0.054 they would be about 30x and 4x.
4. **PID J.** The PID value is quoted as 0.96 (`DECISIONS.md` D-freeze), 0.946 (r1, `results/tuning/pid_r1.md`), 0.958 for the best and 0.964 for re-flown defaults (r2, `results/tuning/r2.md`), and "0.958 to 0.964" (`results/tuning/r4f.md`). The differences come from hover noise, not from different parameters.
5. **Same parameters, different J.** INDI c13 hover 0.740, 1.306, 0.767 and c10 hover 0.782 against 0.945 (7.4). These are same-parameter re-flights and are reported as noise by the source files, not corrected.
6. **Tuning space size.** `plan/tuning_driver_spec.md` lists 11 PID and 7 INDI dimensions; `bench/tune.py` has 12 and 8 because MC_YAW_TQ_CUTOFF and MC_INDI_YAW_CUT were added (`DECISIONS.md` D-yawfilt).
7. **Planned re-tune on the full T set.** `bench/scenarios.md` (v1.1) says both controllers are re-tuned on the full T set once T5 to T9 exist. It was not run: D-freeze moved directly to the held-out scenarios, and once those had been flown a re-tune would have been informed by held-out results, so it was dropped (`DECISIONS.md` D-noretune). Both controllers are therefore tuned on T1 to T4 only.
8. **Unverified causal claims in the sources.** The scheduling explanation of hover non-determinism, the cause of vertical hover excursions, and the explanation of the c07 and c08 yaw behaviour in `results/tuning/r4.md` ("probably a lag-limited yaw oscillation, not checked") are hypotheses, not measurements.

8. Results

8.1 Experimental protocol

Plant and scenarios. All flights are PX4 SITL with the Gazebo `gz_x500` model. Scenarios, metrics and the four acceptance criteria were frozen before any INDI code existed (`bench/scenarios.md`, v1, 2026-11-13). The tuning set is T1-T9 and the held-out set is H1-H6. Each H scenario is a variant of a T mechanism with different parameters (`bench/scenarios.md`, section 3.2). Held-out scenarios were never used to choose gains. Several T scenarios (T5-T9) also turned out not to be used for tuning in practice, because the simulator extensions they need did not exist when tuning ran (`bench/scenarios.md`, change log; `results/t8_motor_loss_s13.md`; `results/heldout_s15.md`).

Controllers.

Label	What it is	Source
PID (`pid_default`)	PX4 defaults for `gz_x500`, untouched	`bench/scenarios.md` section 2
INDI (`indi_tuned`)	Frozen candidate `results/tuning/indi/r4f/c00/params` (decision D-freeze)	`results/tuning/r4f.md`

The spec also defines a `pid_tuned` baseline, against which the acceptance criteria are written. No retuned PID candidate beat the defaults: the best retuned PID scored J = 0.958 against 0.964 for the re-flown defaults, which is inside noise (40 candidates over rounds r1 and r2; `STATE.md`, session 10; `results/tuning/r2.md`). `acceptance_s15.md` therefore uses the PX4 defaults as `pid_tuned`. Parameters outside the rate and attitude loops (position control, estimator, allocator, failsafes) are identical for both controllers. Tuning is described in chapter 7 (`report/07_tuning.md`) and is not repeated here.

Seeds and statistics. Final validation uses 10 seeds per scenario and controller, with the same seed list for both (paired design; `bench/scenarios.md`, section 3.3). The seed varies simulator sensor noise, and gust onset and direction jitter where the scenario has them. It does not vary the trajectory start phase, which is a deviation from the spec (`bench/scenarios.md`, change log, session 15). For each scenario the test is a two-sided exact Wilcoxon signed-rank test on the per-seed differences of the primary metric, alpha = 0.05 (`bench/scenarios.md`, section 6; `bench/summarize_seeds.py`). With n = 10 the smallest attainable p-value is 0.00195 (printed as 0.002 in several tables), so p = 0.002 with "10/10" means INDI was better (or worse) on every seed. With n = 5 the smallest p would be 0.0625, and with n = 3 it would be 0.25 (arithmetic, not stated in the sources for n = 3). Tables below give mean ± std over seeds, the ratio INDI/PID, and "INDI lower" as the number of seeds on which INDI's value is smaller (lower is better for every error metric).

Simulation speed. Scenarios whose metrics depend on position run at real-time speed (factor 1). Attitude and rate scenarios run at factor 4, because factor 4 is equivalent to factor 1 for attitude and rate metrics but not for hover position RMS (0.027 m against 0.064 m; decision D-speed, `STATE.md` session summary). Concretely, H6 `step_diag` and T9 `mismatch_inertia` run at factor 4 (`results/heldout_s14.md`), H5 at factor 4 (`results/heldout_h5_s15.md`), and the disturbance, trajectory and hover scenarios at factor 1 (`results/heldout_s14.md`, `results/heldout_s15.md`).

Primary metrics. For the disturbance and diagonal-step scenarios the sources use the attitude rotation-angle RMS over the scenario window (`att rot rms`) as the primary metric. T1 uses position RMS, T4 and H5 rate pulses use roll-rate RMS, and trajectory scenarios use position RMS against the logged `trajectory_setpoint` (`bench/scenarios.md` change log v1.1; `results/heldout_s15.md`). Whole-window RMS values are used throughout; peak deviation and recovery time are reported only where the sources give them (T8 fault onset, position maximum deviation).

Limits of the data.

- Not every number comes from 10 seeds. T8 motor loss was first summarised on seeds 1-3 (`results/t8_motor_loss_s13.md`); its 10-seed values (`results/t8_motor_loss_10seeds_s26.md`, same firmware and parameters, seeds 4-10 flown in the same session) are used here. The tuning-set step scenarios in `results/tuning/r4f.md` use 3 seeds.
- Hover position RMS is noise-dominated: the same parameters and seeds gave hover scores of 0.77 and 1.31 in two batches, because of sporadic vertical excursions in both controller families (decision D-hovernoise, `STATE.md` session 13). Hover position results are therefore not used to rank the controllers.

- Position rows in attitude-mode and rate-mode scenarios (step_diag, mismatch_inertia, H5) are meaningless, since the vehicle drifts tens of metres without position hold. They are ignored, though they appear in the source tables (`results/heldout_s14.md`, `results/heldout_h5_s15.md`).
- The trajectory table computation was an inline script that is not yet in `bench/` (`results/heldout_s15.md`, TODO in that file).
- The simulator motor model, delay and noise are those of `gz_x500` with the extensions listed in `bench/scenarios.md`. All conclusions are simulation-only.

8.2 Acceptance verdict

Outcome under the frozen rules: INDI does not outperform PID (`results/acceptance_s15.md`). All six held-out scenarios were flown with 10 paired seeds each, using the frozen candidates. The spec's three possible outcomes (outperforms, matches, does not outperform) are all written up with the same rigour, and "matches" is available only if criteria 1 and 3 hold (`bench/scenarios.md`, section 6). Here criteria 1 and 3 both fail.

#	Criterion (`bench/scenarios.md` section 6)	Result	Met?
1	No INDI run fails in any H or T scenario	INDI crashes on H4 on 10/10 seeds (PID also 10/10; the two behave alike until impact), but the criterion is stated for INDI alone and is applied as written	**No**
2	Significantly better (median improvement >= 10 % and p < 0.05) on at least 4 of 6 H scenarios	H1, H2, H3 clearly; H6 and the T2 part of H5 exactly at the 10 % threshold	Yes (marginally)
3	Not significantly worse by more than 10 % on any H scenario	H5 rate pulses: roll-rate RMS 1.58x PID, median improvement -58 %, p = 0.002, 0/10 seeds better	**No**
4	Hover motor-rate effort (T1) at most 1.5x `pid_tuned`	0.0387 against 0.0371 /s, ratio 1.04, p = 0.16	Yes

Per held-out scenario, from `results/acceptance_s15.md`:

H	Primary metric	INDI/PID	Median improvement	p	Verdict
H1 traj_circle_fast	position RMS	0.73	about 27 %	0.002	better, but the path is infeasible for both controllers
H2 gust_diag	attitude RMS	0.57	44 %	0.002	better
H3 com_shift	attitude RMS	0.16	84 %	0.002	better
H4 motor_loss_2	position RMS	1.15	not given	0.11	not significant; both crash 10/10
H5 mismatch_combined, T2 part	stepped-axis attitude RMS	0.91	10 %	0.002	borderline better, with a 5-7 Hz limit cycle
H5 mismatch_combined, T4 part	roll-rate RMS during pulses	1.58	-58 %	0.002	**significantly worse**
H6 step_diag	attitude RMS	0.92	10 %	0.002	borderline better (exactly at the threshold)

(The H2 and H3 ratios come from `results/heldout_s14.md`; `acceptance_s15.md` leaves them blank.)

Two points about reading the verdict. First, criterion 2 counts H5 as one scenario but H5 has two parts with opposite outcomes, and the spec does not say how to combine them. Counting H5 as a win would be wrong given its limit cycle, and even without H5 the count of wins is H1, H2, H3, H6 = 4, at the threshold. Second, the verdict does not depend on H4: `acceptance_s15.md` states that even setting H4 aside as infeasible for both controllers, the outcome is unchanged, because criterion 3 fails on its own through H5.

Results against PID are summarised in the source as follows. INDI is much better at rejecting disturbance torques (27-84 % lower attitude RMS on H2, H3, T6 and T7, and 5.7x lower in the T8 fault window; the T8 whole-window RMS is 14 % worse because of the yaw step) and slightly better at tracking, but it is fragile to an error in its actuator model (motor time constant x2 gives a limit cycle), which PID tolerates (`results/acceptance_s15.md`). That trade-off is the main finding of this thesis.

8.3 Disturbance rejection

The disturbance scenarios test the mechanism INDI is expected to win on: a disturbance torque appears in the measured angular acceleration and is cancelled within one control increment, whereas PID has to wait for its integrator (`results/heldout_s14.md`). All rows below are 10 paired seeds, PID defaults against the frozen INDI candidate, exact two-sided Wilcoxon (`results/heldout_s14.md`, 120 flights, 0 failures). Scenarios: T6 `gust_x` (3 m/s steady wind plus a 6 m/s, 1 s gust), T7 `payload_add` (+25 % mass), H2 `gust_diag`, H3 `com_shift`, T9 `mismatch_inertia` (Ixx, Iyy x1.3, run on the T2 steps; a plant-mismatch scenario rather than a disturbance).

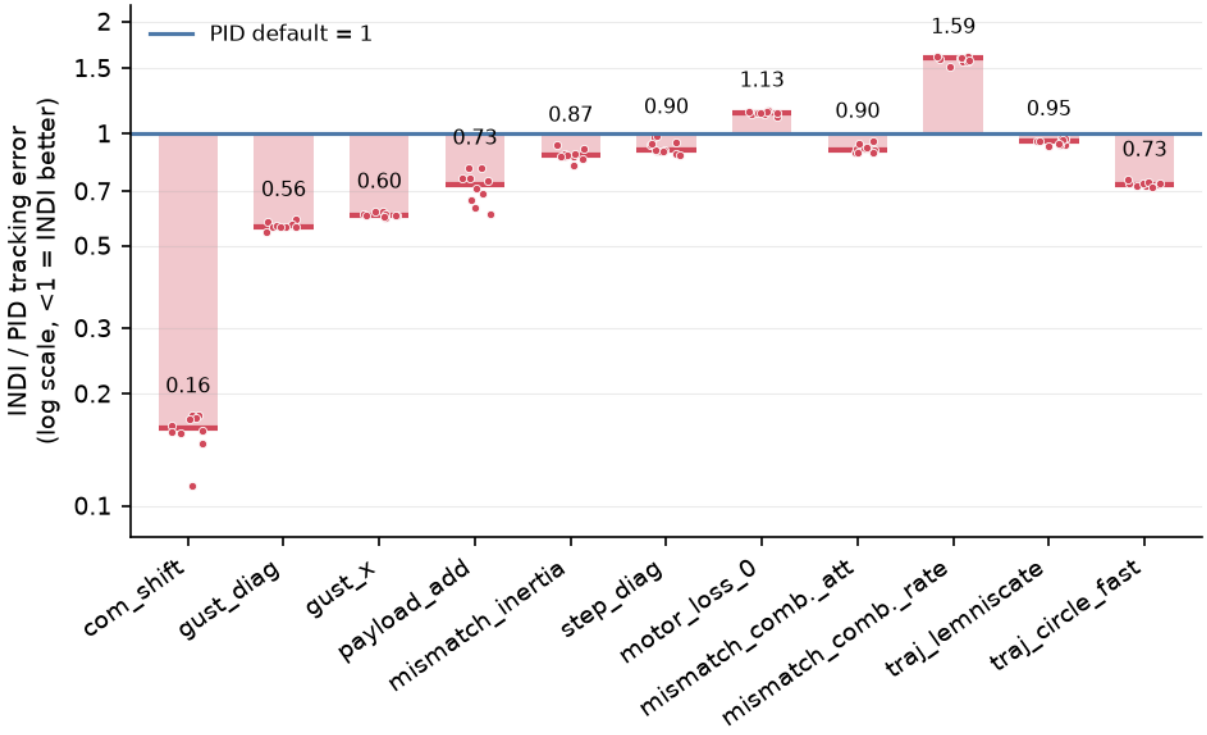


Figure 1: Figure 8.1: INDI/PID ratio of the primary tracking metric per scenario, paired by seed (10 seeds each). Dots are seeds, bars and numbers are the median ratio, the line at 1 is PID; below 1 INDI is better.

Figure 8.1 summarises the primary-metric ratios of every held-out and disturbance scenario of this chapter; the bars are medians of the per-seed ratios, so they can differ in the last digit from the ratios of means in the tables. It is generated by `bench/make_figures.py` (`figures/README.md`).

Attitude rotation-angle RMS (primary metric), deg, mean ± std:

Scenario	PID	INDI	INDI/PID	INDI lower	Median impr.	p
T6 gust_x	1.19 ± 0.007	0.717 ± 0.0096	0.60	10/10	40 %	0.002
H2 gust_diag	1.05 ± 0.0091	0.593 ± 0.014	0.57	10/10	44 %	0.002
T7 pay-load_add	0.0903 ± 0.0069	0.0646 ± 0.0052	0.72	10/10	27 %	0.002
H3 com_shift	0.566 ± 0.063	0.0889 ± 0.0049	0.16	10/10	84 %	0.002
T9 mis-match_inertia	3.33 ± 0.071	2.92 ± 0.069	0.88	10/10	13 %	0.002

The absolute attitude errors on T7 and H3 are tenths of a degree or less, so the large ratios there describe small quantities. The largest gain is H3 on the roll axis (0.559 against 0.0719 deg, 0.13x; `results/heldout_s14.md`).

Position metrics (horizontal RMS and 3D maximum deviation, m). Position gains are much smaller than the attitude gains and are not significant for T7:

Scenario	Metric	PID	INDI	INDI/PID	INDI lower	p
T6 gust_x	pos RMS horiz	0.106 ± 0.0016	0.089 ± 0.0019	0.84	10/10	0.002
T6 gust_x	pos max dev 3D	0.602 ± 0.01	0.482 ± 0.013	0.80	10/10	0.002
H2 gust_diag	pos RMS horiz	0.107 ± 0.002	0.0955 ± 0.0025	0.89	10/10	0.002
H2 gust_diag	pos max dev 3D	0.438 ± 0.019	0.343 ± 0.016	0.78	10/10	0.002
T7 pay-load_add	pos RMS horiz	0.0187 ± 0.0022	0.0177 ± 0.0017	0.95	7/10	0.375 (n.s.)
T7 pay-load_add	pos max dev 3D	0.354 ± 0.016	0.352 ± 0.0097	0.99	7/10	0.557 (n.s.)
H3 com_shift	pos RMS horiz	0.0351 ± 0.0026	0.0183 ± 0.0013	0.52	10/10	0.002
H3 com_shift	pos max dev 3D	0.276 ± 0.014	0.227 ± 0.0044	0.82	10/10	0.002

For T9 the position rows are not meaningful (the vehicle is in attitude mode and drifts about 19 m RMS for both controllers) and are not tabulated.

Yaw is the exception. Yaw RMS (deg) on the same scenarios:

Scenario	PID	INDI	INDI/PID	INDI lower	p
T6 gust_x	0.0878 ± 0.031	0.0877 ± 0.022	1.00	4/10	1 (n.s.)
H2 gust_diag	0.25 ± 0.027	0.109 ± 0.012	0.44	10/10	0.002
T7 payload_add	0.027 ± 0.0055	0.0302 ± 0.0035	1.12	2/10	0.131 (n.s.)
H3 com_shift	0.0732 ± 0.009	0.0344 ± 0.0051	0.47	10/10	0.002
T9 mis-match_inertia	0.167 ± 0.068	0.138 ± 0.059	0.83	8/10	0.322 (n.s.)

INDI is therefore equal or marginally worse in yaw on two of the five scenarios (neither difference significant) and better on the other three, of which two are significant. Yaw is a secondary metric and does not enter the acceptance criteria. The sources report yaw as INDI's weak axis whenever yaw is commanded (see section 8.4).

Earlier 5-seed ratios. The first batch (5 seeds; per-scenario values from `STATE.md`, session 14, the range 0.15-0.75x from `results/heldout_s14.md`, "Reading" section) gave slightly different ratios (com_shift 0.15x, gust_diag 0.57x, gust_x 0.61x, payload_add 0.75x). Those were superseded by the 10-seed values above. `STATE.md` still quotes the 5-seed figures in its session 14 entry.

T8 motor loss (10 seeds). T8 (motor 0 thrust x0.7 at runtime, then a +45 deg yaw step, attitude mode, speed 1) was not used in tuning. It was flown with 10 seeds per controller, 0 failures; the seed spread is negligible because the fault is deterministic and seeds change only sensor noise. Every difference below has the same sign on 10 of 10 seeds (exact paired Wilcoxon p = 0.002) (`results/t8_motor_loss_10seeds_s26.md`; seeds 1-3 first in `results/t8_motor_loss_s13.md`). Ranges over seeds 1-10 (whole window: mean ± standard deviation):

Metric	PID	INDI
Rotation RMS, whole window (deg)	9.06 ± 0.09	10.3 ± 0.05
Roll RMS (deg)	0.715 ± 0.001	0.082 ± 0.001
Pitch RMS (deg)	0.684 ± 0.002	0.155 ± 0.001
Yaw RMS (deg)	9.01 ± 0.09	10.3 ± 0.05
Motor-rate RMS (/s)	0.391 ± 0.003	0.508 ± 0.003
Saturated fraction	0.076 ± 0.001	0.121 ± 0.002
Peak tilt error at fault onset (deg)	5.11-5.16	1.61-1.64
Attitude RMS in fault window (deg)	1.116-1.121	0.194-0.199
Saturation time in fault window (s)	1.73-1.76	2.72-2.88
Faulty motor at high limit (s)	1.54-1.57	2.07-2.22

INDI holds roll and pitch 5-9x better than PID under the fault and has a 3.2x lower peak tilt at onset, but the whole-window rotation RMS is about 14 % worse because the window is dominated by the yaw step, which INDI tracks worse, and INDI saturates more (12 % against 7.5 % of the window) with about 1.3x the motor-rate effort. The sources have not separated how much of that saturation comes from the later yaw step and how much from holding roll and pitch with a weakened motor (`results/t8_motor_loss_s13.md`, "Open").

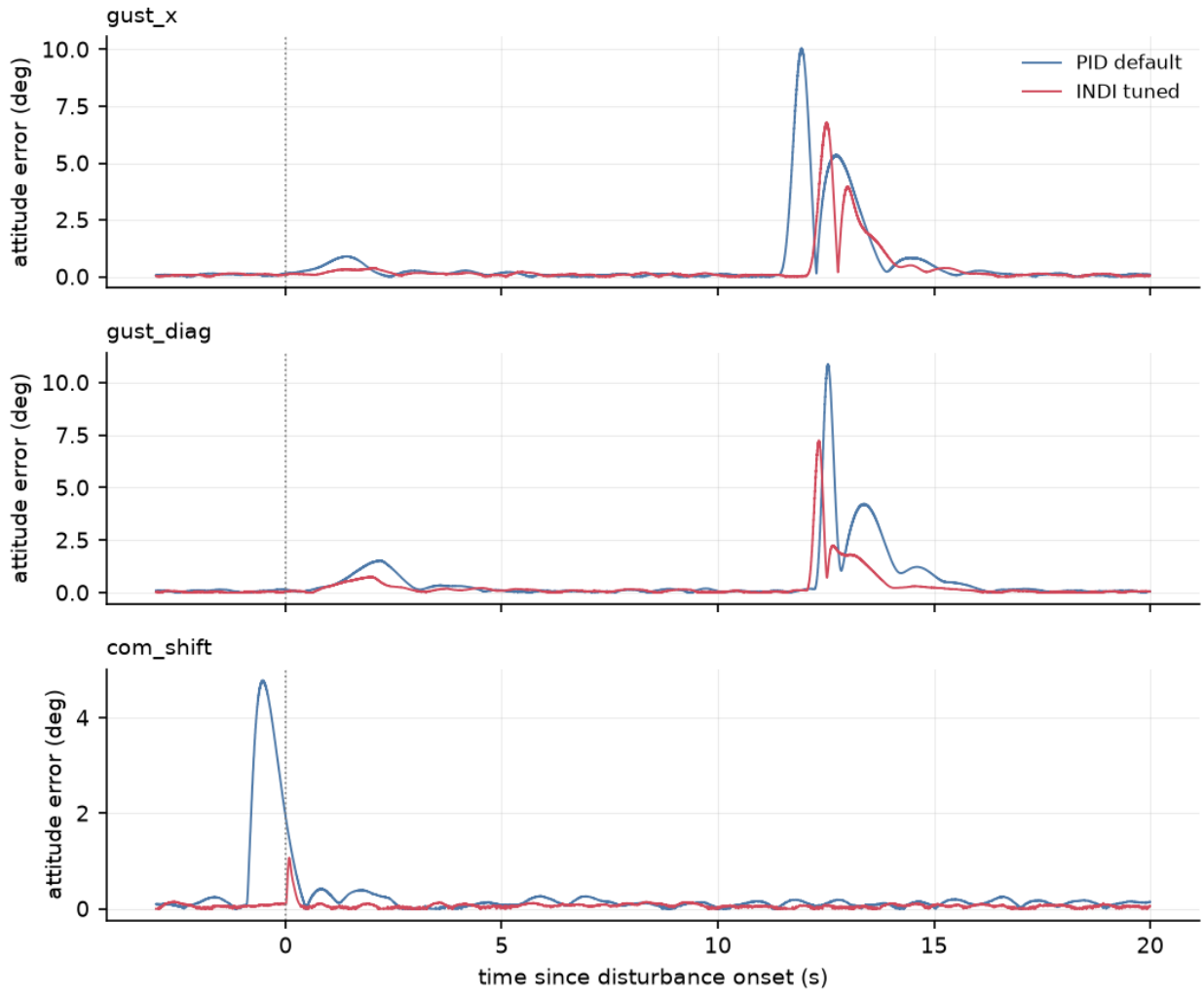


Figure 2: Figure 8.2: attitude error around the disturbance onset on gust_x (T6), gust_diag (H2) and com_shift (H3), seed 1, PID against INDI. Peak error in the plotted window: 10.1 against 6.8 deg (T6), 10.9 against 7.2 deg (H2), 4.8 against 1.1 deg (H3).

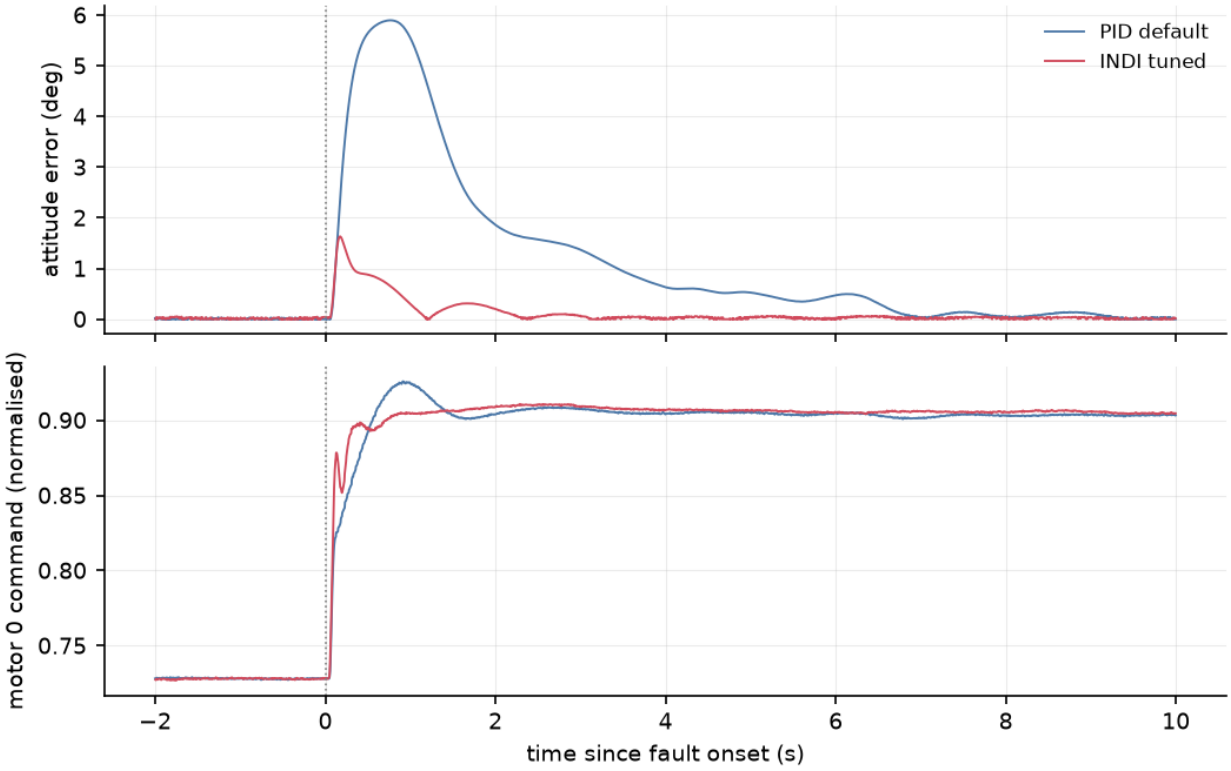


Figure 3: Figure 8.3: T8 motor_loss_0, seed 1: attitude error (top) and faulty motor 0 command (bottom) around the fault onset. The plotted error peaks at 5.9 deg for PID and 1.6 deg for INDI; the tabulated peak tilt error (5.1 and 1.6 deg) is a tilt-only metric.

8.4 Setpoint tracking

Tuning set (3 seeds, ratios to PID defaults). The frozen INDI candidate "r4f" against PID on the tuning scenarios (`results/tuning/r4f.md`, 15 flights, 0 failures). These scenarios were used to choose INDI's gains, so they are not evidence of generalisation, and no significance test is given for them:

Scenario	Metric ratio INDI/PID	Motor-rate effort ratio (min-max over seeds)
T1 hover	0.945	0.99-1.03
T2 step_roll	0.878	1.16-1.26
T2 step_pitch	0.935	0.63-0.71
T3 step_yaw	**1.096**	1.11-1.15
T4 rate_step_roll	1.021	0.67
Tuning cost J	0.978 (PID 0.958-0.964)	-

The tuning cost difference (0.978 against about 0.96) lies inside the hover-term noise of about 0.1, so on the tuning set the comparison is a tie under the frozen cost, not a win (`results/tuning/r4f.md`). The hover ratio of 0.945 is the noise-dominated position RMS and carries no weight. INDI tracks roll steps 12 % and pitch steps 7 % better, equals PID on rate pulses and is 10 % worse on yaw steps.

Hover (T1, 10 seeds). Attitude RMS 0.76x PID, lower on 10/10 seeds, p = 0.002. Motor-rate RMS 0.0387 against 0.0371 /s (1.04x, p = 0.16) (`results/acceptance_s15.md`, citing `results/hover_s15.log`). The 10-seed position RMS for hover is not reported in the source files read for this chapter.

Diagonal steps (H6 `step_diag`, 10 seeds, simultaneous 20 deg roll and pitch steps, speed 4). From `results/heldout_s14.md`:

Metric	PID	INDI	INDI/PID	INDI lower	Median impr.	p
Attitude rotation RMS (deg)	6.36 ± 0.14	5.83 ± 0.12	0.92	10/10	10 %	0.002
Roll RMS (deg)	4.41 ± 0.11	3.85 ± 0.097	0.87	10/10	14 %	0.002
Pitch RMS (deg)	4.5 ± 0.1	4.24 ± 0.086	0.94	9/10	7 %	0.004
Yaw RMS (deg)	0.802 ± 0.02	1.05 ± 0.032	**1.31**	0/10	-34 %	0.002
Motor-rate RMS (/s)	1.28 ± 0.055	1.21 ± 0.028	0.94	9/10	5 %	0.004

The 10 % median improvement on the primary metric is exactly the criterion 2 threshold, so H6 is borderline. The source notes the yaw loss does not count against criterion 3 because that criterion is evaluated on the primary metric. Position rows are meaningless in this scenario.

Trajectories (10 seeds, speed 1, 60/60 runs completed). Errors are measured against the logged `trajectory_setpoint`; att RMS is dominated by the commanded tilt and is not a pure rate-loop tracking error (`results/heldout_s15.md`).

Scenario	Metric	PID	INDI	INDI/PID	INDI lower	p
T5 lemniscate	position RMS 3D (m)	0.46 ± 0.0045	0.437 ± 0.0047	0.95	10/10	0.002
T5 lemniscate	max deviation 3D (m)	1.2 ± 0.026	1.3 ± 0.026	**1.08**	0/10	0.002
T5 lemniscate	att RMS (deg)	10.8 ± 0.017	7.89 ± 0.017	0.73	10/10	0.002
T5 lemniscate	min altitude (m)	5.03 ± 0.13	4.94 ± 0.029	0.98	-	0.23
H1 circle fast	position RMS 3D (m)	3.43 ± 0.042	2.51 ± 0.022	0.73	10/10	0.002
H1 circle fast	max deviation 3D (m)	8.07 ± 0.083	6.67 ± 0.078	0.83	10/10	0.002
H1 circle fast	att RMS (deg)	16.9 ± 0.059	15.3 ± 0.036	0.90	10/10	0.002
H1 circle fast	min altitude (m), lower is worse	4.0 ± 0.16	3.84 ± 0.031	0.96	-	0.006

On T5, INDI gains 5 % in position RMS but has an 8 % larger peak deviation, on every seed. The shared PX4 position controller dominates the error. On H1 the specified path needs about 52 deg of tilt, above the default `MPC_TILTMAX_AIR` of 45 deg, so it is infeasible for both controllers and errors are metres. INDI's 27 % lower position RMS there is presumably due to its faster attitude response using the tilt limit sooner, but this was not analysed (`results/heldout_s15.md`). Also, INDI flies lower on H1 (minimum altitude 3.84 against 4.0 m, p = 0.006), which is a loss on that row. H1 counts as a win under criterion 2, with the caveat that it is a comparison on an infeasible path.

Yaw is worse whenever yaw is commanded. Evidence from the sources: tuning step_yaw 1.096x (3 seeds), step_diag yaw 1.31x (10/10 seeds worse, p = 0.002), T8 yaw about 14 % worse (10/10 seeds, p = 0.002) (`results/tuning/r4f.md`, `results/heldout_s14.md`, `results/t8_motor_loss_10seeds_s26.md`). The sources attribute this to a structural effect: PID filters its yaw torque at 2 Hz by default, and for INDI to avoid a 5x motor-rate effort on yaw steps the yaw-rate error (setpoint included) must be filtered by `MC_INDI_YAW_CUT` (decision D-yawsp). With that filter no INDI yaw set tried in round 4 beat PID on the yaw step at acceptable effort (best 1.089x at 1.13x effort) (`STATE.md`, session 13; `results/tuning/r4.md` as quoted there). This is a trade between yaw tracking and yaw-path noise, not a gain-tuning accident, but it has been tested only inside this one filter structure.

8.5 Robustness to actuator-model mismatch (H5)

H5 `mismatch_combined` modifies the plant to Ixx, Iyy, Izz x0.8, mass x1.15 (2.3 kg) and motor time constants x2 (0.025 s up, 0.05 s down), with controllers unchanged. It is flown as two runs, the T2 step pattern and the T4 rate-pulse pattern, 10 seeds each at speed 4 (`results/heldout_h5_s15.md`). **INDI loses this scenario.**

Roll-rate RMS (rad/s), mean ± std over 10 seeds:

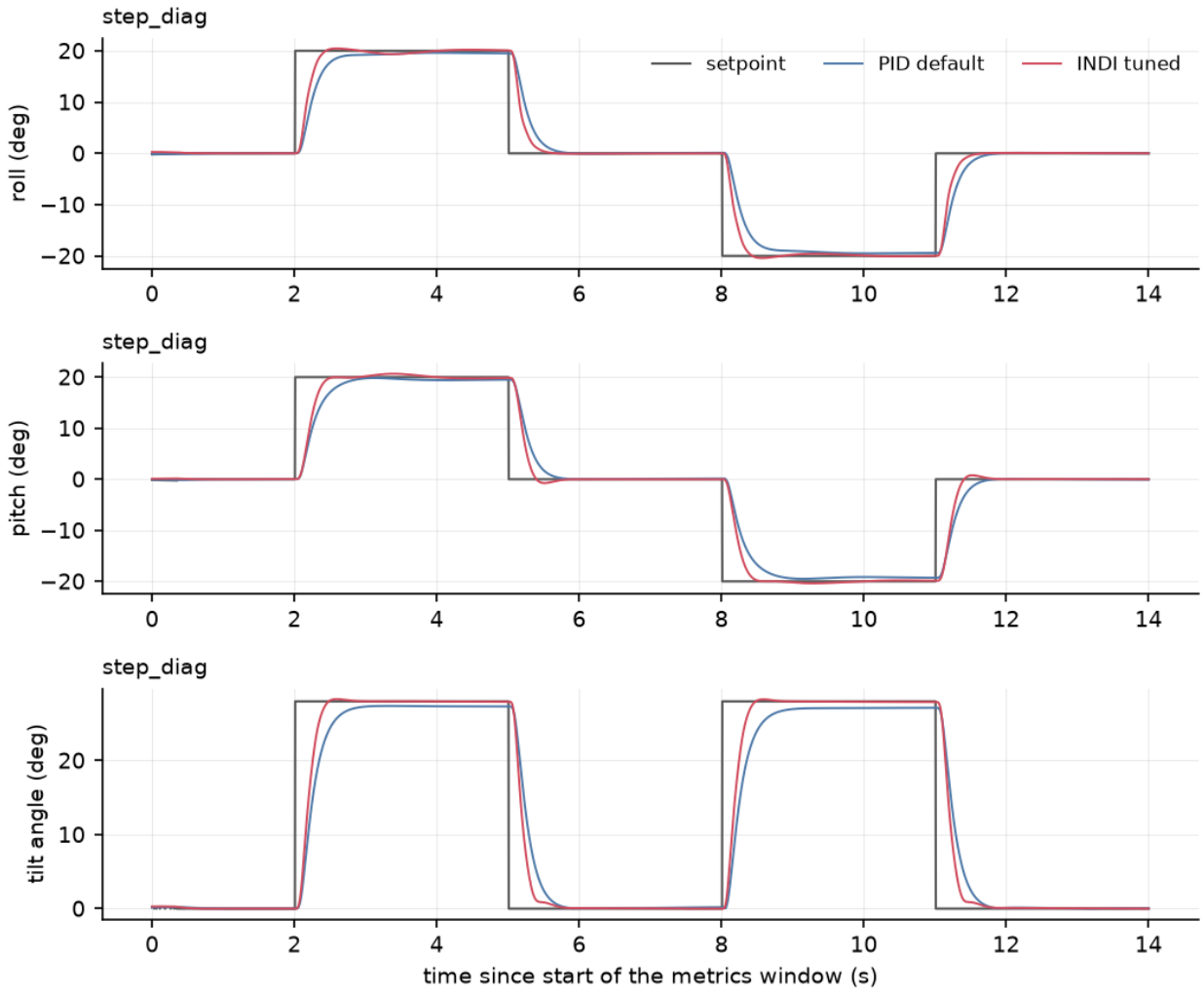


Figure 4: Figure 8.4: H6 step_diag, seed 1 (speed 4): roll, pitch and tilt angle against their setpoints, PID against INDI. Only H6 is plotted: the ULogs of the 3-seed tuning-set step scenarios T2/T3 are not kept in `results/runs/`.

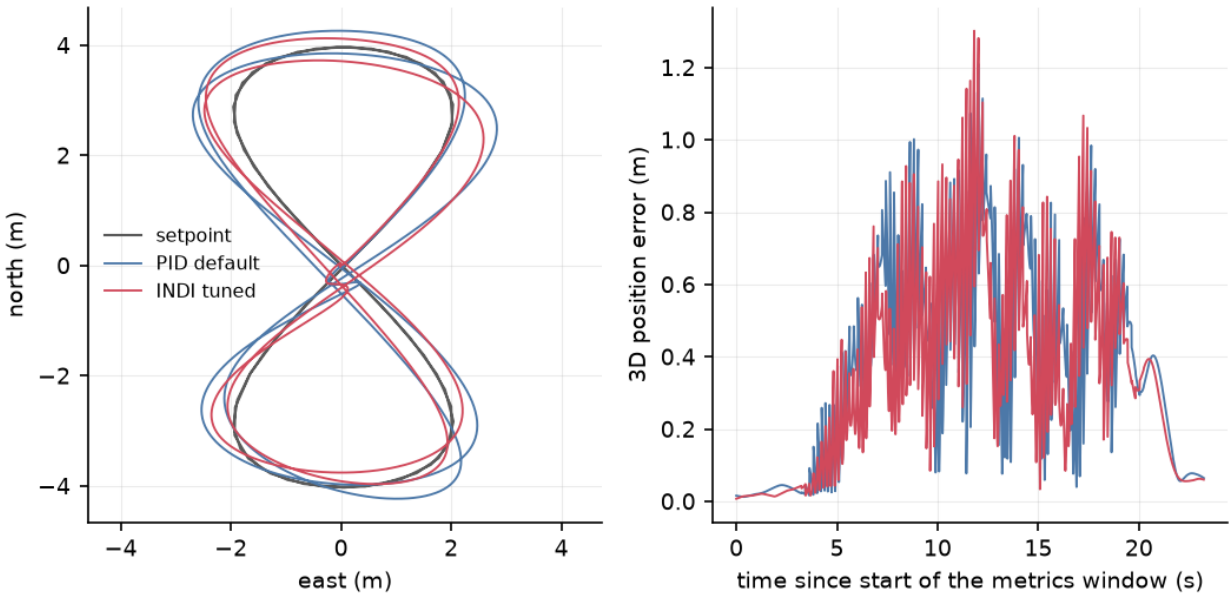


Figure 5: Figure 8.5: T5 lemniscate, seed 1: ground track against the setpoint and position error over the metrics window. 3D error RMS 0.456 m (PID) against 0.442 m (INDI), equal to the values in `metrics.json`.

Run	PID	INDI	INDI/PID	INDI lower
mismatch_combined_att (T2 pattern)	0.883 ± 0.0018	0.340 ± 0.039	3.86	0/10
mismatch_combined_rate (T4 pattern, primary metric of T4)	0.356 ± 0.004	0.562 ± 0.006	1.58	0/10

Other metrics (`results/heldout_h5_s15.md`):

Run	Metric	PID	INDI	INDI/PID	INDI lower	p
att pattern	roll attitude RMS (deg)	3.27 ± 0.077	2.97 ± 0.073	0.91	10/10	0.002
att pattern	motor-rate RMS (/s)	0.623 ± 0.094	1.92 ± 0.24	**3.08**	0/10	0.002
rate pattern	roll attitude RMS (deg)	17.7 ± 0.26	19.3 ± 0.62	1.09	0/10	0.002
rate pattern	motor-rate RMS (/s)	3.34 ± 0.037	3.64 ± 0.034	1.09	0/10	0.002
rate pattern	saturated fraction	0.0164 ± 0.00016	0.0338 ± 0.00085	2.06	0/10	0.002

The lower roll attitude RMS on the T2 pattern (0.91x) is misleading: the steps dominate that metric, while INDI carries a sustained roll oscillation. On the rate pattern INDI is worse in

every roll metric. Off-axis, INDI is better on pitch and yaw (pitch 0.17x on the rate pattern, yaw 0.43x), but these are small quantities next to the roll loss.

Diagnosis (INDI only, seeds 1-2, diagnostic scenarios `diag_h5_*`, never used for tuning; `results/heldout_h5_s15.md`). Roll-rate standard deviation over the last 2 s of the level-hold segment:

Plant	INDI `MC_INDI_ACT_TC`	Hover roll-rate std (rad/s)	Peak frequency
H5 combined, seed 1	0.0114 (frozen)	0.256	6.5 Hz
PID on H5 combined, seed 1	-	0.001	-
T9 inertia x1.3	0.0114	0.002	-
inertia x0.8 and mass x1.15 only	0.0114	0.002	-
motor time constants x2 only	0.0114	0.267 / 0.278	5.2 / 5.6 Hz
motor time constants x2 only	0.0228 (matched x2)	0.004 / 0.004	-

The motor lag mismatch alone produces the 5-7 Hz limit cycle. INDI builds its actuator-state estimate from a first-order model with time constant `MC_INDI_ACT_TC`. When the real motors are twice as slow, the modelled state leads the real one, the synchronisation between the actuator-state and angular-acceleration filters is broken, and at the frozen gain K_R of 23.9 rad/s the loop settles into a limit cycle. Inertia and mass errors scale only G1 and are tolerated. With `MC_INDI_ACT_TC` doubled to match the plant the oscillation vanishes, and the remaining extra rate error is due to the slower actuator itself. The matched-time-constant run is a diagnosis, not a retune; the held-out result remains the frozen-candidate result. PID has no actuator model and degrades gracefully. A robustness margin on `ACT_TC` or a lower K_R is a design trade that has not been evaluated, and the vehicle's motor time constant would have to be identified, or ESC RPM telemetry used, before relying on INDI on hardware (`results/heldout_h5_s15.md`).

Caveats: the diagnosis uses 2 seeds, one INDI parameter set and one mismatch factor (x2). Whether INDI would survive a smaller mismatch, or a mismatch in the other direction, is not tested in the sources.

Figure 8.6 shows the roll rate of both controllers on the H5 rate-pulse run, seed 1. A power spectrum of the level-hold segment and the `diag_h5_*` runs with matched `MC_INDI_ACT_TC` are not plotted; the 5-7 Hz peak frequencies in the diagnosis table come from `results/heldout_h5_s15.md`.

8.6 Double failure (H4)

H4 `motor_loss_2` applies motor 2 thrust x0.6 from the start of the T5 lemniscate path (path acceleration 6.6 m/s^2). As specified and frozen, this is beyond what the x500 can deliver with either controller (`results/heldout_s15.md`). **Both controllers crash on 10/10 seeds** (ground contact, altitude below 0.3 m). Under criterion 1 this counts as an INDI failure, since the criterion is written for INDI alone. PID seed 1 also touched the ground in the earlier harness check (`bench/scenarios.md`, change log).

Up to first ground contact (inline script, session 15):

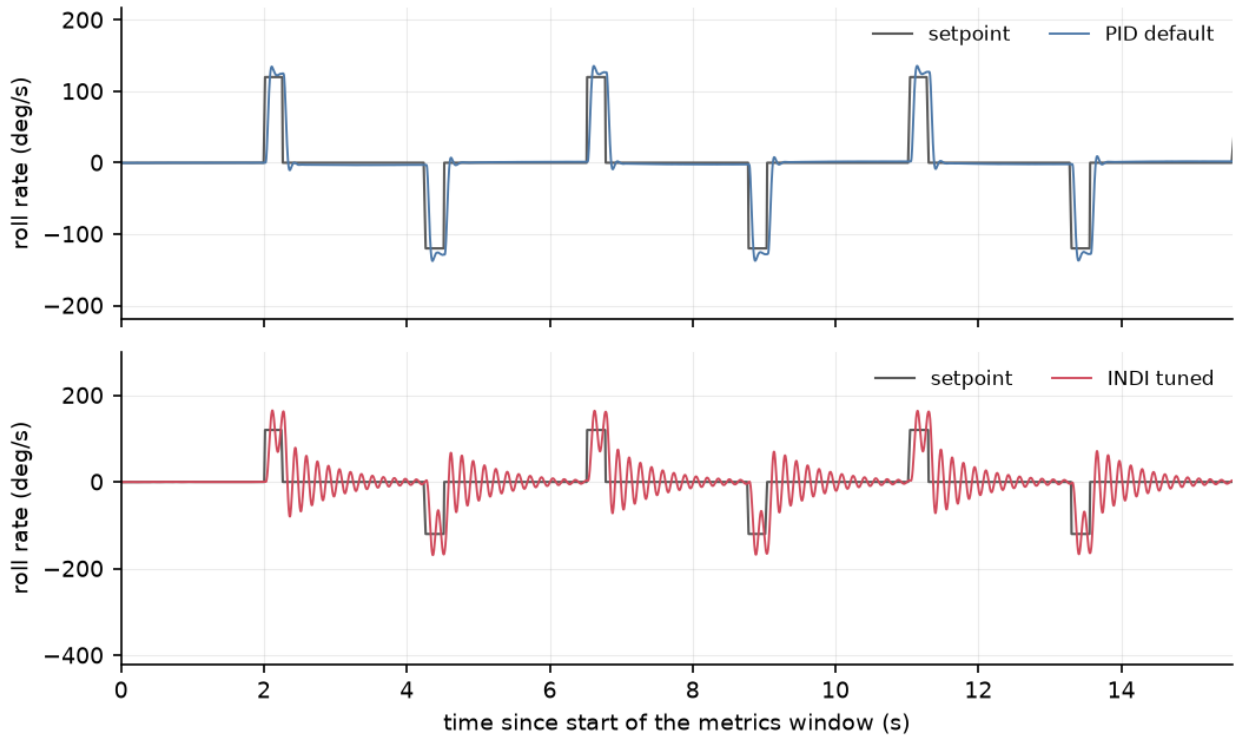


Figure 6: Figure 8.6: roll rate against its setpoint on mismatch_combined_rate, seed 1, metrics window; PID top, INDI bottom.

	PID	INDI
Time from fault to ground contact, seeds 1-10 (s)	7.1-7.2 (7.2 on all but one)	7.3-7.6 (mean 7.45)
Max tilt before contact (deg)	17 on every seed	18-23 (18 on 6 seeds)

Whole-window metrics (these include post-impact samples, so they describe the crash and not the control):

Metric	PID	INDI	INDI/PID	INDI lower	p
Position RMS 3D (m)	4.35 ± 0.24	5.01 ± 1.0	1.15	3/10	0.11
Attitude RMS (deg)	38.6 ± 25	69.3 ± 22	1.80	1/10	0.010
Peak tilt error (deg)	32.1 ± 7.9	111 ± 56	3.46	1/10	0.004
Saturation time (s)	8.86 ± 2.7	8.97 ± 1.6	1.01	4/10	0.85

Read literally, the whole-window attitude and peak-tilt metrics are significantly worse for INDI. The source argues that INDI's large tilt errors (above 90 deg) all occur after impact on the ground, and that in flight the two controllers fail in the same way, INDI staying airborne about 0.3 s longer (`results/heldout_s15.md`). That reading comes from an inline script on `vehicle_local_position` and `vehicle_attitude` and has not been moved into `bench/`. The metrics window should end at ground contact for scenarios that can crash; that change to `metrics.py` is listed as to-do in the source and had not been made. H4 is therefore a draw in substance, but the record still shows a formal loss on criterion 1. Note also that `bench/scenarios.md` does not list a primary metric for H4, and `acceptance_s15.md` uses position RMS.

8.7 Control effort and saturation

Effort is measured as the RMS of the time derivative of the `actuator_motors` outputs (noise transmission, "motor-rate RMS") and as the fraction of the window with any motor at or be-

Scenario	Ratio	INDI lower	p	Source
T2 step_pitch (3 seeds)	0.63-0.71	-	-	`results/tuning/r4f.md`
T3 step_yaw (3 seeds)	1.11-1.15	-	-	`results/tuning/r4f.md`
T4 rate_step_roll (3 seeds)	0.67	-	-	`results/tuning/r4f.md`
T6 gust_x	0.89	9/10	0.084	`results/heldout_s14.md`
T7 payload_add	1.01	4/10	0.193	`results/heldout_s14.md`
H2 gust_diag	1.13	0/10	0.002	`results/heldout_s14.md`
H3 com_shift	1.09	1/10	0.084	`results/heldout_s14.md`
H6 step_diag	0.94	9/10	0.004	`results/heldout_s14.md`
T9 mis-match_inertia	1.27	0/10	0.002	`results/heldout_s14.md`
H5 att pattern	3.08	0/10	0.002	`results/heldout_h5_s15.md`
H5 rate pattern	1.09	0/10	0.002	`results/heldout_h5_s15.md`
T8 motor loss (10 seeds)	1.30	0/10	0.002	`results/t8_motor_loss_10see`

INDI uses significantly more effort on H2 (1.13x), T9 (1.27x) and H5 (3.08x and 1.09x), and significantly less on H6 (0.94x). Only H5 approaches or exceeds the 1.5x hover limit, and criterion 4 is defined for hover only. The H5 result is the limit cycle of section 8.5.

Saturation (fraction of window with a motor at a limit, or time saturated):

Scenario	PID	INDI	Note
Most disturbance scenarios (T6, T7, H2, H3, T9, H5 att pattern)	0	0	`results/heldout_s14.md`, `results/heldout_h5_s15.md`
H6 step_diag	0.000143 ± 0.00031	0.0000864 ± 0.00019	negligible, p = 1
T8 motor loss	0.076 ± 0.001	0.121 ± 0.002	10 seeds, 0/10 lower, p = 0.002; INDI saturates more
H5 rate pattern	0.0164	0.0338	2.06x, p = 0.002
H4 motor_loss_2 (time, s)	8.86 ± 2.7	8.97 ± 1.6	crash scenario, p = 0.85

Saturation is therefore a non-issue on most scenarios but INDI saturates more than PID on both of the scenarios where the plant is degraded (T8, H5 rate pulses). No scenario shows INDI saturating less. The benchmark spec also asks for the `control_allocator_status` unallocated torque; the sources do not report it.

8.8 Summary of all scenarios

Primary-metric ratio INDI/PID (below 1 favours INDI) and verdict. “n” is the number of paired seeds. Where a source reports no p-value, the cell is empty.

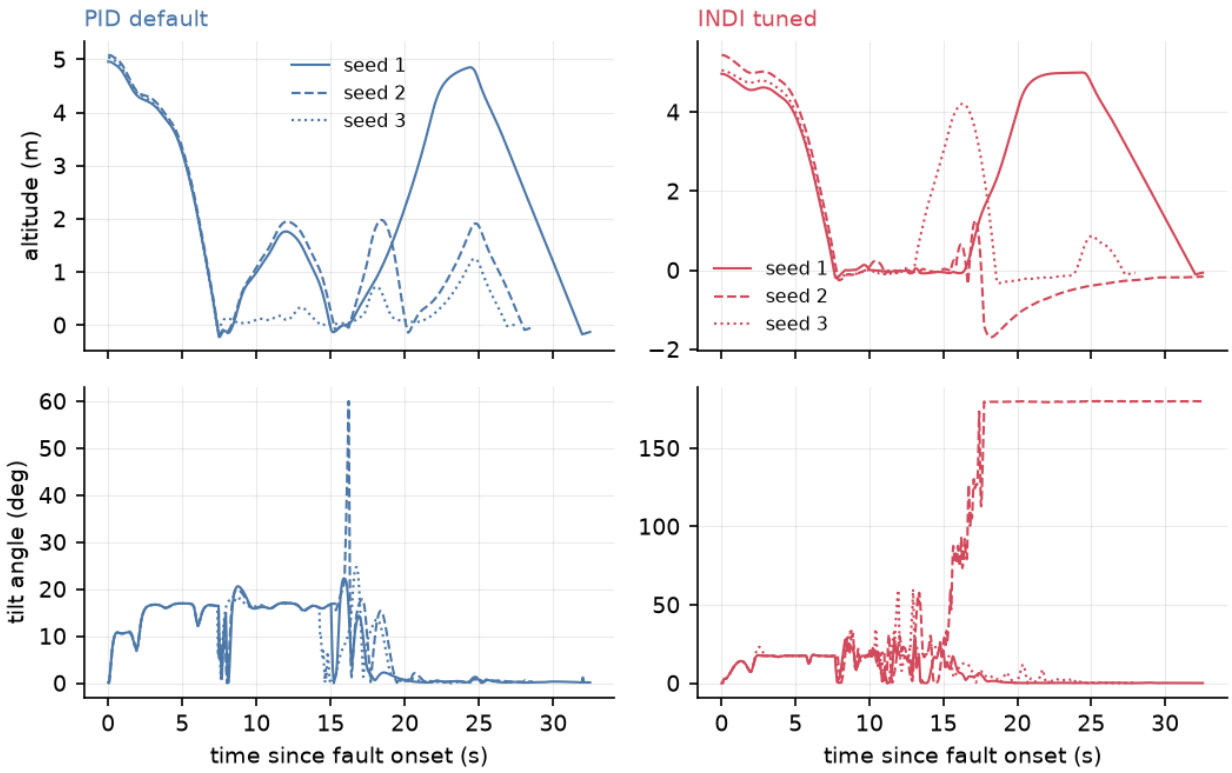


Figure 7: Figure 8.7: H4 motor_loss_2, seeds 1-3: altitude and tilt from fault onset. Both controllers lose altitude identically and first touch the ground about 7.5-8 s after onset. What follows (bounces, re-climbs, INDI seed 2 at 180 deg and negative altitude) is post-impact behaviour of the simulation and the estimator and is not interpreted.

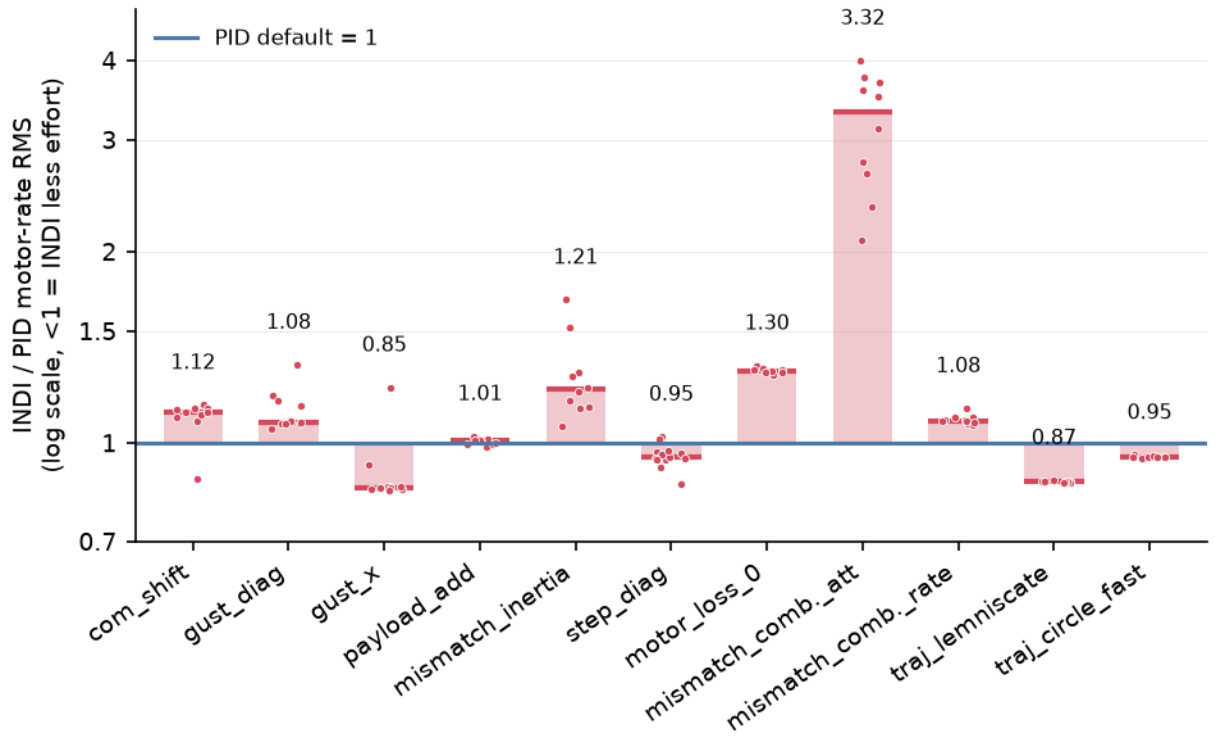


Figure 8: Figure 8.8: INDI/PID ratio of motor-rate RMS per scenario, same layout as Figure 8.1.

ID	Scenario	n	Primary metric	INDI/PID	INDI lower	p	Verdict	Source
T1	hover	10	attitude RMS (the T1 primary metric, position RMS, is noise-dominated)	0.76	10/10	0.002	no primary-metric result (position RMS noise-dominated); attitude RMS 0.76x, secondary; effort 1.04x	`acceptance_s15.md`
T2	step_roll	3	stepped-axis attitude RMS	0.878	-	-	INDI better, tuning set	`tuning/r4f.md`

ID	Scenario	n	Primary metric	INDI/PID	INDI lower	p	Verdict	Source
T2	step_pitch	3	stepped-axis attitude RMS	0.935	-	-	INDI better, tuning set	`tuning/r4f.md`
T3	step_yaw	3	yaw attitude RMS	1.096	-	-	**INDI worse**, tuning set	`tuning/r4f.md`
T4	rate_step_roll	3	roll-rate RMS	1.021	-	-	tie, tuning set	`tuning/r4f.md`
T5	traj_lemniscate	10	position RMS 3D	0.95	10/10	0.002	small INDI gain; max deviation 1.08x (worse)	`heldout_s15.md`
T6	gust_x	10	attitude RMS	0.60	10/10	0.002	INDI better	`heldout_s14.md`
T7	payload_add	10	attitude RMS	0.72	10/10	0.002	INDI better (small absolute values)	`heldout_s14.md`
T8	motor_loss_10	10	fault-window attitude RMS	0.18 (0.197 against 1.118 deg)	10/10	0.002	INDI better at onset; whole-window rotation RMS 1.14x (worse, yaw, 0/10)	`t8_motor_loss_10seed`
T9	mismatch_inertia	10	attitude RMS	0.88	10/10	0.002	INDI better; effort 1.27x	`heldout_s14.md`
H1	traj_circle_fast	10	position RMS 3D	0.73	10/10	0.002	INDI better, path infeasible for both	`heldout_s15.md`
H2	gust_diag	10	attitude RMS	0.57	10/10	0.002	INDI better; effort 1.13x	`heldout_s14.md`

ID	Scenario	n	Primary metric	INDI/PID	INDI lower	p	Verdict	Source
H3	com_shift	10	attitude RMS	0.16	10/10	0.002	INDI better	`heldout_s14.md`
H4	motor_loss_12	10	position RMS 3D	1.15	3/10	0.11	both crash 10/10; formal criterion 1 failure	`heldout_s15.md`
H5	mismatch_t0 T2 part	10	combined, stepped-axis attitude RMS	0.91	10/10	0.002	borderline better, but limit cycle, effort 3.08x	`heldout_h5_s15.md`
H5	mismatch_t0 T4 part	10	combined, roll-rate RMS	1.58	0/10	0.002	**INDI significantly worse**	`heldout_h5_s15.md`
H6	step_diag	10	attitude RMS	0.92	10/10	0.002	borderline better; yaw 1.31x worse	`heldout_s14.md`

The T8 ratio of 0.18 is the ratio of the 10-seed means in `results/t8_motor_loss_10seeds_s26.md` (0.197/1.118), equivalently “5.7x lower” for attitude RMS in the fault window.

Overall reading. INDI has a clear and statistically solid advantage on disturbance torques (T6, T7, H2, H3, T8 at onset) in attitude, a smaller advantage in position, and about parity on nominal tracking. It loses on yaw whenever yaw is commanded, it has a larger peak deviation on the lemniscate, it saturates more under degraded actuators, and it is fragile to a wrong actuator time constant (H5), which PID tolerates. Under the rules fixed before INDI flew, that is “INDI does not outperform PID”.

Not available in the sources used for this chapter: 10-seed results for T2, T3 and T4 (only the 3-seed tuning-set values), the 10-seed hover position RMS, peak deviation and recovery-time metrics for T6, T7, H2 and H3 (the sources report RMS and position maximum deviation), and any comparison against a separately retuned `pid_tuned` (it equals the defaults). A p-value cannot be given for T8.

9 Discussion

This chapter interprets the results of chapter 8 and states what they do and do not support. It does not repeat the tables. The verdict under the acceptance rules frozen before INDI flew is “INDI does not outperform PID” (`results/acceptance_s15.md`). The chapter explains why that verdict coexists with a clear INDI advantage in disturbance rejection, treats the actuator-model weakness (H5) as the main finding, explains why yaw parity is structural, lists the limits of the evidence, and lists what would have to be done before any real flight.

9.1 Interpretation

Why INDI rejects disturbance torques better. The incremental law of chapter 5 computes a torque increment from the measured angular acceleration. An unmodelled torque, whether from a shifted centre of mass, a gust acting through a centre of pressure, or an inertia error, changes the measured acceleration within one controller cycle, and the next increment compensates it. The law needs no model of the disturbance, only the effectiveness G1 and a correct estimate of what the actuators are already doing. The PID rate loop has no such measurement. It reacts to the resulting rate error, and a constant torque offset is removed only as fast as its integrator accumulates. The data are consistent with this picture. The attitude RMS ratio INDI/PID is 0.16 on the centre-of-mass shift (H3), 0.57 and 0.60 on the two gust scenarios (H2, T6), and 0.72 on payload addition (T7), lower on 10 of 10 seeds in each case (`results/heldout_s14.md`). On T8 the peak tilt error at the onset of the motor fault is 1.6 deg for INDI against 5.1 deg for PID, lower on 10 of 10 seeds (`results/t8_motor_loss_10seeds_s26.md`).

Two readings of the ordering are offered as interpretation, not as tested claims. First, the largest gain is on the scenario whose disturbance is a constant torque offset (H3, roll 0.13x), which is the case where a PID integrator is slowest relative to a direct measurement. Time-varying gusts give a smaller gain. Second, the position gains are much smaller than the attitude gains (horizontal position RMS ratio 0.84 on T6, 0.89 on H2, none significant on T7). Both controllers share the PX4 position loop, and the disturbance forces act on position regardless of attitude. No ablation was run that would isolate the mechanism, for example a PID with a retuned integrator or a disturbance feed-forward, so the explanation rests on the control-theoretic argument and the consistency of the results, not on a controlled experiment.

Size of the effect in absolute terms. The attitude errors on T7 and H3 are tenths of a degree or less for both controllers (T7: 0.090 against 0.065 deg; H3: 0.566 against 0.089 deg). A ratio of 0.16 describes a small quantity. Whether it matters depends on the application. For a gimbal-free survey platform a 0.5 deg transient is probably irrelevant, while for a vehicle with a large lever arm or a tight position requirement it could matter. The position metrics, which are closer to a mission-level quantity, show smaller gains (H3 horizontal RMS 0.52x, T7 not significant).

Why a clear advantage still counts as a loss. The acceptance rules were written before INDI existed and were applied as written (decision D-accept, `DECISIONS.md`). They are a conjunction of conditions: no INDI run may fail (criterion 1), and no held-out scenario may be significantly worse by more than 10 % (criterion 3), in addition to the requirement of wins on at least 4 of 6 held-out scenarios (criterion 2). A conjunction of “no worse anywhere” conditions penalises a controller with one sharp weakness, however large its strengths elsewhere. Here criterion 3 fails on H5 (roll-rate RMS 1.58x, 0 of 10 seeds better), and criterion 1 fails on H4, where both controllers crash on 10 of 10 seeds. Criterion 2 is met only marginally: H1, H2 and H3 are clear, while H6 and the T2 part of H5 sit exactly at the 10 % threshold, and H1 is a comparison on a path that is infeasible for both controllers (`results/acceptance_s15.md`, `results/heldout_s15.md`). The spec allows “matches” only if criteria 1 and 3 hold, so that outcome is also unavailable.

Two rule changes were considered and rejected as after-the-fact: averaging the two parts of H5, and excluding H4 as infeasible. Neither would change the outcome, because criterion 3 fails on H5 alone (D-accept). The verdict should therefore be read as precisely what it says: INDI did not satisfy a strict no-regression standard. It is not a finding that INDI is the worse controller on average, nor that it is the better one. The honest summary is a trade. INDI has a large, statistically solid advantage against disturbance torques. It is about equal on nominal tracking: the tuning-set cost is 0.978 against about 0.96, which is inside the hover noise of about 0.1 (`results/tuning/r4f.md`, D-hovernoise), roll steps are 12 % better and yaw steps 10 % worse (tuning set, 3 seeds). And it has a specific, severe vulnerability to a wrong actuator

model.

A caveat on how fair the disturbance comparison is. The tuning cost did not include any disturbance scenario. T5 to T9 were not used in tuning in the documented rounds (`report/07_tuning.md`, section 7.6), for either controller. The PID defaults were therefore not tuned for disturbance rejection, and neither was INDI. A PID with a higher integral gain, found by a search that included gust and shift scenarios, might narrow the gap. The sources do not test this. The scenario spec says both controllers would be re-tuned on the full T set once T5 to T9 existed, but that round was never run, and it was not run afterwards because the held-out results were by then known (`DECISIONS.md` D-noretune).

9.2 Actuator-model sensitivity

The finding. On H5 (inertia x0.8, mass x1.15, motor time constants x2) the frozen INDI candidate develops a 5 to 7 Hz roll limit cycle. Roll-rate RMS is 3.86x PID on the step pattern and 1.58x on the rate-pulse pattern, motor-rate effort is 3.08x PID on the step pattern, and the saturated fraction on the rate pattern doubles (`results/heldout_h5_s15.md`). The diagnostic runs isolate the cause to the motor lag: motor time constants x2 alone give a hover roll-rate standard deviation of about 0.27 rad/s at 5 to 6 Hz, whereas inertia and mass errors alone give 0.002 rad/s, the same as the T9 inertia case. When `MC_INDI_ACT_TC` is doubled to match the changed plant, the oscillation disappears (0.004 rad/s). The PID, which has no actuator model, shows 0.001 rad/s on the combined mismatch. This is the main robustness weakness of the INDI controller as implemented here. It is sufficient on its own for the verdict to be a loss (criterion 1 also fails on H4).

Mechanism. The explanation in the source is that INDI builds its actuator-state estimate from a first-order model with time constant `MC_INDI_ACT_TC`. If the real motors are slower, the modelled state leads the real one, the pairing between the actuator-state and angular-acceleration filters that the incremental law relies on no longer holds, and at the frozen rate gain K_R of 23.9 rad/s the loop settles into a limit cycle (`results/heldout_h5_s15.md`). This is consistent with the matched-time-constant experiment and with the literature's statement that actuator dynamics must be known or measured. It has not been confirmed by a stability analysis or by a frequency-domain check of the loop with a mismatched lag, so it should be taken as a well-supported diagnosis, not a proof. Inertia and mass errors scale only G1 and are tolerated, which agrees with the argument that the increment form is insensitive to moderate effectiveness errors.

A candidate mechanism from a linear analysis. The independent control-law review (`review/control_law_review_s16.md`, section 5; derived from the code structure, not reproduced numerically) makes the diagnosis more precise. With the filtered actuator-state estimate built from a first-order model `A_est` and the measured acceleration produced by the true actuator `A_true`, the increment law reduces to `u = (K/G1) e / (1 - H (A_est - A_true))`. With a matched model this is a plain proportional law on the rate error and the loop is `K A_true / s`. The mismatch term `A_est - A_true = (tau_t - tau_e) s / ((tau_e s + 1)(tau_t s + 1))`, with `tau_t` the true and `tau_e` the modelled time constant, is a band-pass centred at `f* = 1/(2 pi sqrt(tau_e tau_t))`. When the plant is slower than the model (`tau_t > tau_e`) it raises the effective gain near `f*`; for the H5 plant and the frozen model (tau_e = 11.4 ms against tau_t = 25 ms up and 50 ms down) this puts `f*` at about 6.7-9.4 Hz, close to but somewhat above the observed 5-7 Hz, which the added actuator phase lag may explain, and it lies below the 30 Hz synchronisation filters H, which therefore neither cause nor suppress it. Together with the extra actuator phase lag and the pipeline delay, the phase margin near the roughly 3 Hz crossover collapses, and the plant saturations (increment clamp, allocator, asymmetric motor constants) turn the instability into a bounded limit cycle. The same analysis explains the `MC_INDI_ACT_TC` sweep below: a model slower than the plant gives a negative mismatch term, which is a gain reduction (sluggish but safe), and a model three times faster

than the frozen one puts f* at about 16-23 Hz on the stock plant, well above crossover, where the boost is small. The dangerous case is specifically a plant slower than the model with f* near crossover. This is an analysis, not a measured loop transfer function, and it should be read as the most likely mechanism.

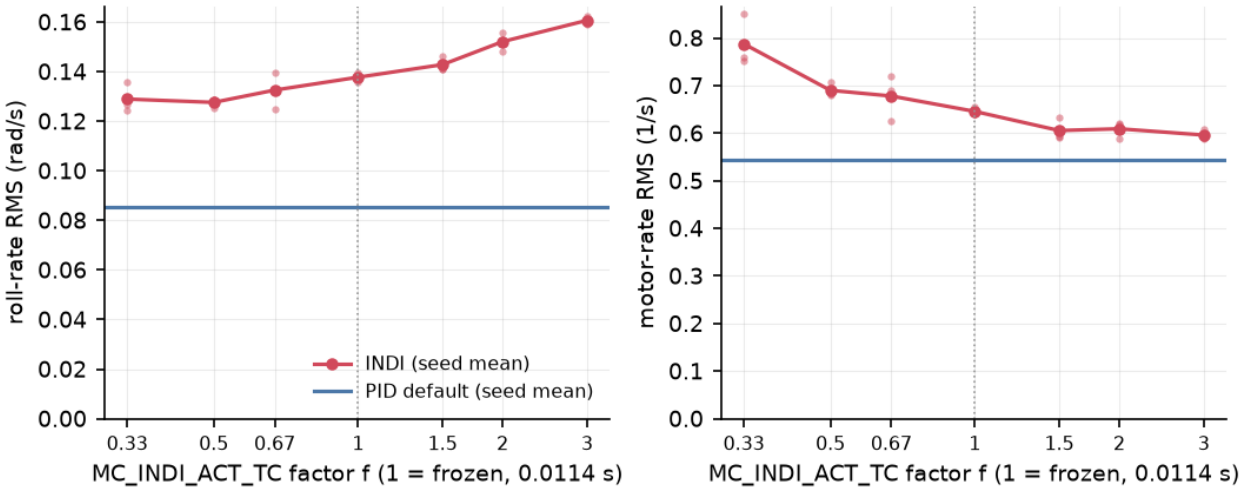


Figure 9: Figure 9.1: roll-rate RMS and motor-rate RMS against the MC_INDI_ACT_TC factor on the stock plant (step_roll, speed 4, 3 seeds); the blue line is the PID mean.

What the test does not cover. The diagnosis used 2 seeds, one INDI parameter set and one mismatch factor, in one direction (motors slower). The tolerance to a mismatch of x1.5, or of x0.5 (motors faster than modelled), is unknown. The frozen `MC_INDI_ACT_TC` is 0.0114 s, a value selected in tuning round r3 (`DECISIONS.md` D-freeze), and the simulator motor constants are 0.0125 s up and 0.025 s down (`timeConstantUp/timeConstantDown` in `Tools/simulation/gz/models/x500/model.sdf`, checked in session 17). A post-acceptance sweep that scaled `MC_INDI_ACT_TC` from 0.33x to 3x on the stock plant produced no limit cycle at any setting: roll-rate RMS moved only from 0.129 to 0.161 rad/s (`results/act_tc_sweep_s17.md`, 3 seeds per setting; Figure 9.1). So a mis-set actuator model on a fast actuator is benign here, and the H5 instability needs a physically slower actuator that the model does not know about. A sweep of the plant time constant itself, which would map the real stability boundary, was not run. A single time constant therefore stands for an asymmetric motor, and the nominal plant already carries some mismatch that the controller tolerates. The x2 test is a stress case of unknown severity relative to real hardware spread.

What it means for a real vehicle. ESC firmware, motor, propeller and battery combinations differ, and the time constant of the thrust response is rarely known to a factor better than 2. It also depends on the operating point and on ESC braking settings. A PX4 user cannot be asked to tune `MC_INDI_ACT_TC` per airframe by trial in flight, since a wrong value can excite an oscillation at several hertz, as H5 shows. Under the present design, INDI therefore moves the burden from "tune three PID gains" to "know the actuator lag". For the x500 in simulation the burden is low, because the model is known and fixed.

Possible remedies (future work, none tested here).

- Measured rotor speed. Using ESC RPM telemetry as the actuator state would remove the need for the model. The Tal and Karaman work cited in the research notes uses measured motor speed (`research/sources.md`, tal2021). In Gazebo `esc_status.esc_rpm` only echoes the command (`STATE.md`, integration facts), so this cannot be tested in the present simulator. The PX4 SIH simulator has a first-order motor lag and a modelled

esc_rpm (STATE.md, session 9) and might be used for a first test. Real ESC telemetry rate, latency and resolution would also have to be checked.
- Online identification of the actuator time constant and of G1, in the manner of adaptive INDI (Smeur et al. 2016, research/sources.md, smeur2016). It adds a state that can itself misbehave, and it needs excitation that the vehicle may not have in hover.
- Lower gains or a margin on the time constant. The source suggests a robustness margin on ACT_TC or a lower K_R as an unevaluated trade. A lower K_R would reduce the bandwidth that gives INDI its advantage, so the cost of robustness would have to be measured on the disturbance scenarios.
- Supervision. Detecting a sustained oscillation and falling back to the PID is a plausible safeguard. It is a suggestion of this report and has no support in the sources.

9.3 Yaw

What was observed. INDI is worse than PID wherever yaw is commanded: 1.096x on the step_yaw tuning scenario (3 seeds), 1.31x on the H6 diagonal steps with 0 of 10 seeds better, and about 14 % on the whole-window rotation RMS of T8 (0 of 10 seeds better, dominated by the yaw step) (results/tuning/r4f.md, results/heldout_s14.md, results/t8_motor_loss_10seeds_s26.md). Where yaw is disturbed but not commanded, INDI is better or equal: 0.44x on H2 and 0.47x on H3 with 10 of 10 seeds, 1.00x on T6 and 1.12x on T7, neither significant (results/heldout_s14.md). Yaw is not part of the acceptance criteria, which are evaluated on the primary metric.

Why the parity is structural. Yaw torque in a multicopter comes from rotor drag, not from thrust differences times a lever arm. Its authority is small and slow. The G1 values used in the frozen candidate are 241 for roll and pitch and 7.4 for yaw (report/07_tuning.md, section 7.5), a ratio of about 33. Two consequences follow.

1. The yaw increment is the angular-acceleration error divided by G1_Y. The noise of the filtered yaw acceleration is therefore multiplied by a large gain. The first INDI version had about 8 times the hover motor-rate effort of PID, entirely from the yaw path, with spectral content around 35 Hz (results/sync_check_v0.md; STATE.md). The identified G1_Y of 7.4, replacing an earlier value of 30, made that noise gain 4 times larger still (results/sync_check_v0.md). Yaw acceleration has a poor signal-to-noise ratio, so a filter is unavoidable.
2. The filters needed to bring effort down also remove the bandwidth. The sources introduce a low-pass on the synchronised yaw acceleration and torque pair (D-yawfilt), on the yaw rate used in the error (D-yawnu), and finally on the yaw rate error including the setpoint (D-yawsp). The last was motivated by the measured step_yaw effort of about 5x PID, caused by the unfiltered setpoint step passing through K_Y/G1_Y. PID smooths the same step with its default 2 Hz yaw output filter (DECISIONS.md, D-yawsp). The decision gives INDI the same structure PID already has. It was not an advantage offered to INDI alone, but it is also what makes the yaw loop of INDI "a P-type rate loop behind a filter at about 1.5 to 2 Hz", in the words of results/tuning/r4.md.

Round 4 swept only the three yaw parameters (12 candidates, 3 seeds, selection rule fixed before the results). The tracking and effort frontier it found is: a ratio of 0.98 to PID needs 3.8x the effort, and at effort at most 1.25x the best ratio is 1.05 to 1.09 (results/tuning/r4.md). The frozen set (c10) is 1.089 at effort 1.13 to 1.14. With the filter in place INDI's yaw inversion through a weak and slow G1_Y has little to gain, and near-parity or slight inferiority is the expected result.

Limits of this explanation. The claim "no gain setting recovers PID-level yaw tracking" was tested inside one filter structure only. Another structure, for example a yaw-specific feedforward, a notch instead of a low-pass, or a model with an identified rotor spin-up term G2, was not tried. G2 was not identifiable from the data (results/sync_check_v0.md: G2 0.05 with R^2

unchanged), and the yaw fit has R^2 of about 0.5. The explanation of the two worst candidates in round 4 as a lag-limited yaw oscillation was not checked on the logs (`results/tuning/r4.md`). A related model mismatch is noted in `STATE.md`: PX4 `CA_ROTOR_KM` is 0.05 whereas the Gazebo moment constant is 0.016 (`bench/sim_fidelity.md`). It does not bias the yaw G1 value: the in-loop identification regresses the filtered angular acceleration on the controller's own filtered normalised torque, taken before the allocator (`results/sync_check_v0.md`), so the allocator's moment-constant error is part of the identified effectiveness. Both controllers command the same allocator, so the PID baseline sees the same scaled yaw plant, and its default gains were kept only after the retuning of chapter 7 found nothing better. What the mismatch does change is how a normalised yaw torque maps to differential motor speed, and hence the yaw authority at saturation; this applies equally to both controllers and was not quantified separately. A real vehicle with a correct `CA_ROTOR_KM` would have a different yaw G1, which must then be re-identified. The yaw result is also specific to the x500, whose yaw authority depends on its propellers.

9.4 Limitations of the evidence

Single simulated airframe. Everything comes from one model, `gz_x500`, with G1 identified at a loop rate of 250 Hz on that model (`report/05_design_implementation.md`, section 5.7). Nothing here shows how the results would transfer to a larger vehicle, a different motor and propeller combination, or an airframe with lower stiffness. The H5 finding suggests that transfer cannot be assumed.

Motor and sensor models. The Gazebo model has a first-order motor lag with different up and down constants, which H5 modifies (`results/heldout_h5_s15.md`). It does not provide a measured rotor speed: `esc_rpm` echoes the command, so INDI uses an onboard model (`STATE.md`). Whether the thrust-to-speed relation, ESC dead time, battery sag and propeller inflow are representative of hardware is not established. The scenario spec required a fidelity document covering motor constants, IMU noise and bias, gyro rate and latency, and body drag before the disturbance and held-out scenarios were run (`bench/scenarios.md`, section 7). That document was written only after the benchmark had been flown (`bench/sim_fidelity.md`, session 17). It records a 250 Hz IMU with white gyro noise of 0.05 deg/s per axis and no bias, drift or vibration, and it leaves SITL latency uncharacterised. The report's conclusions on noise transmission, for instance the yaw acceleration noise, depend on that model. The disturbance injector is also a simplification: drag ignores vehicle velocity and rotor inflow, and the payload adds weight but not inertial mass (`DECISIONS.md`, D-dist). There is no battery model, so no voltage-dependent effectiveness (`report/05_design_implementation.md`, section 5.7).

Simulation speed. Attitude and rate scenarios (T9, H5, H6) ran at speed factor 4, and position-based ones at factor 1. The justification is a check on PID defaults, seeds 1 to 3: attitude and rate metrics agree within the seed spread, while hover position RMS does not (0.027 m against 0.064 m) (`DECISIONS.md`, D-speed). The cause is open, and the check was not repeated for INDI. A post-acceptance check repeated it for the frozen INDI candidate on H6 `step_diag` and H5 `mismatch_combined_rate`, seeds 1 to 3 (`results/speedcheck_indi_s22.md`). Rate-tracking and effort metrics agree within 4 % per seed, and the H5 limit cycle and its loss are unchanged at speed 1. The H6 attitude RMS, however, is 4 % higher on average at speed 1 (up to 7 %), which is half of INDI's 8 % H6 margin over PID; PID re-flown on H6 at speed 1 changes by less than 0.2 %, so the INDI/PID ratio on these three seeds moves from 0.874-0.904 at speed 4 to 0.881-0.973 at speed 1: INDI is still better on every seed, with a smaller margin. On H6 only INDI is sensitive to the speed factor (PID moved 2.5 % on step_roll in the earlier check, `results/speedcheck_pid_default.md`); the cause was not identified, and a timing effect on the angular-acceleration path is the most likely candidate. H6 was already at the acceptance threshold, and this check weakens it further. The speed-1 runs also include the post-freeze fixes, and n = 3.

Hover metric noise. Identical parameters and seeds gave hover scores between 0.77 and 1.31 in different batches, caused by sporadic vertical excursions in both controller families (`DECISIONS.md`, D-hovernoise; `results/tuning/r4.md`). Hover position RMS was therefore not used to rank controllers, and J differences below about 0.1 were treated as noise. The consequence for tuning is that the hover term gave almost no information, so the tuning decided effectively on four step scenarios.

The PID baseline. The baseline is the PX4 default parameter set for `gz_x500`. The sources report that no retuned PID beat it, with a best retuned J of 0.958 against 0.964 for the re-flown defaults. The search covered 40 candidates (24 in r1 with 2 seeds, 16 in r2 with 3 seeds, about 480 flights; `results/tuning/pid/r{1,2}/summary.csv`). In either case the search was random sampling in a 12-dimensional box, which shows that the defaults are hard to beat that way, not that they are optimal. A PID candidate that improved roll and pitch steps at a cost on yaw was seen (`results/tuning/r2.md`, via chapter 7).

Budget asymmetry. PID received two rounds (480 flights). INDI received four rounds (about 792 to 852 flights, depending on the source), plus three structural changes motivated by tuning-set effort (D-yawfilt, D-yawnu, D-yawsp). The spec asks for equal budgets. PID stopped early because the defaults won. The asymmetry favours INDI in tuning effort, but INDI's structural changes were also fixes of a defect that PID did not have, so the direction of the net bias cannot be stated from the data. No equivalent structural search was done on the PID side, for example on its D-term filtering (`report/07_tuning.md`, section 7.6).

Size and independence of the held-out set. There are six held-out scenarios, each with 10 seeds. The seeds vary sensor noise, and gust onset and direction where the scenario has them. They do not vary the trajectory start phase, which deviates from the spec (`bench/scenarios.md`, change log, session 15). The plant and the scenario are otherwise deterministic, so the seed spread is very small (for instance 0.0096 deg on a mean of 0.717) and a p-value of 0.002 is the floor of the test with n = 10 (`report/08_results.md`, section 8.1). It means that INDI was better on every seed under sensor noise. It is not evidence of generalisation across plants, parameters or scenarios. The effective sample for generalisation is the number of scenarios, which is six, and the "at least 4 of 6" rule is coarse. Two of the six (H1, H4) are infeasible for both controllers and discriminate poorly. T8 has only 3 seeds and no test. The acceptance depends on H5, which has a single mismatch factor.

Implementation risks found by review and not exercised by the benchmark. An independent review of the control law (`review/control_law_review_s16.md`, summary and triage) found one bug, fixed after the freeze and checked not to change the metrics (yaw filters reset on every landed cycle, D-landyaw, `results/landyaw_check_s19.md`), and the following risks, which are left as limitations because the benchmark runs at a steady SITL loop rate, with parameters fixed per flight and `MC_INDI_G2_Y = 0`, so none of them affects a reported number: (a) the filter redesign on loop-period jitter re-initialises the filtered torque to the current actuator-state estimate, desynchronising the feedback pair and stepping the torque on hardware; (b) the allocator saturation feedback used to stop increments is stale on half the cycles; (c) while landed the controller runs a proportional-only law without the actuator model; (d) the synchronisation filters are designed at the controller rate and gyro notch filters are not mirrored on the actuator-state path; (e) controller selection is latched while disarmed, the fault path resets all states on a single invalid sample, and there is no thrust- or voltage-dependent effectiveness scaling; (f) changing `MC_INDI_YAW_CUT` or `MC_INDI_DELAY` in flight re-initialises filters or delay history and can step the torque command; (g) the yaw filter toggling on a long cycle gives a one-cycle G2 spike (zero with the default `G2 = 0`; the `MC_INDI_G2_Y` upper bound was lowered from 100 to 1 after the freeze); (h) nothing enforces a margin between `MC_INDI_YAW_CUT` and the yaw gain `MC_INDI_K_Y` (the frozen tune has a ratio of 4.7, about 17 deg of filter lag at crossover, but the parameter bounds allow far worse); (i) the 19 unit tests all run at constant loop period with settled inputs; none covers period jitter, landed/airborne transitions with a moving torque, runtime parameter changes, fault recovery, the actuator-mismatch loop, or

the module-level integration in `MulticopterRateControl`. Each of these must be closed or tested before flight (section 9.5).

Reproducibility gaps. The trajectory and H4 tables were produced by inline scripts not yet in `bench/`. The metric window for crash scenarios includes post-impact samples, so the whole-window H4 metrics describe the crash and not the control (`report/08_results.md`, section 8.6).

9.5 What remains before real flight

The following are required or advisable before flying the controller on hardware. They are a proposed list, not a plan from the sources, except where a source is cited.

1. **Close the actuator-model gap first.** Decide how the actuator state is obtained on hardware (ESC RPM telemetry, or a measured and verified time constant) and test the sensitivity to mismatch in both directions, at several factors, with more than 2 seeds. Without this, the H5 result predicts a likely limit cycle on some airframes.
2. **Identification on the real vehicle.** Measure G1 (roll, pitch, yaw) and the motor time constant, for example on a thrust stand and by in-flight excitation with the PID active, as was done in simulation (`results/identification_g1_v0.md`). Check how effectiveness varies with rotor speed and battery voltage; the present code has no battery scaling on the INDI branch and uses a fixed G1 (`report/05_design_implementation.md`, section 5.7).
3. **Hardware-in-the-loop and SIH tests.** SIH has a modelled motor lag and `esc_rpm` and can exercise the real flight stack at a real loop rate, with real timing, before any propellers spin. Gyro filter and notch behaviour on a real, vibrating IMU must be checked, because synchronisation of the filters is approximate and notches are not replicated (section 5.7).
4. **Failure handling.** The controller resets its states on invalid data, on disarm, and while landed (`report/05_design_implementation.md`, sections 5.3.5 and 5.5.4). Behaviour under sustained saturation, motor failure and allocator reconfiguration has been tested only in the 3-seed T8 scenario, where INDI saturated more than PID (12 % against 7.5 %). Real motor-out and failsafe sequences, including what happens when the allocator changes its effectiveness matrix, have not been tested. H4 crashes with both controllers, so it carries no information on the margin.
5. **Mode switching and runtime selection.** `MC_RATE_CTL_TYPE` is latched while disarmed, so selection is tested only at arming. Transitions between flight modes while armed (manual, acro, position, offboard, return-to-launch, landing) were not exercised on hardware, and in simulation only the sequence of `bench/run_scenario.py` was exercised: arming and takeoff, offboard mode with position, attitude, rate or trajectory setpoints, and landing on `MAV_CMD_NAV_LAND`. Manual, acro, position-control and return-to-launch modes, and switches between them while airborne, were not flown with INDI.
6. **Outer loops.** Only the rate loop is incremental. Attitude and position loops are the standard PX4 P and PID controllers (`report/05_design_implementation.md`, section 5.3.6). The position results are limited by the shared position loop (T5 position RMS 0.95x with a larger maximum deviation of 1.08x). An INDI outer loop is not implemented and not evaluated.
7. **Yaw.** Decide whether the yaw filtering is acceptable on a real airframe, with its real G1_Y and noise, and whether yaw should stay on PID in a hybrid configuration. This hybrid has not been implemented or tested.
8. **More vehicles and a wider benchmark.** At least one second airframe with different inertia and motor dynamics, mismatch factors on both sides, a re-tune of both controllers on the full set including disturbance scenarios, and the benchmark reproducibility items of section 9.4 (inline scripts into `bench/`, a crash-aware metric window, trajectory start phase as a seed parameter).
9. **A staged flight protocol.** First tethered or net-protected hover with an immediate switch back to PID, then small attitude steps, then disturbances. The criteria for each

stage should be fixed before flying, as the simulation benchmark did.

10 Conclusion

10.1 Answer to the research question

The question was whether an INDI controller can replace the PID angular-rate loop of a PX4 multicopter and do better, judged by a benchmark whose scenarios, metrics and acceptance criteria were frozen before INDI first flew. In simulation, on the `gz_x500` airframe, the answer is no. Under the frozen rules the outcome is "INDI does not outperform PID" (`results/acceptance_s15.md`).

The verdict follows from two failed criteria. Criterion 3 requires that INDI is not significantly worse by more than 10 % on any held-out scenario, and it fails on H5, the combined mismatch scenario: roll-rate RMS during the rate pulses is 1.58 times that of PID, with 0 of 10 seeds better and p = 0.002 (`results/acceptance_s15.md`). Criterion 1 requires that no INDI run fails, and it fails on H4, the double motor failure, where INDI crashes on 10 of 10 seeds. PID also crashes on 10 of 10, and the two controllers behave alike until impact, but the criterion is stated for INDI alone and was applied as written. The verdict does not depend on H4, because criterion 3 fails on H5 by itself. The spec's "matches" outcome is available only if criteria 1 and 3 hold, so it is unavailable too (`bench/scenarios.md`, section 6; see section 8.2).

This is a loss and is reported as one. Two post-hoc rule changes were considered, averaging the two parts of H5 and excluding H4, and both were rejected because they would not change the outcome and would be made after seeing the data (chapter 9, section 9.1). The result is not evidence that INDI is the worse controller on average, nor that it is the better one. It is evidence that INDI did not meet a strict no-regression standard on this vehicle in this simulator.

10.2 Main quantitative findings

Attitude disturbance rejection is better on all four disturbance scenarios (T6, T7, H2, H3) and at T8 fault onset. The attitude RMS ratio INDI/PID is 0.16 on the centre-of-mass shift (H3), 0.57 on the diagonal gust (H2), 0.60 on the gust along x (T6) and 0.72 on payload addition (T7), lower on 10 of 10 seeds in each case, with p = 0.002 (`results/heldout_s14.md`). On the motor-loss scenario T8 the peak tilt error at fault onset is 1.6 deg for INDI against 5.1 deg for PID, lower on 10 of 10 seeds, but the whole-window rotation RMS is 14 % worse because of the yaw step that follows (`results/t8_motor_loss_10seeds_s26.md`). On the plant-mismatch scenario T9 (inertia error) the ratio is 0.88, also 10 of 10 seeds. Several of the absolute errors are tenths of a degree, so a low ratio can describe a small quantity (section 9.1).

Nominal tracking is at parity. The tuning-set cost of the frozen INDI candidate is 0.978 against about 0.96 for PID, a tie within the hover noise (`results/tuning/r4f.md`). Hover motor-rate effort is 0.0387 against 0.0371 /s, a ratio of 1.04 (`results/acceptance_s15.md`). On the lemniscate (T5) the position RMS ratio is 0.95, with a maximum deviation 1.08 times that of PID (`results/heldout_s15.md`).

Yaw is worse whenever it is commanded. On the diagonal step H6 the yaw ratio is 1.31 (`results/heldout_s14.md`), and the yaw sweep of round r4 found no INDI parameter set that beat PID on the yaw step at acceptable effort (`results/tuning/r4.md`; section 9.3).

Under actuator-model mismatch INDI develops a limit cycle. In H5 a 5-7 Hz roll limit cycle gives a control effort of 3.08 times PID (`results/heldout_h5_s15.md`). INDI also saturates more than PID on T8: 12 % against 7.5 % (`results/t8_motor_loss_s13.md`).

The PID baseline was searched over 40 candidates and none beat the PX4 defaults, so the comparison is against defaults that a 40-candidate random search did not beat; this shows they are hard to improve by sampling, not that they are optimal (`results/tuning/r2.md`).

10.3 The main weakness and its mechanism

The weakness that decides the verdict is the dependence of INDI on its actuator model. The incremental law adds a torque increment to an estimate of the torque already applied, and that estimate depends on the modelled actuator time constant (`MC_INDI_ACT_TC`). The control-law review argues that a model slower than the plant amounts to a gain reduction and should be benign, which the post-acceptance sweep on the stock plant (factor 1.5-3, 3 seeds) is consistent with. The dangerous case is a plant slower than the model, which produces a band-pass boost of the loop near 1/(2 pi sqrt(tau_e tau_t)), with tau_e and tau_t the modelled and true time constants.

The experiments agree with this. In H5, doubling the motor time constant alone reproduces the limit cycle, inertia and mass errors alone do not, and the limit cycle vanishes when `MC_INDI_ACT_TC` is matched (`results/heldout_h5_s15.md`). A post-acceptance sweep (session 17, not part of the frozen result) of `MC_INDI_ACT_TC` from 0.33 to 3 times nominal on the stock plant gave no limit cycle, with roll-rate RMS between 0.129 and 0.161 (`results/act_tc_sweep_s17.md`). The ratio alone therefore does not explain H5: the limit cycle needs a physically slower actuator that the model does not know. The sensitivity is to an actuator slower than modelled; whether lower gains would buy tolerance to it was not evaluated.

10.4 Limits and future work

The conclusion rests on one simulated airframe and no hardware flight. The disturbance comparison is also conditional on neither controller having been tuned for disturbance rejection (section 9.1), and a different vehicle could change the ranking either way (section 9.4).

Section 9.5 lists what remains before real flight and is not repeated here. The first item follows directly from this report: decide how the actuator state is obtained on hardware, from ESC telemetry or from a measured and verified time constant, and test sensitivity to mismatch in both directions on more than one airframe. Identification of the control effectiveness and motor time constant on a real vehicle, SIH and hardware-in-the-loop tests, a decision on whether yaw should stay on PID, and a staged flight protocol with criteria fixed in advance complete the list.

On this evidence, INDI is worth pursuing where disturbance rejection matters and the actuator dynamics can be measured. It should not replace the PID rate loop as a default without a way to detect or bound the actuator-model error.

Appendix A Reproducing the results

Commands run from the repository root (`PX4-Indi/`) unless stated otherwise. Project files are under `indi_project/`.

A.1 Requirements

- **Docker** runs everything that builds or simulates: the image, the SITL firmware and the batch runs. The image is built from `indi_project/docker/Dockerfile` (Ubuntu 24.04, PX4 build dependencies, Gazebo Harmonic, MAVSDK, pytest, scipy) and is tagged `px4-indi-sim`.

- **bash** on the host runs the wrapper scripts (`docker/build.sh`, `docker/run.sh`, `bench/run_batch.sh`).
- The figure step (A.6) is the one place that needs host Python packages: `make_figures.py` needs numpy, matplotlib and pyulog. `summarize_seeds.py` uses only the standard library.

A.2 Build the image and the firmware

Build the image (uses the host user's uid and gid, so files written into the bind-mounted repository stay owned by the user):

```
indi_project/docker/build.sh
```

Build the SITL firmware inside the image:

```
indi_project/docker/run.sh -n build -t 3000 'make px4_sitl_default'
```

This command is the one recorded in `indi_project/plan/integration_log.md`; it is not wrapped in a script (`docker/run.sh` options `-n` container name, `-t` timeout in seconds). It produces `build/px4_sitl_default`, the directory `bench/run_scenario.py` uses. Nothing is flashed to hardware.

A.3 Run one scenario for one controller and seed

```
indi_project/docker/run.sh -n step1 -t 600 -N -- python3 indi_project/bench/run_scenario.py \
    --scenario step_roll --controller pid_default --seed 1 --speed 4 \
    --out indi_project/results/runs/step_roll/pid_default/seed1_x4
```

- `-N` gives the run its own loopback-only network namespace; `-t` is the wall-clock timeout in seconds.
- `--scenario` takes a key of `bench/scenarios.py` (for example `hover`, `step_roll`, `gust_x`). `--controller` is `pid_default` or `indi_tuned`. `--seed` (default 1) seeds the Gazebo sensor noise. `--speed` (default 1.0) is the simulation speed factor. `--params FILE` applies `NAME VALUE` lines before arming. `--out` is required.
- Output in `--out`: `flight.ulg` (the newest ULog of the run), `run.json` (scenario window, seeds and event records), `px4.log`, `gz.log` and `runner.log`.
- `run_scenario.py` does not write `metrics.json`. Create it with:

```
python3 indi_project/bench/metrics.py <out>/flight.ulg
```

This writes `<out>/metrics.json`, reading `<out>/run.json` for the evaluation window.

A.4 Run a batch over seeds

```
SCENARIOS="step_roll" CONTROLLERS="pid_default" SEEDS="1 2 3 4 5 6 7 8 9 10" \
SPEEDS="4" TIMEOUT=600 JOBS=4 indi_project/bench/run_batch.sh
```

Environment variables, with their defaults: `SCENARIOS` ("step_roll hover"), `CONTROLLERS` ("pid_default"), `SEEDS` ("1"), `SPEEDS` ("1 4"), `TIMEOUT` (600 s per run), `JOBS` (1 run at a time), `PARAMS_FILE` (overlay file, absolute path), `RESULTS_DIR` (default `indi_project/results/runs`), `FLAT_LAYOUT=1` (drops the controller directory), `NAME_TAG` (suffix for container names).

- Output: `indi_project/results/runs/<scenario>/<controller>/seed<N>[_x<speed>]/`, with the A.3 files plus `metrics.json`, which the batch writes itself.
- **Speed rule.** Attitude and rate scenarios (`mode` "attitude" in `bench/scenarios.py`) run at speed 4; position scenarios (`mode` "position", such as `hover` and the trajectory scenarios) run at speed 1. This is decision D-speed in `indi_project/STATE.md`: hover position RMS differs between speeds (0.027 m vs 0.064 m), attitude and rate metrics do not. The script does not apply the rule, and its default is `SPEEDS="1 4"`, so set `SPEEDS` per batch.

- **Controller overlay.** `bench/params/<controller>.params` is applied automatically if it exists. There is no `indi_tuned.params`; the frozen INDI candidate is set with `PARAMS_FILE`, which applies to every controller in that call. Run `indi_tuned` in its own call:

```
PARAMS_FILE="$PWD/indi_project/results/tuning/indi/r4f/c00/params" CONTROLLERS=indi_tuned \
SCENARIOS="step_roll" SEEDS="1 2 3 4 5 6 7 8 9 10" SPEEDS="4" indi_project/bench/run_batch.sh
```

The frozen candidate is `results/tuning/indi/r4f/c00/params` (`NAME  VALUE` lines; `DECISIONS.md`, D-freeze). Every `indi_tuned` run records this path and the applied values in its `run.json` (`params_file`, `params_applied`), which is the check that a run used the frozen set. The JSON files in `bench/params/` are tuning-space centres for `bench/tune.py`, not overlays. - **Parallel batches.** Container names are `px4-indi-<scenario>-<seed><NAME_TAG>`, so give each concurrent batch its own `NAME_TAG`. Each run has its own network namespace, so port clashes do not occur, and each run has its own working directory. - `run_batch.sh` does not skip finished runs. Re-running a batch re-flies and overwrites its output.

A.5 Summaries and statistics

```
python3 indi_project/bench/summarize_seeds.py indi_project/results/runs step_roll \
    --a pid_default --b indi_tuned
```

The first argument is the runs directory, followed by one or more scenarios. `--a` (default `pid_default`) and `--b` (default `indi_tuned`) choose the two controllers. For each scenario the script prints a markdown table of mean ± standard deviation over seeds per metric, the ratio mean(b)/mean(a), the number of paired seeds where b is lower, the median improvement, and the exact two-sided Wilcoxon signed-rank p-value on the paired differences (the acceptance test in `bench/scenarios.md`, section 6).

A.6 Figures

From the repository root, with plain `python3` (not `python3 -I`, because matplotlib and pyulog live in the user site-packages):

```
python3 indi_project/bench/make_figures.py
```

The script reads `results/runs/` and the reference tables in `indi_project/results/*.md`, and writes nine PNGs at 150 dpi into `indi_project/report/figures/`: `fig_attitude_ratio.png`, `fig_effort_ratio.png`, `fig_act_tc_sweep.png`, `fig_rate_timeseries.png`, and the seed-1 time series `fig_disturbance_onset.png`, `fig_t8_fault.png`, `fig_step_responses.png`, `fig_lemniscate.png` and `fig_h4_crash.png` (these five read the ULogs under `results/runs/`, which are not committed). It prints the plotted numbers and flags any difference above 2 % from the reference tables as `DISCREPANCY`. Figure ratios are medians of per-seed paired ratios, so they can differ from the ratios of means in the tables (`report/figures/README.md`). The reference tables are committed under `indi_project/results/`.

A.7 Known gaps

- The T8 10-seed table is `python3 indi_project/bench/summarize_seeds.py indi_project/results/runs motor_loss_0` plus the `motor_fault` block of each `metrics.json` (`results/t8_motor_loss_10seeds_s26.m`
- The trajectory tables and the H4 table were produced by inline scripts that are not yet in `bench/`. Their numbers cannot be regenerated from the repository until those scripts are added.
- Seeding is partial. `--seed` controls the Gazebo sensor noise (`gz sim --seed`). The baro simulation in the PX4 SITL sensor module (`SensorBaroSim`) calls `srand(1234)` and is not seed-controlled. `run.json` records both values (`noise_seed`).
- The SITL firmware build command in A.2 is recorded in a log, not wrapped in a script.

Bibliography

Every source below was opened and read for this work; a note written from its text is kept under `indi_project/research/notes/`. Citation keys in the chapters (e.g. [smeur2016]) refer to the entries below.

Published works

- **[acquatella2017]** P. Acquatella B., W. van Ekeren, Q.P. Chu, "PI(D) tuning for Flight Control Systems via Incremental Nonlinear Dynamic Inversion", arXiv:1701.08981v3, 2017. https://arxiv.org/abs/1701.08981. Reading note: `research/notes/acquatella2017.md`.
- **[blaha2024]** T.M. Blaha, E.J.J. Smeur, B.D.W. Remes, "Control of Unknown Quadrotors from a Single Throw", IROS 2024, arXiv:2406.11723v3. https://arxiv.org/abs/2406.11723. Reading note: `research/notes/blaha2024.md`.
- **[smeur2016]** E.J.J. Smeur, Q.P. Chu, G.C.H.E. de Croon, "Adaptive Incremental Nonlinear Dynamic Inversion for Attitude Control of Micro Air Vehicles", J. Guidance, Control, and Dynamics 39(3), 2016, doi:10.2514/1.G001490. https://doi.org/10.2514/1.G001490. Reading note: `research/notes/smeur2016.md`.
- **[smeur2018]** E.J.J. Smeur, G.C.H.E. de Croon, Q.P. Chu, "Cascaded Incremental Nonlinear Dynamic Inversion for MAV Disturbance Rejection", Control Engineering Practice, 2018, arXiv:1701.07254. https://arxiv.org/abs/1701.07254. Reading note: `research/notes/smeur2018.md`.
- **[steffensen2023]** R. Steffensen, A. Steinert, E.J.J. Smeur, "Nonlinear Dynamic Inversion with Actuator Dynamics: an Incremental Control Perspective", J. Guidance, Control, and Dynamics, 2023. https://arxiv.org/abs/2201.09805 (copy read: arXiv v2, 25 Nov 2022, `raw/robustness/steffensen_arxiv.pdf`). Reading note: `research/notes/steffensen2023.md`.
- **[tal2021]** E. Tal, S. Karaman, "Accurate Tracking of Aggressive Quadrotor Trajectories using Incremental Nonlinear Dynamic Inversion and Differential Flatness", IEEE Trans. Control Syst. Technol., 2021 (venue per task brief, not confirmed in text read); arXiv:1809.04048v2. https://arxiv.org/pdf/1809.04048. Reading note: `research/notes/tal2021.md`.
- **[wang2019]** X. Wang, E.-J. van Kampen, Q.P. Chu, P. Lu, "Stability Analysis for Incremental Nonlinear Dynamic Inversion Control", J. Guidance, Control, and Dynamics 42(5), 2019, doi:10.2514/1.G003791. https://doi.org/10.2514/1.G003791 (copy read: TU Delft repository green open-access PDF, `raw/robustness/wang_pure.pdf`). Reading note: `research/notes/wang2019.md`.

Open-source implementations and prior PX4 work

- **[impl_paparazzi]** Paparazzi UAV, `sw/airborne/firmwares/rotorcraft/stabilization/stabilization_`
commit c04ffe40a035b997dc71e7d7965cbddb07606695. https://github.com/paparazzi/paparazzi. Reading note: `research/notes/impl_paparazzi.md`.
- **[impl_px4_prior]** PX4 PRs #25216 (closed WIP MC INDI), #25166 (FW iNDI), #25863/#25864, forum threads 10953/36293, and lei-ss/indi (Paparazzi simple INDI ported to PX4). https://github.com/PX4/PX4-Autopilot/pull/25216 , https://github.com/lei-ss/indi. Reading note: `research/notes/impl_px4_prior.md`.

Project-internal documents

- **[review_s16]** Independent review of the INDI control law and its PX4 integration, carried out during this project (sessions 16-21), with findings triaged into fixes and report limitations. `indi_project/review/control_law_review_s16.md`.